



**TRƯỜNG ĐẠI HỌC QUẢN LÝ VÀ CÔNG NGHỆ
THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ**

KHÓA LUẬN TỐT NGHIỆP

CHRONIX: HỆ THỐNG LẬP LỊCH CÁ NHÂN HOÁ TỐI ƯU NĂNG SUẤT DỰA TRÊN AI VÀ NHỊP ĐIỆU SINH HỌC

Họ và tên sinh viên : Nguyễn Ngọc Thạch

Mã số sinh viên : 2201700077

Ngành đào tạo : Công nghệ thông tin

Khóa : 2022 – 2026

Giảng viên hướng dẫn : ThS. Nguyễn Quân Bá Hồng

Thành phố Hồ Chí Minh, tháng 4 năm 2026



**TRƯỜNG ĐẠI HỌC QUẢN LÝ VÀ CÔNG NGHỆ
THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ**

KHÓA LUẬN TỐT NGHIỆP

CHRONIX: HỆ THỐNG LẬP LỊCH CÁ NHÂN HOÁ TỐI ƯU NĂNG SUẤT DỰA TRÊN AI VÀ NHỊP ĐIỆU SINH HỌC

Họ và tên sinh viên : Nguyễn Ngọc Thạch

Mã số sinh viên : 2201700077

Ngành đào tạo : Công nghệ thông tin

Khóa : 2022 – 2026

Giảng viên hướng dẫn : ThS. Nguyễn Quân Bá Hồng

Thành phố Hồ Chí Minh, tháng 4 năm 2026

LỜI CAM ĐOAN

Tôi, Nguyễn Ngọc Thạch, xin cam đoan rằng Khóa luận tốt nghiệp với đề tài “Chronix: Hệ thống lập lịch cá nhân hoá tối ưu năng suất dựa trên AI và nhịp điệu sinh học” và các kết quả trình bày trong khóa luận này là do tôi tự thực hiện. Tôi xác nhận rằng:

- Khóa luận này được thực hiện hoàn toàn trong quá trình học tập và nghiên cứu tại Trường Đại học Quản lý và Công nghệ Thành phố Hồ Chí Minh.
- Không có phần nào trong khóa luận này đã được nộp để lấy bằng cấp hay chứng chỉ tại bất kỳ cơ sở đào tạo nào khác.
- Khi tham khảo các công trình nghiên cứu đã công bố của người khác, tôi đều trích dẫn rõ ràng và đầy đủ nguồn gốc.
- Khi trích dẫn trực tiếp từ tài liệu của người khác, nguồn gốc luôn được ghi rõ. Ngoài các trích dẫn đó, khóa luận này hoàn toàn là công trình của riêng tôi.
- Tôi đã ghi nhận tất cả các nguồn hỗ trợ chính trong quá trình thực hiện khóa luận.

TP. Hồ Chí Minh, ngày tháng năm 2026

Tác giả

LỜI CẢM ƠN

Trước tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến ThS. Nguyễn Quân Bá Hồng, giảng viên Khoa Công Nghệ, Trường Đại học Quản lý và Công nghệ Thành phố Hồ Chí Minh, người đã trực tiếp hướng dẫn em trong suốt quá trình thực hiện khóa luận tốt nghiệp. Thầy đã tận tình chỉ dẫn, đặt những câu hỏi đúng lúc, và luôn sẵn sàng hỗ trợ em vượt qua các khó khăn trong quá trình nghiên cứu và phát triển hệ thống.

Con xin cảm ơn ba mẹ và gia đình đã luôn ủng hộ, động viên, và tạo mọi điều kiện tốt nhất để con có thể tập trung học tập và hoàn thành khóa luận. Sự quan tâm và tin tưởng của gia đình là nguồn động lực lớn nhất giúp con vượt qua những giai đoạn khó khăn.

Tôi cũng xin gửi lời cảm ơn đến các bạn bè đã luôn bên cạnh, chia sẻ kinh nghiệm, và tiếp thêm động lực trong suốt quá trình học tập và thực hiện đề tài.

Cuối cùng, tôi muốn cảm ơn chính bản thân mình vì đã không bỏ cuộc, kiên trì theo đuổi đề tài từ ý tưởng ban đầu đến sản phẩm hoàn chỉnh, và luôn nỗ lực hết mình để đạt được kết quả tốt nhất có thể.

TRƯỜNG ĐẠI HỌC QUẢN LÝ VÀ
CÔNG NGHỆ THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ

PHIẾU CHẤM ĐIỂM
KHÓA LUẬN TỐT NGHIỆP

Họ tên người chấm điểm:

Giảng viên hướng dẫn: ☒

Giảng viên phản biện: ☐

Sinh viên (SV) thực hiện (Số SV trong nhóm: 1)

1. SV1: Nguyễn Ngọc Thạch MSSV: 2201700077

Lớp: IT2207A

Ngành: Công nghệ thông tin

Tên đề tài: Chronix: Hệ thống lập lịch cá nhân hoá tối ưu năng suất dựa trên AI và nhịp điệu sinh học

Nhận xét

1. Hình thức:

.....
.....
.....

2. Nội dung:

.....
.....
.....
.....
.....

Kết quả đánh giá	
Điểm hình thức:	 / 10
Điểm nội dung:	 / 10
Điểm tổng kết:	 / 10

Ghi chú: Điểm tổng kết được làm tròn đến 0,5.

Đồng ý cho sinh viên báo cáo:

Có ☐

Không ☐

TP. Hồ Chí Minh, ngày tháng năm 2026

Người chấm điểm
(Ký và ghi rõ họ tên)

TRƯỜNG ĐẠI HỌC QUẢN LÝ VÀ
CÔNG NGHỆ THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ

PHIẾU CHẤM ĐIỂM
KHÓA LUẬN TỐT NGHIỆP

Họ tên người chấm điểm:

Giảng viên hướng dẫn: ☐

Giảng viên phản biện: ☒

Sinh viên (SV) thực hiện (Số SV trong nhóm: 1)

1. SV1: Nguyễn Ngọc Thạch MSSV: 2201700077

Lớp: IT2207A

Ngành: Công nghệ thông tin

Tên đề tài: Chronix: Hệ thống lập lịch cá nhân hoá tối ưu năng suất dựa trên AI và nhịp điệu sinh học

Nhận xét

1. Hình thức:

.....
.....
.....

2. Nội dung:

.....
.....
.....
.....
.....

Kết quả đánh giá	
Điểm hình thức:	 / 10
Điểm nội dung:	 / 10
Điểm tổng kết:	 / 10

Ghi chú: Điểm tổng kết được làm tròn đến 0,5.

Đồng ý cho sinh viên báo cáo:

Có ☐

Không ☐

TP. Hồ Chí Minh, ngày tháng năm 2026

Người chấm điểm
(Ký và ghi rõ họ tên)

MỤC LỤC

DANH MỤC CÁC CHỮ VIẾT TẮT	vi
1 Giới thiệu	2
1.1 Đặt vấn đề	2
1.2 Mục tiêu đề tài	3
1.3 Đối tượng và phạm vi nghiên cứu	4
1.3.1 Đối tượng nghiên cứu	4
1.3.2 Phạm vi nghiên cứu	5
1.4 Phương pháp nghiên cứu	6
1.5 Cấu trúc luận văn	6
2 Tổng quan lý thuyết	8
2.1 Chronobiology và chronotype	8
2.1.1 Nhịp sinh học (Circadian Rhythm)	8
2.1.2 Mô hình 4 chronotype của Breus	9
2.1.3 Đường cong năng lượng (Energy Curve)	9
2.1.4 Morningness-Eveningness Questionnaire (MEQ)	10
2.2 Các phương pháp lập lịch công việc	10
2.2.1 Lập lịch đơn yếu tố	10
2.2.2 Lập lịch đa yếu tố (Multi-factor Scheduling)	11
2.2.3 Thuật toán tham lam (Greedy Algorithm)	12
2.2.4 Yêu cầu năng lượng theo loại công việc	12
2.3 Chi phí chuyển đổi ngữ cảnh (Context Switching)	13
2.4 Tỷ lệ làm việc và nghỉ ngơi	13
2.5 Exponential Moving Average (EMA)	14
2.5.1 Khái niệm và công thức	14
2.5.2 Ứng dụng trong học thích ứng	14
2.5.3 Tín hiệu phản hồi ngẫu nhiên	15
2.6 Các công trình liên quan	18
2.6.1 Nghiên cứu về ảnh hưởng của chronotype đến hiệu suất	18
2.6.2 Hệ thống gợi ý thích ứng (Adaptive Recommender Systems)	18
2.6.3 Tối ưu hóa lập lịch đa mục tiêu (Multi-objective Scheduling)	19

2.6.4	So sánh với các ứng dụng hiện có	19
2.7	Tóm tắt	20
3	Thiết kế hệ thống	22
3.1	Kiến trúc tổng thể	22
3.1.1	Mô hình kiến trúc	22
3.1.2	Tổ chức mã nguồn theo module	23
3.1.3	Luồng dữ liệu chính	24
3.2	Thiết kế xác thực và onboarding	24
3.2.1	Xác thực với Better Auth và Google OAuth	24
3.2.2	Quy trình onboarding	24
3.3	Thiết kế cơ sở dữ liệu	25
3.3.1	Tổng quan	25
3.3.2	Bảng users	26
3.3.3	Bảng events	27
3.3.4	Bảng tasks	28
3.3.5	Bảng weightAdjustments	29
3.3.6	Các bảng hỗ trợ	29
3.4	Thiết kế hàm chi phí 6 yếu tố	30
3.4.1	Tổng quan	30
3.4.2	Hình phạt năng lượng sinh học - Chronotype Penalty (P_{chrono})	31
3.4.3	Hình phạt rủi ro deadline - Deadline Risk ($P_{deadline}$)	31
3.4.4	Hình phạt độ ưu tiên - Priority Penalty ($P_{priority}$)	32
3.4.5	Hình phạt khoảng nghỉ - Buffer Penalty (P_{buffer})	32
3.4.6	Hình phạt chuyển đổi ngữ cảnh - Context Switch Penalty ($P_{context}$)	33
3.4.7	Hình phạt phân mảnh lịch trình - Fragmentation Penalty (P_{frag})	34
3.5	Thiết kế thuật toán lập lịch	34
3.5.1	Tìm khe thời gian khả dụng (Slot Finder)	34
3.5.2	Thuật toán tham lam (Greedy Scheduling)	35
3.5.3	Giải thích kết quả lập lịch	35
3.6	Hệ thống trọng số thích ứng 3 lớp	36
3.6.1	Tổng quan kiến trúc	36
3.6.2	Lớp 1: Hồ sơ thiết lập trước (Preset Profile)	37
3.6.3	Lớp 2: Điều chỉnh thủ công $\Delta w_i^{manual} (\pm 5\%)$	38
3.6.4	Lớp 3: Tự động điều chỉnh EMA $\Delta w_i^{auto} (\pm 15\%)$	38
3.7	Thiết kế trợ lý AI Chat	39
3.7.1	Kiến trúc đường ống xử lý (Pipeline)	39
3.7.2	Các công cụ hàm (Function Tools)	40
3.7.3	Giới hạn tốc độ (Rate Limiting)	40
3.8	Đồng bộ Google Calendar	41
3.9	Tóm tắt	42

4	Triển khai	43
4.1	Công nghệ sử dụng	43
4.2	Cấu trúc dự án	44
4.3	Triển khai Scheduling Engine (Bộ lập lịch)	45
4.3.1	Đường cong năng lượng (Energy Curves)	45
4.3.2	Hàm chi phí (Cost Function)	46
4.3.3	Bộ tìm khe thời gian (Slot Finder)	50
4.3.4	Luồng lập lịch đơn tác vụ (Single-Task Scheduling)	51
4.3.5	Hành động phía máy chủ: scheduleTask	53
4.4	Triển khai hệ thống trọng số thích ứng	53
4.4.1	Hồ sơ trọng số mặc định (Preset Profiles)	53
4.4.2	Tính trọng số cuối cùng	54
4.4.3	Bộ xử lý phản hồi (Feedback Processor)	55
4.4.4	Nguồn tín hiệu phản hồi	56
4.5	Triển khai giao diện AI Chat	57
4.5.1	Kiến trúc đường ống xử lý (Pipeline)	57
4.5.2	Câu lệnh hệ thống động (Dynamic System Prompt)	58
4.5.3	Định nghĩa 7 công cụ (Tool Definitions)	58
4.5.4	Giới hạn tần suất (Rate Limiting)	59
4.6	Triển khai giao diện người dùng	60
4.6.1	Bảng điều khiển (Dashboard)	60
4.6.2	Quy trình giới thiệu (Onboarding Flow)	62
4.6.3	Đánh giá sau hoàn thành (Post-Task Rating)	63
4.7	Triển khai đồng bộ Google Calendar	64
4.7.1	Tạo máy khách Google Calendar	64
4.7.2	Đồng bộ toàn phần (Initial Sync)	65
4.7.3	Đồng bộ gia tăng (Incremental Sync)	65
4.7.4	Đẩy sự kiện lên Google (Push Sync)	66
4.8	Bảo mật	66
4.9	Triển khai lên môi trường production	67
5	Đánh giá	68
5.1	Phương pháp đánh giá	68
5.1.1	Bộ sinh số ngẫu nhiên tắt định	68
5.2	Kiểm chuẩn chất lượng thuật toán	69
5.2.1	Thiết lập thí nghiệm	69
5.2.2	5 phương pháp so sánh	70
5.2.3	8 chỉ tiêu đánh giá (Evaluation Metrics)	71
5.2.4	Kết quả	72
5.2.5	Phân tích kết quả	73
5.2.6	Kết quả theo chronotype	73

5.3	Mô phỏng hội tụ EMA	74
5.3.1	Mục đích	74
5.3.2	Thiết lập thí nghiệm	74
5.3.3	5 hồ sơ hành vi	74
5.3.4	Phương pháp mô phỏng	75
5.3.5	Kết quả EMA	76
5.3.6	Phân tích kết quả EMA	76
5.4	Mối đe dọa tính hợp lệ (Threats to Validity)	77
5.4.1	Tính hợp lệ nội tại (Internal Validity)	77
5.4.2	Tính hợp lệ ngoại tại (External Validity)	78
5.4.3	Tính hợp lệ cấu trúc (Construct Validity)	78
5.5	Tính tái tạo (Reproducibility)	78
6	Kết luận	79
6.1	Tóm tắt đóng góp	79
6.1.1	Hàm chi phí đa yếu tố (Multi-Factor Cost Function)	79
6.1.2	Hệ thống trọng số thích ứng 3 lớp (3-Layer Adaptive Weight System)	80
6.1.3	Ứng dụng full-stack hoàn chỉnh	80
6.2	Hạn chế	81
6.3	Hướng phát triển tương lai	81
6.3.1	Ngắn hạn (3–6 tháng)	81
6.3.2	Trung hạn (6–12 tháng)	82
6.3.3	Dài hạn (12+ tháng)	82
6.4	Kết luận	82
A	Bài khảo sát chronotype trong CHRONIX	84
A.1	5 câu hỏi khảo sát	84
A.2	Thuật toán phân loại	85

DANH MỤC CÁC CHỮ VIẾT TẮT

Viết tắt	Nghĩa đầy đủ
AI	Artificial Intelligence (Trí tuệ nhân tạo)
API	Application Programming Interface (Giao diện lập trình ứng dụng)
BCR	Break Compliance Rate (Tỷ lệ tuân thủ nghỉ)
CRUD	Create, Read, Update, Delete (Tạo, Đọc, Cập nhật, Xóa)
CSS	Cascading Style Sheets (Bảng định kiểu xếp chồng)
DB	Database (Cơ sở dữ liệu)
DSR	Deadline Satisfaction Rate (Tỷ lệ đáp ứng deadline)
EMA	Exponential Moving Average (Trung bình trượt hàm mũ)
ERD	Entity Relationship Diagram (Sơ đồ thực thể quan hệ)
ETA	Energy-Task Alignment (Sự phù hợp năng lượng-công việc)
FCFS	First Come First Served (Đến trước phục vụ trước)
GVHD	Giảng viên hướng dẫn
GVPB	Giảng viên phản biện
KLTN	Khóa luận tốt nghiệp
MEQ	Morningness-Eveningness Questionnaire (Bảng câu hỏi sáng-tối)
OAuth	Open Authorization (Ủy quyền mở)
ORM	Object-Relational Mapping (Ánh xạ đối tượng-quan hệ)
PKCE	Proof Key for Code Exchange (Khóa chứng minh trao đổi mã)
PWU	Peak Window Utilization (Sử dụng cửa sổ đỉnh)
RLS	Row Level Security (Bảo mật cấp hàng)
SFI	Schedule Fragmentation Index (Chỉ số phân mảnh lịch)
TCS	Total Cost Score (Điểm chi phí tổng)
UI	User Interface (Giao diện người dùng)
UX	User Experience (Trải nghiệm người dùng)

DANH MỤC BẢNG

2.1	Đặc điểm 4 chronotype theo mô hình Breus	9
2.2	Yêu cầu năng lượng theo loại công việc	13
2.3	So sánh CHRONIX với các ứng dụng lập lịch hiện có	20
3.1	Danh sách các bảng trong cơ sở dữ liệu	26
3.2	Yêu cầu năng lượng theo loại công việc	31
3.3	Trọng số preset w_i^{preset} cho 5 hồ sơ lập lịch	37
3.4	Danh sách công cụ hàm trong trợ lý AI Chat	40
4.1	Công nghệ và thư viện chính	43
4.2	Cửa sổ đỉnh và suy giảm năng lượng theo chronotype	45
4.3	7 công cụ AI Chat	58
5.1	Phân bố loại công việc trong kịch bản tổng hợp	70
5.2	5 phương pháp lập lịch được so sánh	70
5.3	Kết quả kiểm chuẩn chất lượng thuật toán (trung bình trên 6.000 lượt)	72
5.4	Điểm composite theo chronotype	73
5.5	Kết quả mô phỏng EMA - so sánh chất lượng trước và sau thích ứng	76
A.1	Tóm tắt phân loại chronotype	85

DANH MỤC HÌNH

3.1	Kiến trúc tổng thể hệ thống CHRONIX	23
3.2	Hệ thống trọng số thích ứng 3 lớp	37
3.3	Giao diện modal Fine-Tune Scheduling với 6 thanh trượt điều chỉnh thủ công trọng số.	38
4.1	Giao diện bảng điều khiển chính với lịch tuần.	60
4.2	Khung chat AI với tính năng lịch sử hội thoại và phát trực tuyến phản hồi. . . .	61
4.3	Hộp thoại tạo nhanh công việc.	61
4.4	Bảng trọng số trên thanh bên.	62
4.5	Hộp thoại tinh chỉnh trọng số thủ công.	62
4.6	Giao diện đánh giá sau khi hoàn thành công việc với 4 mức cảm xúc.	64

DANH MỤC ĐOẠN MÃ

3.1	Cấu trúc bảng users	26
3.2	Cấu trúc bảng events	27
3.3	Cấu trúc bảng tasks	28
3.4	Cấu trúc bảng weightAdjustments (tóm tắt)	29
4.1	Cấu trúc thư mục chính	44
4.2	Cấu trúc đường cong năng lượng	45
4.3	Tính năng lượng trung bình cho khe thời gian	45
4.4	Giao diện hàm chi phí – đầu vào	46
4.5	Giao diện hàm chi phí – đầu ra	46
4.6	Tính tổng chi phí	47
4.7	Hình phạt năng lượng sinh học	47
4.8	Hình phạt deadline	48
4.9	Chi phí chuyển đổi ngữ cảnh	48
4.10	Hình phạt phân mảnh	49
4.11	Hình phạt khoảng nghỉ	49
4.12	Hình phạt độ ưu tiên	50
4.13	Giao diện bộ tìm khe thời gian	51
4.14	Luồng lập lịch đơn tác vụ	51
4.15	Tính thời lượng nghỉ theo tỷ lệ 52:17	52
4.16	Xử lý xung đột trong scheduleTask	53
4.17	5 bộ trọng số mặc định	53
4.18	Kết hợp 3 lớp trọng số	54
4.19	Cập nhật EMA nguyên tử bằng SQL	55
4.20	Kết hợp tín hiệu khi hoàn thành công việc	56
4.21	Ngữ cảnh trong câu lệnh hệ thống	58
4.22	Ví dụ hướng dẫn LLM phân biệt ngày và hạn chót	58
4.23	Bộ giới hạn tần suất	59
4.24	Xác định chronotype từ điểm số	63
4.25	Tự động làm mới token	64
4.26	Xử lý lỗi sync token hết hạn	65
5.1	Bộ sinh số ngẫu nhiên có hạt giống	68
5.2	Phương pháp Chronix trong benchmark	70

5.3	Chạy bộ kiểm chuẩn	78
-----	------------------------------	----

*“Ngày đi, tháng chạy, năm bay
Thời gian nước chảy chẳng quay được về.”*

– Ca dao

Chương 1

Giới thiệu

1.1 Đặt vấn đề

Trong môi trường làm việc và học tập hiện đại, việc quản lý thời gian hiệu quả đóng vai trò then chốt đối với năng suất cá nhân. Với lượng công việc ngày càng tăng, sự phức tạp trong việc phân bổ thời gian hợp lý trở thành một thách thức lớn đối với nhiều người. Các ứng dụng lịch và quản lý công việc phổ biến như Google Calendar, Todoist, hay Notion cung cấp nhiều tính năng tổ chức, phân loại, và nhắc nhở. Tuy nhiên, chúng đều có một hạn chế chung: **đối xử với mọi người dùng như nhau**, bỏ qua sự khác biệt cơ bản về nhịp sinh học (circadian rhythm) giữa các cá nhân.

Nghiên cứu về chronobiology (sinh học thời gian) đã chỉ ra rằng mỗi người có một chronotype (kiểu thời gian sinh học) riêng, là mô hình năng lượng sinh lý tự nhiên trong ngày (Breus, 2016; Roenneberg, 2012). Cụ thể, Tiến sĩ Michael Breus đã phân loại con người thành 4 nhóm chronotype chính: “Lion” (sư tử), “Bear” (gấu), “Wolf” (sói), và “Dolphin” (cá heo). Mỗi nhóm có thời điểm đạt đỉnh năng lượng khác nhau trong ngày. Ví dụ, người thuộc nhóm “Lion” đạt đỉnh năng lượng vào buổi sáng sớm (6h đến 10h), trong khi người “Wolf” hoạt động tốt nhất vào buổi tối (17h đến 21h). Việc sắp xếp công việc đòi hỏi tư duy cao (analytical tasks) vào thời điểm năng lượng thấp không chỉ giảm hiệu suất làm việc mà còn gây ra mệt mỏi, stress, và giảm chất lượng kết quả đầu ra.

Hiện tại, hầu hết các hệ thống lập lịch tự động trên thị trường chỉ dựa trên **một hoặc hai yếu tố đơn giản**: độ ưu tiên công việc hoặc deadline. Cách tiếp cận này dẫn đến nhiều vấn đề thực tế. Trước hết, công việc đòi hỏi tập trung cao được xếp vào buổi sáng sớm cho tất cả người dùng, bất kể họ thuộc nhóm “Wolf” hay “Lion”. Bên cạnh đó, không có cơ chế nào đánh giá sự phù hợp giữa mức năng lượng sinh học tại một thời điểm cụ thể và yêu cầu năng lượng của công việc được xếp vào thời điểm đó. Chi phí chuyển đổi ngữ cảnh (context switching) giữa các công việc khác loại cũng không được xem xét; ví dụ, chuyển từ viết code sang soạn email rồi quay lại viết code gây lãng phí thời gian và năng lượng đáng kể. Ngoài ra, thời gian nghỉ ngơi giữa các công việc liên tiếp không được đảm bảo, dẫn đến tình trạng quá tải và giảm hiệu suất dần theo thời gian. Cuối cùng, lịch trình được tạo ra là cố định, không có khả năng học hỏi từ phản hồi

của người dùng để cải thiện chất lượng sắp xếp theo thời gian.

Từ những hạn chế trên, bài toán đặt ra là: làm thế nào để xây dựng một hệ thống lập lịch có khả năng **cá nhân hóa** dựa trên đặc điểm sinh học của từng người dùng, đồng thời tối ưu hóa **đa yếu tố** và có khả năng **tự thích ứng** theo thời gian?

1.2 Mục tiêu đề tài

Khóa luận này hướng đến việc xây dựng CHRONIX, một ứng dụng web lập lịch công việc thông minh dựa trên chronotype cá nhân. Hệ thống được thiết kế để giải quyết các hạn chế đã nêu ở phần đặt vấn đề, với các mục tiêu cụ thể như sau.

Thứ nhất, hệ thống cần có khả năng **nhận diện và phân loại chronotype người dùng** (chronotype identification and classification) thông qua bài khảo sát trực tuyến dựa trên bảng câu hỏi AutoMEQ (Automated Morningness-Eveningness Questionnaire), một công cụ đánh giá chronotype đã được kiểm chứng trong nghiên cứu chronobiology. Từ kết quả khảo sát, hệ thống tự động phân loại người dùng vào 1 trong 4 nhóm chronotype: Lion, Bear, Wolf, và Dolphin, đồng thời xây dựng hồ sơ năng lượng cá nhân (personalized energy profile) dưới dạng đường cong năng lượng 24 giờ đặc trưng cho từng nhóm. Hồ sơ năng lượng này đóng vai trò là nền tảng dữ liệu sinh học (biological data foundation) cho toàn bộ quá trình tối ưu hóa lịch trình, cho phép hệ thống đưa ra quyết định lập lịch dựa trên bằng chứng khoa học (evidence-based scheduling) thay vì áp dụng một mô hình chung cho mọi người dùng.

Thứ hai, đề tài cần **thiết kế hàm chi phí đa mục tiêu đa yếu tố** (multi-objective multi-factor cost function) $f: \mathbb{R}^d \rightarrow \mathbb{R}$, với $d \geq 3$. Bên cạnh 2 yếu tố cơ bản mà hầu hết các hệ thống hiện tại sử dụng là độ ưu tiên công việc và deadline, hàm chi phí bổ sung thêm các yếu tố sâu sắc hơn dựa trên đặc trưng nhịp sinh học của người dùng: sự phù hợp năng lượng sinh học (chronotype alignment), chi phí chuyển đổi ngữ cảnh (context switching), mức độ phân mảnh lịch trình (fragmentation), và tuân thủ thời gian nghỉ (buffer compliance). Tính đa mục tiêu (multi-objective) của hàm chi phí thể hiện ở chỗ hệ thống không chỉ tối ưu hiệu suất công việc (maximize productivity), mà còn đồng thời đảm bảo cân bằng giữa công việc và cuộc sống, bảo vệ sức khỏe toàn diện của người dùng (minimize negative health effects). Việc xét bài toán đa mục tiêu này có sự cải thiện đáng kể so với các mô hình truyền thống chỉ tập trung vào tiến độ công việc mà bỏ qua sự hao tổn sức khỏe, một yếu tố gây thiệt hại nghiêm trọng đến kết quả công việc về lâu dài. Ở quy mô hiện tại của khóa luận tốt nghiệp đại học, đề tài trước mắt quan tâm đến 6 yếu tố đánh giá chất lượng ($d = 6$). Tuy nhiên, thông qua cách thiết kế thuật toán và cài đặt, mô hình này hoàn toàn có thể được mở rộng cho nhiều yếu tố hơn ($d > 6$) một cách tự nhiên và dễ dàng, cho thấy tính mở rộng (extendability) và khả năng nâng cấp (upgradability) của mô hình kết hợp giữa khía cạnh sinh học con người và AI được đề xuất trong khóa luận này là linh hoạt và mang nhiều tiềm năng phát triển trong tương lai. Mỗi yếu tố được tính toán độc lập và kết hợp thông qua hệ thống trọng số.

Thứ ba, đề tài **phát triển hệ thống trọng số thích ứng 3 lớp** (3-layer adaptive weight architecture). Lớp 1 là hồ sơ cài đặt sẵn (preset profile) phù hợp với từng nhóm chronotype. Lớp

2 cho phép người dùng điều chỉnh thủ công trong phạm vi $\pm 5\%$, đảm bảo tính linh hoạt cao, cho phép cá nhân hóa theo nhu cầu (highly user-customizable flexibility). Lớp 3 là hệ thống tự động điều chỉnh dựa trên thuật toán EMA (Exponential Moving Average) với phạm vi $\pm 15\%$, mang lại tính thích ứng (adaptability) cho hệ thống. Kiến trúc 3 lớp này cho phép hệ thống vừa hoạt động tốt ngay từ lần sử dụng đầu tiên mà không cần dữ liệu phản hồi, vừa liên tục cải thiện dần theo thói quen thực tế của người dùng thông qua cơ chế học ngầm định.

Thứ tư, hệ thống cần được xây dựng như một **hệ thống tích hợp AI** (AI-integrated system) với khả năng **đồng bộ lịch hai chiều** (bidirectional schedule synchronization). Về phía AI, hệ thống cung cấp giao diện trò chuyện thông minh (intelligent conversational interface) sử dụng mô hình ngôn ngữ lớn (Large Language Model - LLM) kết hợp với kỹ thuật gọi hàm (function calling), cho phép người dùng tạo, sửa đổi, và quản lý công việc bằng ngôn ngữ tự nhiên song ngữ Việt-Anh. Trợ lý AI không chỉ đóng vai trò giao diện nhập liệu mà còn là lớp trừu tượng hóa (abstraction layer) giúp người dùng tương tác với scheduling engine mà không cần hiểu cơ chế hoạt động bên dưới. Về phía đồng bộ lịch, hệ thống triển khai cơ chế đồng bộ hai chiều thời gian thực (real-time bidirectional sync) với Google Calendar, bao gồm đồng bộ ban đầu (initial sync), đồng bộ tăng dần (incremental sync) qua syncToken, và chiến lược local-first khi xảy ra sự kiện ngược lên Google Calendar, đảm bảo tính nhất quán dữ liệu (data consistency) và trải nghiệm liền mạch (seamless user experience) trên nhiều nền tảng.

Cuối cùng, đề tài thực hiện **đánh giá định lượng toàn diện** (comprehensive quantitative evaluation) nhằm chứng minh tính hiệu quả và tính vượt trội của thuật toán lập lịch đề xuất. Bộ benchmark được thiết kế theo phương pháp khoa học với quy mô 6.000 lượt chạy (3 kịch bản quy mô $\times$ 4 chronotype $\times$ 5 phương pháp $\times$ 100 seed), sử dụng bộ sinh số ngẫu nhiên có hạt giống (seeded PRNG) để đảm bảo tính tái lập (reproducibility) - một tiêu chuẩn quan trọng trong nghiên cứu thực nghiệm. Hệ thống được so sánh với 4 phương pháp lập lịch cơ sở (baseline) trên 8 chỉ tiêu đánh giá đa chiều (multi-dimensional evaluation metrics), bao gồm cả các chỉ tiêu về hiệu suất lập lịch lẫn chất lượng cuộc sống người dùng. Ngoài ra, đề tài còn kiểm chứng tính hội tụ (convergence) và tính ổn định (stability) của hệ thống trọng số thích ứng EMA thông qua việc mô phỏng 5 hồ sơ hành vi người dùng giả lập, khẳng định khả năng học và thích ứng dài hạn (long-term adaptive learning) của hệ thống.

1.3 Đối tượng và phạm vi nghiên cứu

1.3.1 Đối tượng nghiên cứu

Đề tài tập trung nghiên cứu các đối tượng sau:

- **Người dùng mục tiêu:** Người làm việc tri thức (knowledge workers) bao gồm lập trình viên, nhà thiết kế, nhà nghiên cứu, và sinh viên. Đây là những người có nhu cầu quản lý nhiều công việc đòi hỏi mức độ tập trung và sáng tạo khác nhau trong ngày.
- **Mô hình năng lượng sinh học:** Nghiên cứu và ứng dụng đường cong năng lượng (energy curve) theo từng chronotype, được xây dựng dựa trên nghiên cứu của Breus (2016) và

Roenneberg (2012). Mỗi chronotype có đường cong năng lượng riêng biệt, mô tả mức năng lượng theo từng giờ trong ngày.

- **Thuật toán tối ưu hóa lập lịch** (scheduling optimization algorithm): Nghiên cứu và phát triển thuật toán lập lịch dựa trên hàm chi phí đa mục tiêu đa yếu tố, sử dụng phương pháp tham lam (greedy algorithm) kết hợp các cơ chế heuristic cho việc xác định khe thời gian tối ưu (heuristic mechanisms for optimal time-slot selection), tập trung vào các yếu tố nhịp sinh học và năng suất (biological and productivity rhythms). Việc lựa chọn hướng tiếp cận heuristic thay vì phương pháp chính xác (exact method) như vét cạn (brute force) hay quy hoạch động (dynamic programming) là cần thiết, bởi vì các phương pháp chính xác này có độ phức tạp thuật toán cao về cả mặt thời gian lẫn bộ nhớ (time and space complexity), không phù hợp với yêu cầu phản hồi tức thời (real-time responsiveness) của người dùng trong bài toán lập lịch tương tác.
- **Hệ thống học thích ứng** (adaptive learning system): Nghiên cứu và áp dụng thuật toán EMA (Exponential Moving Average) để tự động điều chỉnh trọng số dựa trên phản hồi ngầm định của người dùng (implicit user feedback), cho phép hệ thống liên tục cải thiện chất lượng lập lịch theo thời gian mà không yêu cầu sự can thiệp chủ động từ người dùng.

1.3.2 Phạm vi nghiên cứu

Bao gồm:

- Xây dựng ứng dụng web đầy đủ từ giao diện người dùng (frontend) đến máy chủ (backend), triển khai trên nền tảng Next.js 15 với App Router.
- Thiết kế và triển khai cơ sở dữ liệu PostgreSQL với Drizzle ORM, bao gồm các bảng dữ liệu cho người dùng, công việc, lịch trình, và trọng số thích ứng.
- Tích hợp xác thực người dùng qua Google OAuth 2.0 với giao thức PKCE (Proof Key for Code Exchange).
- Đồng bộ hai chiều với Google Calendar, bao gồm đồng bộ ban đầu (initial sync) và đồng bộ tăng dần (incremental sync).
- Xây dựng giao diện chat AI sử dụng OpenRouter API để tạo và quản lý công việc bằng ngôn ngữ tự nhiên.
- Thiết kế bộ benchmark với 6.000 lượt chạy để đánh giá hiệu suất thuật toán.

Không bao gồm:

- Phát triển ứng dụng di động (mobile application) cho iOS hoặc Android.
- Thu thập dữ liệu sinh trắc học từ thiết bị đeo (wearable devices) như đồng hồ thông minh hay vòng đeo tay.
- Nghiên cứu lâm sàng hoặc thử nghiệm y khoa về chronotype trên đối tượng thực tế.
- Tối ưu hóa lịch trình cho nhóm làm việc (team scheduling), chỉ tập trung vào lập lịch cá nhân.
- Triển khai hệ thống trên môi trường production với khả năng phục vụ số lượng lớn người

dùng đồng thời.

1.4 Phương pháp nghiên cứu

Đề tài sử dụng kết hợp các phương pháp nghiên cứu sau:

1. **Nghiên cứu lý thuyết:** Tổng hợp và phân tích tài liệu khoa học về chronobiology, đặc biệt là các mô hình chronotype của Breus (2016) và Roenneberg (2012). Nghiên cứu các phương pháp tối ưu hóa lập lịch hiện có, bao gồm thuật toán tham lam, quy hoạch tuyến tính, và các heuristic phổ biến. Tìm hiểu về thuật toán EMA và ứng dụng trong bài toán học thích ứng.
2. **Thiết kế và phát triển phần mềm:** Xây dựng ứng dụng theo mô hình phát triển Agile với các vòng lặp ngắn (sprint). Sử dụng bộ công nghệ hiện đại bao gồm Next.js 15 (App Router, React Server Components), TypeScript, Tailwind CSS, shadcn/ui cho giao diện, PostgreSQL với Drizzle ORM cho lưu trữ dữ liệu, và OpenRouter API cho tích hợp AI.
3. **Đánh giá thực nghiệm:** Thiết kế và thực hiện bộ benchmark toàn diện trên 6.000 lượt chạy với dữ liệu tổng hợp (synthetic data). So sánh thuật toán CHRONIX với 4 phương pháp lập lịch cơ bản: Random, FCFS, Priority-only, và Deadline-only. Đánh giá trên 8 chỉ tiêu định lượng.
4. **Mô phỏng học thích ứng:** Kiểm chứng hệ thống trọng số tự động điều chỉnh (EMA) bằng cách mô phỏng 5 hồ sơ hành vi người dùng giả lập, mỗi hồ sơ được chạy qua 50 quan sát để đánh giá tốc độ hội tụ và độ ổn định.

1.5 Cấu trúc luận văn

Khóa luận được tổ chức thành 6 chương với nội dung như sau:

- **Chương 1: Giới thiệu** trình bày bối cảnh nghiên cứu, đặt vấn đề, xác định mục tiêu, đối tượng, phạm vi, và phương pháp nghiên cứu của đề tài.
- **Chương 2: Tổng quan lý thuyết** giới thiệu cơ sở khoa học về chronobiology và các mô hình chronotype, tổng hợp các công trình nghiên cứu liên quan, và trình bày các thuật toán lập lịch cùng phương pháp EMA.
- **Chương 3: Thiết kế hệ thống** mô tả kiến trúc tổng thể của hệ thống CHRONIX, bao gồm thiết kế cơ sở dữ liệu, hàm chi phí 6 yếu tố, hệ thống trọng số thích ứng 3 lớp, quy trình đồng bộ Google Calendar, và pipeline xử lý AI chat.
- **Chương 4: Triển khai** trình bày chi tiết quá trình cài đặt các module chính của hệ thống, bao gồm scheduling engine, hệ thống trọng số, giao diện người dùng, xác thực OAuth, và các biện pháp bảo mật.
- **Chương 5: Đánh giá** trình bày phương pháp benchmark, kết quả thực nghiệm so sánh các thuật toán lập lịch, phân tích hội tụ EMA, và thảo luận về tính hợp lệ của kết quả.

- **Chương 6: Kết luận** tóm tắt các đóng góp chính của đề tài, nêu các hạn chế còn tồn tại, và đề xuất hướng phát triển trong tương lai.

Chương 2

Tổng quan lý thuyết

Chương này trình bày các cơ sở lý thuyết và công trình nghiên cứu liên quan làm nền tảng cho việc thiết kế và phát triển hệ thống CHRONIX. Nội dung bao gồm kiến thức về chronobiology và chronotype, các phương pháp lập lịch công việc, thuật toán Exponential Moving Average (EMA), và phân tích các công trình nghiên cứu liên quan.

2.1 Chronobiology và chronotype

2.1.1 Nhịp sinh học (Circadian Rhythm)

Nhịp sinh học (circadian rhythm) là chu kỳ sinh lý tự nhiên kéo dài khoảng 24 giờ, được điều chỉnh bởi đồng hồ sinh học nội tại nằm trong nhân trên chéo thị giác (suprachiasmatic nucleus, viết tắt SCN) của não bộ (Roenneberg, 2012). Thuật ngữ “circadian” bắt nguồn từ tiếng Latin “circa” (khoảng) và “diem” (ngày), phản ánh bản chất chu kỳ xấp xỉ 24 giờ của hiện tượng này.

Nhịp sinh học ảnh hưởng trực tiếp đến nhiều khía cạnh quan trọng trong hoạt động hàng ngày của con người. Trước hết, nhịp sinh học điều chỉnh chu kỳ ngủ và thức (sleep-wake cycle), quyết định thời điểm cơ thể cảm thấy buồn ngủ và thời điểm tỉnh táo nhất. Bên cạnh đó, mức năng lượng và sự tỉnh táo trong ngày cũng biến đổi theo nhịp sinh học, với các đỉnh và đáy năng lượng xảy ra ở những thời điểm khác nhau tùy theo từng cá nhân. Nhịp sinh học còn ảnh hưởng đến khả năng tập trung, ra quyết định, và xử lý thông tin phức tạp. Về mặt sinh lý, nhiệt độ cơ thể và việc tiết các hormone quan trọng như cortisol (hormone tỉnh táo) và melatonin (hormone ngủ) đều tuân theo nhịp sinh học.

Nghiên cứu của Roenneberg và cộng sự (2012) đã chỉ ra rằng nhịp sinh học khác nhau đáng kể giữa các cá nhân. Sự khác biệt này phụ thuộc vào nhiều yếu tố, bao gồm yếu tố di truyền (chiếm vai trò chủ đạo), tuổi tác (thanh thiếu niên thường có xu hướng “cú đêm” hơn người trưởng thành), và các yếu tố môi trường như ánh sáng tự nhiên và múi giờ sinh sống. Chính sự khác biệt cá nhân này tạo ra cơ sở khoa học cho việc cá nhân hóa lịch trình làm việc.

2.1.2 Mô hình 4 chronotype của Breus

Chronotype là thuật ngữ dùng để mô tả xu hướng sinh học tự nhiên của một cá nhân về thời điểm ngủ và thức trong ngày. Tiến sĩ Michael Breus (2016), một chuyên gia về giấc ngủ tại Hoa Kỳ, đã phát triển mô hình phân loại con người thành 4 nhóm chronotype dựa trên đặc điểm năng lượng và thói quen sinh hoạt. Mô hình này mở rộng từ phân loại truyền thống 2 nhóm (sáng/tối) của Horne và Östberg (1976) thành 4 nhóm chi tiết hơn, mỗi nhóm được đặt tên theo một loài động vật có mô hình ngủ tương tự.

Bảng 2.1: Đặc điểm 4 chronotype theo mô hình Breus

Chronotype	Đỉnh năng lượng	Thời gian ngủ	Tỷ lệ dân số
Lion (Sư tử)	6:00 đến 10:00	22:00 đến 6:00	15 đến 20%
Bear (Gấu)	10:00 đến 14:00	23:00 đến 7:00	50 đến 55%
Wolf (Sói)	17:00 đến 21:00	01:00 đến 9:00	15 đến 20%
Dolphin (Cá heo)	10:00 đến 12:00	23:00 đến 7:00	khoảng 10%

Lion (Sư tử) là nhóm người dậy sớm tự nhiên, thường thức giấc trước 6 giờ sáng mà không cần báo thức. Họ đạt đỉnh năng lượng trong khoảng 6h đến 10h sáng, sau đó năng lượng giảm dần trong buổi chiều và tối. Nhóm này chiếm khoảng 15 đến 20% dân số và thường phù hợp với các công việc đòi hỏi sự tập trung cao vào buổi sáng.

Bear (Gấu) là nhóm phổ biến nhất, chiếm 50 đến 55% dân số. Lịch sinh hoạt của nhóm Bear gần với chu kỳ mặt trời, với đỉnh năng lượng từ 10h đến 14h. Họ có thể thích ứng tốt với lịch làm việc tiêu chuẩn 9h đến 17h, tuy nhiên vẫn có sự sụt giảm năng lượng đáng kể vào đầu giờ chiều (khoảng 14h đến 16h).

Wolf (Sói) là nhóm “cú đêm”, chiếm 15 đến 20% dân số. Họ gặp khó khăn khi phải thức dậy sớm và thường cảm thấy mệt mỏi vào buổi sáng. Đỉnh năng lượng của nhóm Wolf nằm trong khoảng 17h đến 21h, khiến họ hoạt động hiệu quả nhất vào buổi tối. Đây là nhóm bị ảnh hưởng nhiều nhất khi phải tuân theo lịch làm việc truyền thống.

Dolphin (Cá heo) là nhóm đặc biệt nhất, chiếm khoảng 10% dân số. Đặc điểm nổi bật của nhóm này là giấc ngủ không ổn định, thường xuyên bị gián đoạn. Đỉnh năng lượng của Dolphin tập trung trong khoảng 10h đến 12h, nhưng mức năng lượng tổng thể thường thấp hơn và biến động nhiều hơn so với ba nhóm còn lại.

2.1.3 Đường cong năng lượng (Energy Curve)

Để ứng dụng mô hình chronotype vào bài toán lập lịch, cần lượng hóa mức năng lượng theo từng giờ trong ngày cho mỗi nhóm chronotype. Đường cong năng lượng (energy curve) biểu diễn mức năng lượng trên thang điểm 0 đến 100 cho mỗi giờ trong 24 giờ, dựa trên dữ liệu tổng hợp từ các nghiên cứu về chronobiology (Breus, 2016; Roenneberg, 2012; Facer-Childs et al., 2019).

Ví dụ, đường cong năng lượng của nhóm Lion cho thấy mức năng lượng tăng nhanh từ 5h sáng (50 điểm) đến đỉnh 100 điểm lúc 8h, sau đó giảm dần xuống 40 điểm lúc 14h và tiếp tục giảm về 10 điểm vào cuối ngày. Ngược lại, đường cong của nhóm Wolf bắt đầu rất thấp vào buổi sáng (10 đến 20 điểm), tăng dần trong ngày và đạt đỉnh 100 điểm lúc 18h đến 19h.

Sự khác biệt rõ rệt giữa các đường cong năng lượng chứng minh rằng việc áp dụng một lịch trình chung cho tất cả mọi người là không hợp lý. Một công việc đòi hỏi tư duy cao (ví dụ: viết code, phân tích dữ liệu) nếu được xếp vào lúc 8h sáng sẽ rất phù hợp với người Lion (năng lượng 100 điểm) nhưng hoàn toàn không phù hợp với người Wolf (năng lượng chỉ 20 điểm).

2.1.4 Morningness-Eveningness Questionnaire (MEQ)

MEQ (Morningness-Eveningness Questionnaire) là công cụ đánh giá chronotype được sử dụng rộng rãi nhất trong nghiên cứu khoa học, được phát triển bởi Horne và Östberg (1976). Phiên bản gốc gồm 19 câu hỏi, đánh giá xu hướng sáng/tối của một cá nhân thông qua các câu hỏi về thói quen ngủ, thời điểm tỉnh táo nhất, và sự ưa thích hoạt động vào buổi sáng hoặc buổi tối.

Phiên bản rút gọn AutoMEQ (Automated Morningness-Eveningness Questionnaire) được phát triển để phù hợp hơn với ứng dụng trực tuyến, giảm số lượng câu hỏi nhưng vẫn đảm bảo độ tin cậy trong phân loại. Trong hệ thống CHRONIX, bài khảo sát AutoMEQ gồm 5 câu hỏi chính, bao gồm: thời gian thức dậy ưa thích, mức độ tỉnh táo vào buổi sáng, thời điểm hiệu suất cao nhất, thời điểm cảm thấy mệt mỏi vào buổi tối, và chất lượng giấc ngủ. Mỗi câu hỏi cho điểm từ 1 đến 5, với tổng điểm nằm trong khoảng 5 đến 23. Thuật toán phân loại sử dụng tổng điểm kết hợp với câu trả lời về chất lượng giấc ngủ để ánh xạ kết quả sang 4 nhóm chronotype của Breus.

2.2 Các phương pháp lập lịch công việc

2.2.1 Lập lịch đơn yếu tố

Lập lịch đơn yếu tố (single-factor scheduling) là các phương pháp lập lịch chỉ dựa trên một tiêu chí duy nhất để sắp xếp công việc. Đây là cách tiếp cận đơn giản và được sử dụng rộng rãi trong thực tế, tuy nhiên có nhiều hạn chế khi áp dụng cho bài toán lập lịch cá nhân.

Phương pháp **FCFS (First Come First Served)** sắp xếp công việc theo thứ tự nhận được, không xem xét bất kỳ thuộc tính nào của công việc như độ ưu tiên hay deadline. Phương pháp này có ưu điểm là đơn giản và công bằng, nhưng thường cho kết quả kém vì không tận dụng được thông tin về đặc điểm công việc.

Phương pháp **Priority Scheduling** sắp xếp công việc theo độ ưu tiên giảm dần, đảm bảo công việc quan trọng nhất được xử lý trước. Tuy nhiên, phương pháp này bỏ qua sự phù hợp về năng lượng, có thể dẫn đến việc công việc ưu tiên cao được xếp vào thời điểm năng lượng thấp, làm giảm chất lượng kết quả.

Phương pháp **EDF (Earliest Deadline First)** ưu tiên công việc có deadline sớm nhất, giúp giảm thiểu việc trễ deadline. Phương pháp này hiệu quả trong việc đảm bảo tính kịp thời nhưng hoàn toàn không quan tâm đến khả năng nhận thức và mức năng lượng của người dùng tại thời điểm thực hiện.

Phương pháp **Energy-based Scheduling** chỉ xét sự phù hợp giữa mức năng lượng và yêu cầu công việc, đảm bảo công việc khó được xếp vào thời điểm năng lượng cao. Tuy nhiên, phương pháp này bỏ qua deadline và độ ưu tiên, có thể dẫn đến việc trễ deadline hoặc bỏ qua công việc quan trọng.

Hạn chế chung của tất cả các phương pháp đơn yếu tố là **không có phương pháp nào có thể cân bằng đồng thời nhiều mục tiêu cạnh tranh** như năng lượng, deadline, chi phí chuyển đổi ngữ cảnh, và thời gian nghỉ. Trong thực tế, người dùng cần một hệ thống có khả năng xem xét tổng thể nhiều yếu tố để đưa ra lịch trình tối ưu nhất.

2.2.2 Lập lịch đa yếu tố (Multi-factor Scheduling)

Để khắc phục hạn chế của lập lịch đơn yếu tố, phương pháp lập lịch đa yếu tố sử dụng hàm chi phí (cost function) hoặc hàm mục tiêu (objective function) kết hợp nhiều yếu tố với trọng số khác nhau. Công thức tổng quát của hàm chi phí đa yếu tố có dạng:

$$C(s) = \sum_{i=1}^n w_i \cdot f_i(s), \quad \text{với} \quad \sum_{i=1}^n w_i = 1 \quad (2.1)$$

trong đó s là khe thời gian (time slot) ứng viên, w_i là trọng số của yếu tố thứ i ($w_i \geq 0$), và $f_i(s)$ là giá trị phạt (penalty) của yếu tố thứ i tại slot s , được chuẩn hóa về khoảng $[0, 1]$.

Nhận xét về khả năng mở rộng. Công thức (2.1) sử dụng dạng tuyến tính tách biệt (linearly separable), trong đó mỗi hàm phạt $f_i(s)$ chỉ đánh giá một yếu tố độc lập. Trong lý thuyết tối ưu đa mục tiêu (multi-objective optimization), một dạng tổng quát hơn có thể được xét:

$$C(s) = \sum_{i=1}^n w_i \cdot J_i(f_1, f_2, \dots, f_n) \quad (2.2)$$

trong đó mỗi J_i là một hàm chi phí thành phần phụ thuộc vào *tất cả* n yếu tố $f_1, \dots, f_n$, cho phép mô hình hóa các tương tác chéo (cross-factor interactions) giữa các yếu tố. Ví dụ, mức phạt chuyển đổi ngữ cảnh có thể phụ thuộc đồng thời vào cả mức năng lượng hiện tại và thời gian còn lại trước deadline. Tuy nhiên, việc tối ưu hàm dạng (2.2) đòi hỏi các công cụ toán học phức tạp hơn, bao gồm tính gradient (gradient computation) và các phương pháp tối ưu dựa trên gradient (gradient-based optimization methods), vượt quá khuôn khổ của khóa luận tốt nghiệp đại học. Trong phạm vi đề tài này, dạng tuyến tính tách biệt (2.1) được lựa chọn vì tính đơn giản, dễ triển khai, và khả năng giải thích cao (high interpretability), khi mỗi thành phần chi phí có thể được phân tích và hiểu độc lập. Đồng thời, kiến trúc hệ thống được thiết kế mở để trong tương lai, nếu đề tài được phát triển tiếp, có thể nâng cấp lên dạng hàm chi phí đa mục tiêu tổng quát (2.2) một cách tự nhiên mà không cần thay đổi kiến trúc tổng thể.

Ưu điểm chính của phương pháp này là cho phép cân bằng linh hoạt giữa các mục tiêu cạnh tranh. Bằng cách điều chỉnh trọng số w_i , hệ thống có thể ưu tiên các yếu tố khác nhau tùy theo nhu cầu của từng người dùng. Ví dụ, một người làm freelance có thể muốn ưu tiên sự phù hợp năng lượng, trong khi một sinh viên có nhiều deadline có thể cần ưu tiên yếu tố thời hạn.

Thách thức chính của phương pháp đa yếu tố là **xác định trọng số phù hợp** cho từng người dùng. Trọng số tĩnh (không thay đổi) có thể không phản ánh đúng nhu cầu thực tế, đặc biệt khi thói quen và ưu tiên của người dùng thay đổi theo thời gian. Đây là lý do CHRONIX phát triển hệ thống trọng số thích ứng 3 lớp, được trình bày chi tiết trong Chương 3.

2.2.3 Thuật toán tham lam (Greedy Algorithm)

Thuật toán tham lam (greedy algorithm) là phương pháp tối ưu hóa đưa ra lựa chọn tối ưu cục bộ tại mỗi bước, với kỳ vọng đạt được kết quả tốt toàn cục. Trong bài toán lập lịch, thuật toán tham lam được sử dụng phổ biến do tính hiệu quả về thời gian tính toán, đặc biệt phù hợp với ứng dụng thời gian thực cần phản hồi nhanh cho người dùng.

Quy trình hoạt động của thuật toán tham lam trong bài toán lập lịch gồm 4 bước chính. Đầu tiên, thuật toán sắp xếp các công việc theo thứ tự ưu tiên (priority giảm dần, deadline tăng dần) để đảm bảo công việc quan trọng nhất được chọn slot trước. Tiếp theo, với mỗi công việc, thuật toán tìm tất cả các khe thời gian khả dụng trong cửa sổ tìm kiếm. Sau đó, thuật toán tính chi phí cho mỗi khe thời gian bằng hàm chi phí đa yếu tố và chọn khe có chi phí thấp nhất. Cuối cùng, khe thời gian được đánh dấu đã sử dụng và thời gian nghỉ tương ứng được chặn trước khi chuyển sang công việc tiếp theo.

Độ phức tạp thời gian của thuật toán là $O(n \cdot m)$ với n là số công việc và m là số khe thời gian khả dụng. Mặc dù không đảm bảo tối ưu toàn cục (global optimum), thuật toán tham lam cho kết quả tốt trong thực tế với thời gian tính toán chấp nhận được, phù hợp với yêu cầu phản hồi nhanh của ứng dụng web.

So với các phương pháp tối ưu hóa phức tạp hơn như quy hoạch tuyến tính (linear programming) hay các thuật toán meta-heuristic (genetic algorithm, simulated annealing), thuật toán tham lam có ưu điểm về tốc độ và đơn giản trong triển khai. Đổi lại, kết quả có thể không phải tối ưu tuyệt đối, nhưng sự khác biệt này thường không đáng kể trong bối cảnh lập lịch cá nhân với số lượng công việc vừa phải (dưới 50 công việc mỗi tuần).

2.2.4 Yêu cầu năng lượng theo loại công việc

Để hàm chi phí có thể đánh giá sự phù hợp giữa năng lượng người dùng và yêu cầu công việc, cần định lượng mức năng lượng cần thiết cho từng loại công việc. Dựa trên nghiên cứu về tải nhận thức (cognitive load) (Sweller et al., 2011), các loại công việc được phân loại theo mức năng lượng yêu cầu trên thang 0 đến 100 như trong Bảng 2.2.

Bảng 2.2: Yêu cầu năng lượng theo loại công việc

Loại công việc	Năng lượng	Ví dụ
Analytical (Phân tích)	90	Viết code, debug, phân tích dữ liệu, giải toán
Learning (Học tập)	80	Đọc tài liệu, học kỹ năng mới, nghiên cứu
Creative (Sáng tạo)	70	Thiết kế giao diện, viết bài, brainstorm ý tưởng
Physical (Thể chất)	60	Họp trực tiếp, thuyết trình, tập thể dục
Administrative (Hành chính)	40	Soạn email, sắp xếp tài liệu, cập nhật báo cáo

Công việc loại Analytical đòi hỏi mức năng lượng cao nhất (90 điểm) vì cần sự tập trung sâu và tư duy logic. Ngược lại, công việc Administrative chỉ cần 40 điểm vì có tính lặp lại và không đòi hỏi tư duy phức tạp. Sự phân loại này cho phép hệ thống tự động sắp xếp công việc khó vào thời điểm năng lượng cao và công việc nhẹ vào thời điểm năng lượng thấp.

2.3 Chi phí chuyển đổi ngữ cảnh (Context Switching)

Chuyển đổi ngữ cảnh (context switching) xảy ra khi một người chuyển từ công việc này sang công việc khác thuộc loại khác nhau. Nghiên cứu của Ophir và cộng sự (2009) cho thấy thời gian trung bình để phục hồi hoàn toàn sự tập trung sau khi chuyển đổi ngữ cảnh là khoảng **23 phút**. Điều này có nghĩa là nếu một người liên tục chuyển đổi giữa các loại công việc khác nhau (ví dụ: viết code, soạn email, viết code, họp), thời gian thực sự hiệu quả bị giảm đáng kể.

Trong bài toán lập lịch, chi phí chuyển đổi ngữ cảnh được tính dựa trên khoảng cách thời gian giữa hai công việc liên tiếp khác loại. Nếu khoảng cách đủ lớn (ít nhất 23 phút), người dùng có đủ thời gian phục hồi và chi phí chuyển đổi gần bằng 0. Nếu khoảng cách nhỏ hơn 23 phút, chi phí tăng tỷ lệ nghịch với thời gian phục hồi. Đặc biệt, nếu hai công việc cùng loại (ví dụ: hai công việc Analytical liên tiếp), chi phí chuyển đổi bằng 0 vì không cần thay đổi ngữ cảnh tư duy.

2.4 Tỷ lệ làm việc và nghỉ ngơi

Nghiên cứu về năng suất cho thấy việc xen kẽ thời gian làm việc tập trung và nghỉ ngơi ngắn giúp duy trì hiệu suất cao trong suốt ngày làm việc. Phương pháp Pomodoro (25 phút làm, 5 phút nghỉ) là một trong những kỹ thuật phổ biến nhất. Tuy nhiên, nghiên cứu gần đây của DeskTime cho thấy tỷ lệ **52 phút làm việc : 17 phút nghỉ** cho hiệu suất cao hơn 23% so với Pomodoro truyền thống.

Trong hệ thống CHRONIX, thời gian nghỉ được tính tự động theo công thức:

$$T_{break} = \text{clamp} \left(\text{round} \left(T_{work} \times \frac{17}{52} \right), 5, 20 \right) \quad (2.3)$$

trong đó T_{work} là thời gian làm việc (phút), và hàm clamp giới hạn thời gian nghỉ trong khoảng 5 đến 20 phút. Khoảng nghỉ lý tưởng giữa hai công việc liên tiếp là 15 phút, và khoảng nghỉ tối thiểu chấp nhận được là 5 phút. Nếu khoảng nghỉ nhỏ hơn 5 phút, hệ thống coi đây là vi phạm nghiêm trọng và áp dụng hình phạt tối đa.

2.5 Exponential Moving Average (EMA)

2.5.1 Khái niệm và công thức

Exponential Moving Average (EMA), hay trung bình trượt hàm mũ, là phương pháp tính trung bình có trọng số trong đó các quan sát gần đây có ảnh hưởng lớn hơn các quan sát cũ. Công thức cơ bản của EMA là:

$$S_t = \alpha \cdot x_t + (1 - \alpha) \cdot S_{t-1} \quad (2.4)$$

trong đó S_t là giá trị EMA tại thời điểm t , x_t là giá trị quan sát mới, S_{t-1} là giá trị EMA trước đó, và $\alpha \in (0, 1)$ là hệ số làm mịn (smoothing factor).

Giá trị α đóng vai trò then chốt trong việc cân bằng giữa hai yếu tố: khả năng phản ứng nhanh với dữ liệu mới (reactivity) và độ ổn định trước nhiễu (stability). Giá trị α lớn (gần 1) khiến hệ thống phản ứng nhanh với dữ liệu mới nhưng cũng dễ bị ảnh hưởng bởi nhiễu. Ngược lại, α nhỏ (gần 0) cho hệ thống ổn định hơn nhưng chậm thích ứng với thay đổi thực sự.

2.5.2 Ứng dụng trong học thích ứng

Trong hệ thống CHRONIX, EMA được sử dụng để cập nhật “điểm hài lòng” (satisfaction score) của từng thành phần trọng số dựa trên phản hồi ngầm định từ hành vi người dùng. Công thức cụ thể sử dụng $\alpha = 0.3$:

$$sat_{new} = sat_{old} \times 0.7 + signal \times 0.3 \quad (2.5)$$

trong đó $signal$ là tín hiệu phản hồi được trích xuất từ hành vi người dùng, nằm trong khoảng $[0, 1]$. Giá trị $signal$ gần 1 cho thấy người dùng hài lòng với khía cạnh tương ứng, và gần 0 cho thấy không hài lòng.

Với $\alpha = 0.3$, ảnh hưởng của giá trị ban đầu sau n quan sát là $(1 - \alpha)^n = 0.7^n$. Cụ thể, sau 5 quan sát, ảnh hưởng còn lại là $0.7^5 \approx 16.8\%$; sau 10 quan sát là $0.7^{10} \approx 2.8\%$; và sau 15 quan sát chỉ còn $0.7^{15} \approx 0.5\%$. Điều này có nghĩa là hệ thống cơ bản hội tụ trong khoảng 10 đến 15 quan sát, tương đương 1 đến 2 tuần sử dụng thực tế.

2.5.3 Tín hiệu phản hồi ngầm định

Một đặc điểm quan trọng của hệ thống EMA trong CHRONIX là sử dụng **phản hồi ngầm định** (implicit feedback) thay vì yêu cầu người dùng đánh giá trực tiếp sau mỗi công việc. Phản hồi ngầm định được trích xuất từ hành vi tự nhiên của người dùng trong quá trình sử dụng ứng dụng, bao gồm 5 loại tín hiệu chính. Mỗi tín hiệu tạo ra một giá trị signal S trong khoảng $[0, 1]$, trong đó giá trị gần 1 cho thấy người dùng hài lòng và gần 0 cho thấy không hài lòng.

Tín hiệu 1: Thời gian hoàn thành (Completion Timing). Tín hiệu này so sánh thời gian hoàn thành thực tế với thời gian kết thúc dự kiến trong lịch trình. Gọi Δt là độ trễ (phút), tính bằng thời điểm hoàn thành trừ thời điểm kết thúc dự kiến ($\Delta t < 0$ nghĩa là hoàn thành sớm, $\Delta t > 0$ là hoàn thành trễ):

$$S_{\text{timing}} = \begin{cases} 1.0 & \text{nếu } \Delta t \leq 0 \text{ và } |\Delta t| \leq 120 \text{ phút (sớm } \leq 2 \text{ giờ)} \\ 0.6 & \text{nếu } \Delta t \leq 0 \text{ và } 120 < |\Delta t| \leq 240 \text{ phút (sớm 2-4 giờ)} \\ 0.4 & \text{nếu } \Delta t \leq 0 \text{ và } |\Delta t| > 240 \text{ phút (sớm } > 4 \text{ giờ)} \\ 0.7 & \text{nếu } 0 < \Delta t \leq 60 \text{ phút (trễ } \leq 1 \text{ giờ)} \\ 0.4 & \text{nếu } 60 < \Delta t \leq 240 \text{ phút (trễ 1-4 giờ)} \\ \text{null} & \text{nếu } \Delta t > 240 \text{ phút (trễ } > 4 \text{ giờ, cần hỏi người dùng)} \end{cases} \quad (2.6)$$

Trường hợp trễ hơn 4 giờ trả về null vì sự chênh lệch quá lớn, hệ thống không thể suy đoán nguyên nhân (có thể do quên, bận việc khác, v.v.) nên hiển thị modal hỏi người dùng lý do cụ thể. Tín hiệu này ảnh hưởng đến tất cả 6 thành phần trọng số vì phản ánh mức độ phù hợp tổng thể của lịch trình.

Tín hiệu 2: Lựa chọn phương án (Option Selection). Khi hệ thống đề xuất top 3 khe thời gian, mỗi phương án có bảng phân tích (breakdown) chi tiết từng thành phần hình phạt. Việc người dùng chọn phương án nào phản ánh yếu tố họ coi trọng. Gọi P_i^{best} là giá trị hình phạt của thành phần i trong phương án tối ưu (xếp hạng 1), P_i^{sel} là giá trị tương ứng trong phương án được chọn, và $d_i = P_i^{\text{sel}} - P_i^{\text{best}}$:

$$S_{\text{option},i} = \begin{cases} \text{không tạo tín hiệu} & \text{nếu chọn phương án tối ưu (xếp hạng 1)} \\ 0.35 \text{ (cho tất cả } i) & \text{nếu người dùng tự nhập thời gian tùy chỉnh} \\ \text{clamp}(0.5 - d_i, 0.2, 0.8) \times f_{\text{spread}} & \text{trường hợp còn lại} \end{cases} \quad (2.7)$$

Hàm $\text{clamp}(x, a, b) = \max(a, \min(b, x))$ (hay thường được gọi là clipping activation function trong lĩnh vực AI, ML/DL) giới hạn giá trị x trong khoảng $[a, b]$: nếu $x < a$ thì trả về a , nếu $x > b$ thì trả về b , nếu không thì giữ nguyên x . Ở đây, $\text{clamp}(0.5 - d_i, 0.2, 0.8)$ đảm bảo tín hiệu thô luôn nằm trong khoảng $[0.2, 0.8]$, tránh tín hiệu quá cực đoan từ một lần chọn duy nhất.

f_{spread} là hệ số điều chỉnh dựa trên độ chênh lệch chi phí tổng giữa các phương án. C_{max}

và C_{min} là chi phí tổng $C(s)$ (công thức 3.2) cao nhất và thấp nhất trong top 3 phương án được đề xuất:

$$f_{spread} = \text{clamp}\left(\frac{C_{max} - C_{min}}{20}, 0.25, 1.0\right) \quad (2.8)$$

Chia cho 20 để chuẩn hóa: khi chi phí giữa phương án tốt nhất và kém nhất chênh ≥ 20 điểm, thì $f_{spread} = 1.0$ (tín hiệu có sức mạnh lớn nhất), sự khác biệt lớn đó chứng tỏ sự lựa chọn của người dùng thực sự có ý nghĩa. Cận dưới 0.25 đảm bảo tín hiệu không bị triệt tiêu hoàn toàn ngay cả khi các phương án rất gần nhau, vẫn giữ lại 25% sức mạnh tín hiệu vì người dùng đã chủ động chọn phương án khác phương án tối ưu. Cận trên 1.0 là giới hạn tự nhiên (không khuếch đại tín hiệu).

Ví dụ: Nếu 3 phương án có chi phí $C = 35, 40, 42$ thì $f_{spread} = (42 - 35)/20 = 0.35$, tín hiệu chỉ giữ 35% sức mạnh. Nếu $C = 20, 55, 60$ thì $f_{spread} = (60 - 20)/20 = 2.0$, bị giới hạn xuống 1.0.

Khi $d_i > 0$ (người dùng chấp nhận hình phạt cao hơn ở thành phần i), tín hiệu giảm dưới 0.5, nghĩa là người dùng ít quan tâm đến yếu tố đó. Khi $d_i < 0$ (phương án được chọn tốt hơn ở thành phần i), tín hiệu tăng trên 0.5, nghĩa là người dùng coi trọng yếu tố đó.

Tín hiệu 3: Hành vi dời lịch (Rescheduling). Khi người dùng kéo thả công việc sang thời điểm khác, hệ thống phân tích 3 khía cạnh thay đổi. Gọi $h_{diff} = |h_{new} - h_{old}|$ là độ lệch giờ giữa vị trí cũ và mới, g_{min} là khoảng cách nhỏ nhất (phút) đến sự kiện gần nhất tại vị trí mới:

$$S_{reschedule} = \begin{cases} \text{Chronotype:} & \begin{cases} 0.3 & \text{nếu } h_{diff} \geq 3 \text{ (dời xa, năng lượng không phù hợp)} \\ 0.4 & \text{nếu } 1 \leq h_{diff} < 3 \end{cases} \\ \text{Buffer:} & \begin{cases} 0.3 & \text{nếu } g_{min} < 5 \text{ phút (chấp nhận không nghỉ)} \\ 0.4 & \text{nếu } 5 \leq g_{min} < 15 \text{ phút} \end{cases} \\ \text{Deadline:} & 0.3 \text{ nếu dời từ trước deadline sang sau deadline} \end{cases} \quad (2.9)$$

Mỗi khía cạnh được kiểm tra độc lập. Nếu một khía cạnh không thay đổi đáng kể, tín hiệu tương ứng không được tạo. Tất cả tín hiệu trong nhóm này đều có giá trị < 0.5 , cho thấy sự không hài lòng với yếu tố tương ứng.

Tín hiệu 4: Thay đổi thời lượng (Resizing). Tín hiệu này chỉ được tạo khi người dùng kéo dài thời gian công việc (không áp dụng khi rút ngắn). Gọi g_{old} và g_{new} là khoảng cách (phút) đến sự kiện kế tiếp trước và sau khi kéo dài, với điều kiện $g_{old} \leq 30$ phút (chỉ tạo tín hiệu khi khoảng cách ban đầu đã nhỏ):

$$S_{resize} = \begin{cases} \text{Buffer:} & \begin{cases} 0.3 & \text{nếu } gold \leq 30 \text{ và } g_{new} < 5 \text{ (hy sinh thời gian nghỉ)} \\ 0.4 & \text{nếu } gold \leq 30 \text{ và } 5 \leq g_{new} < 15 \end{cases} \\ \text{Priority:} & 0.7 \text{ nếu kéo dài gây xung đột (chồng lên sự kiện kế tiếp)} \end{cases} \quad (2.10)$$

Tín hiệu $priority = 0.7 > 0.5$ cho thấy người dùng coi trọng công việc hiện tại đủ để chấp nhận xung đột, khiến hệ thống tăng trọng số $priority$ trong các lần lập lịch sau.

Tín hiệu 5: Đánh giá cảm xúc (Emoji Rating). Đây là hình thức phản hồi tường minh duy nhất trong hệ thống, nhưng được thiết kế đơn giản nhất có thể để không gây phiền cho người dùng. Sau khi hoàn thành công việc, người dùng có thể (không bắt buộc) chọn 1 trong 4 mức đánh giá:

$$S_{rating} = \begin{cases} 1.00 & \text{Tuyệt vời (lịch trình rất phù hợp)} \\ 0.75 & \text{Tốt (lịch trình ổn)} \\ 0.50 & \text{Bình thường (trung tính, không ảnh hưởng trọng số)} \\ 0.25 & \text{Mệt mỏi (thời điểm không phù hợp năng lượng)} \\ \text{null} & \text{Người dùng không đánh giá} \end{cases} \quad (2.11)$$

Tín hiệu này ảnh hưởng đến tất cả 6 thành phần trọng số vì phản ánh cảm nhận chủ quan tổng thể của người dùng.

Kết hợp tín hiệu 1 và 5: Tín hiệu thời gian hoàn thành (S_{timing} , khách quan) và đánh giá cảm xúc (S_{rating} , chủ quan) được kết hợp theo trọng số để tạo tín hiệu tổng hợp:

$$S_{combined} = \begin{cases} 0.4 \cdot S_{timing} + 0.6 \cdot S_{rating} & \text{nếu cả hai có giá trị} \\ S_{rating} & \text{nếu } S_{timing} = \text{null và } S_{rating} \neq \text{null} \\ S_{timing} & \text{nếu } S_{rating} = \text{null và } S_{timing} \neq \text{null} \\ \text{null} & \text{nếu cả hai đều null (không cập nhật EMA)} \end{cases} \quad (2.12)$$

Trọng số 60% cho đánh giá cảm xúc phản ánh rằng cảm nhận chủ quan của người dùng quan trọng hơn chỉ số khách quan về thời gian. Trong trường hợp cả S_{timing} và S_{rating} đều null (ví dụ, người dùng hoàn thành trễ hơn 4 giờ so với ước lượng và không đánh giá), $S_{combined} = \text{null}$ và hệ thống EMA không cập nhật trọng số cho lần quan sát đó, giữ nguyên bộ trọng số hiện tại. Cách tiếp cận phản hồi ngầm định này giảm thiểu gánh nặng cho người dùng (không cần điền khảo sát sau mỗi công việc) trong khi vẫn thu thập được đủ dữ liệu để cải thiện lịch trình theo thời gian.

2.6 Các công trình liên quan

2.6.1 Nghiên cứu về ảnh hưởng của chronotype đến hiệu suất

Mối quan hệ giữa chronotype và hiệu suất làm việc đã được nghiên cứu trong nhiều công trình khoa học. Nghiên cứu nổi bật nhất là của Facer-Childs và cộng sự (2019), được công bố trên tạp chí Sleep. Nhóm nghiên cứu đã tiến hành thử nghiệm trên 56 người tham gia, bao gồm cả nhóm sáng (morning type) và nhóm tối (evening type), đo lường hiệu suất nhận thức và thể chất tại nhiều thời điểm khác nhau trong ngày. Kết quả cho thấy sự chênh lệch lên đến **26%** về hiệu suất giữa thời điểm tối ưu (phù hợp chronotype) và thời điểm không tối ưu. Cụ thể, người thuộc nhóm sáng đạt hiệu suất cao nhất vào buổi sáng và suy giảm đáng kể vào buổi chiều tối, trong khi người thuộc nhóm tối có xu hướng ngược lại.

Nghiên cứu này có ý nghĩa quan trọng đối với CHRONIX vì nó cung cấp bằng chứng định lượng rằng việc xếp công việc đúng thời điểm phù hợp chronotype có thể cải thiện hiệu suất lên đến hơn một phần tư. Đây là cơ sở khoa học mạnh mẽ cho việc đưa yếu tố năng lượng sinh học (chronotype alignment) vào hàm chi phí của hệ thống lập lịch.

Ngoài ra, nghiên cứu của Wittmann và cộng sự (2006) đã giới thiệu khái niệm “lệch pha xã hội” (social jetlag), mô tả sự chênh lệch giữa đồng hồ sinh học nội tại và lịch trình xã hội bắt buộc (giờ làm việc, giờ học). Nghiên cứu cho thấy lệch pha xã hội ảnh hưởng tiêu cực đến sức khỏe, tâm trạng, và hiệu suất làm việc. CHRONIX giúp giảm thiểu hiện tượng này bằng cách sắp xếp công việc khó vào thời điểm phù hợp với đồng hồ sinh học cá nhân thay vì ép buộc theo lịch trình cố định.

Adan và cộng sự (2012) cũng đã thực hiện một bài tổng quan toàn diện (comprehensive review) về phân loại nhịp sinh học, tổng hợp hơn 200 nghiên cứu từ nhiều quốc gia. Bài tổng quan khẳng định rằng chronotype là đặc điểm sinh học ổn định, có cơ sở di truyền rõ ràng, và có ảnh hưởng đáng kể đến hiệu suất nhận thức, tâm trạng, cũng như sức khỏe tổng thể. Kết quả này củng cố thêm tính hợp lệ của việc sử dụng chronotype làm cơ sở cho hệ thống lập lịch cá nhân hóa.

2.6.2 Hệ thống gợi ý thích ứng (Adaptive Recommender Systems)

Trong lĩnh vực thương mại điện tử và giải trí trực tuyến, hệ thống gợi ý (recommender system) sử dụng các phương pháp học thích ứng (adaptive learning) để cập nhật sở thích người dùng theo thời gian đã được nghiên cứu và áp dụng rộng rãi. Koren và cộng sự (2009) đã công bố nghiên cứu quan trọng về kỹ thuật phân rã ma trận (matrix factorization) cho hệ thống gợi ý, chứng minh rằng việc kết hợp mô hình cơ sở (baseline model) với học tăng dần (incremental learning) cho hiệu suất dự đoán cao hơn đáng kể so với mô hình tĩnh (static model). Cách tiếp cận này cho phép hệ thống vừa hoạt động tốt ngay từ đầu dựa trên mô hình cơ sở, vừa cải thiện dần khi thu thập thêm dữ liệu từ người dùng.

CHRONIX áp dụng tư tưởng tương tự trong kiến trúc trọng số 3 lớp. Hồ sơ cài đặt sẵn (preset profile) đóng vai trò mô hình cơ sở, cung cấp bộ trọng số hợp lý ngay từ lần sử dụng

đầu tiên dựa trên chronotype và phong cách làm việc được chọn. Thuật toán EMA sau đó thực hiện học tăng dần từ phản hồi ngầm định của người dùng, dần dần tinh chỉnh trọng số để phản ánh đúng ưu tiên thực tế. Sự tương đồng về mặt phương pháp này cho thấy cách tiếp cận của CHRONIX có cơ sở vững chắc từ lĩnh vực hệ thống gợi ý.

2.6.3 Tối ưu hóa lập lịch đa mục tiêu (Multi-objective Scheduling)

Bài toán lập lịch công việc với nhiều ràng buộc đã được nghiên cứu rộng rãi trong lĩnh vực nghiên cứu vận hành (operations research). Pinedo (2016) đã trình bày một cách hệ thống các phương pháp giải bài toán lập lịch, từ các phương pháp chính xác (exact methods) như quy hoạch tuyến tính (linear programming) và quy hoạch nguyên (integer programming) cho bài toán nhỏ, đến các phương pháp siêu nghiệm (meta-heuristic) như thuật toán di truyền (genetic algorithm), mô phỏng luyện kim (simulated annealing), và tìm kiếm tabu (tabu search) cho bài toán lớn.

Các phương pháp chính xác đảm bảo tìm được nghiệm tối ưu toàn cục (global optimum) nhưng có độ phức tạp tính toán cao, thường là NP-hard, khiến chúng không phù hợp với ứng dụng thời gian thực. Các phương pháp siêu nghiệm cho kết quả gần tối ưu trong thời gian hợp lý nhưng đòi hỏi cấu hình phức tạp (tuning tham số) và thời gian tính toán vẫn có thể lên đến vài giây, không lý tưởng cho trải nghiệm người dùng (user experience) trên ứng dụng web.

CHRONIX chọn phương pháp tổng trọng số (weighted sum) kết hợp với thuật toán tham lam vì hai lý do chính. Thứ nhất, trong bối cảnh lập lịch cá nhân với số lượng công việc vừa phải (thường dưới 50 công việc mỗi tuần), sự chênh lệch chất lượng giữa nghiệm tham lam và nghiệm tối ưu toàn cục là không đáng kể. Thứ hai, thuật toán tham lam cho thời gian phản hồi gần như tức thì (dưới 100ms), đáp ứng yêu cầu trải nghiệm người dùng mượt mà trên ứng dụng web.

2.6.4 So sánh với các ứng dụng hiện có

Để đánh giá vị trí của CHRONIX trong bối cảnh các ứng dụng lập lịch hiện có, Bảng 2.3 so sánh các tính năng chính giữa CHRONIX và hai ứng dụng quản lý công việc phổ biến nhất trên thị trường: Google Calendar và Todoist.

Bảng 2.3: So sánh CHRONIX với các ứng dụng lập lịch hiện có

Tính năng	Google Calendar	Todoist	CHRONIX
Nhận biết chronotype	Không	Không	Có
Lập lịch theo năng lượng	Không	Không	Có
Hàm chi phí đa yếu tố	Không	Không	Có (6 yếu tố)
Trọng số thích ứng	Không	Không	Có (3 lớp)
Trợ lý AI chat	Không	Có (cơ bản)	Có
Đồng bộ lịch	Có	Có	Có
Tự động phân bổ thời gian	Không	Không	Có
Học từ phản hồi người dùng	Không	Không	Có (EMA)

Google Calendar là ứng dụng lịch được sử dụng rộng rãi nhất thế giới, cung cấp các tính năng tạo sự kiện, chia sẻ lịch, nhắc nhở, và tích hợp với nhiều dịch vụ của Google. Tuy nhiên, Google Calendar hoạt động như một công cụ *ghi nhận* sự kiện thụ động, người dùng phải tự quyết định thời điểm và thời lượng cho mỗi công việc. Ứng dụng không có bất kỳ cơ chế nào để gợi ý thời điểm tối ưu dựa trên đặc điểm sinh học hay thói quen làm việc của người dùng.

Todoist là ứng dụng quản lý công việc phổ biến, tập trung vào việc tổ chức danh sách công việc với các tính năng như phân loại dự án, đặt độ ưu tiên, lọc theo nhãn (label), và gần đây đã tích hợp trợ lý AI cơ bản để hỗ trợ tạo công việc. Tuy nhiên, Todoist không có tính năng lập lịch tự động, nghĩa là người dùng phải tự quyết định khi nào thực hiện mỗi công việc. Trợ lý AI của Todoist chủ yếu hỗ trợ phân tích ngôn ngữ tự nhiên (natural language processing) để tạo nhanh công việc, chưa có khả năng đề xuất thời gian tối ưu.

CHRONIX khác biệt ở chỗ không chỉ *ghi nhận* và *nhắc nhở* công việc, mà còn có khả năng *tự động tối ưu hóa* lịch trình dựa trên đặc điểm sinh học của người dùng. Sự kết hợp giữa nhận biết chronotype, hàm chi phí đa yếu tố, hệ thống trọng số thích ứng 3 lớp, và trợ lý AI chat tạo nên một hệ thống lập lịch thông minh mà chưa có ứng dụng nào trên thị trường cung cấp.

2.7 Tóm tắt

Qua quá trình tổng hợp và phân tích các cơ sở lý thuyết cùng các công trình nghiên cứu liên quan, có thể rút ra bốn nhận định chính.

Thứ nhất, chronotype có ảnh hưởng đáng kể đến hiệu suất làm việc. Nghiên cứu của Facer-Childs và cộng sự (2019) đã chứng minh sự chênh lệch lên đến 26% về hiệu suất giữa thời điểm tối ưu và không tối ưu theo chronotype. Kết quả này cung cấp cơ sở khoa học vững chắc cho việc đưa yếu tố chronotype vào bài toán lập lịch.

Thứ hai, các ứng dụng lập lịch hiện tại trên thị trường (Google Calendar, Todoist) chưa khai thác thông tin chronotype trong quá trình sắp xếp công việc. Khoảng trống này tạo ra cơ hội cho CHRONIX đóng góp một giải pháp mới, kết hợp kiến thức về sinh học thời gian vào hệ thống lập lịch thực tế.

Thứ ba, phương pháp tổng trọng số (weighted sum) kết hợp thuật toán tham lam (greedy algorithm) là cách tiếp cận phù hợp cho bài toán lập lịch cá nhân. Phương pháp này cân bằng tốt giữa chất lượng kết quả và tốc độ tính toán, đáp ứng yêu cầu phản hồi nhanh của ứng dụng web trong khi vẫn cho kết quả gần tối ưu.

Thứ tư, thuật toán EMA là kỹ thuật hiệu quả để học từ phản hồi ngầm định (implicit feedback) của người dùng. Kết hợp với kiến trúc trọng số 3 lớp lấy cảm hứng từ hệ thống gợi ý thích ứng (Koren et al., 2009), hệ thống có khả năng vừa hoạt động tốt ngay từ đầu, vừa tự cải thiện theo thời gian mà không gây gánh nặng cho người dùng.

CHRONIX đóng góp bằng cách **kết hợp** tất cả các yếu tố trên vào một hệ thống thống nhất: lập lịch dựa trên chronotype, hàm chi phí đa yếu tố, và trọng số thích ứng EMA. Các chương tiếp theo sẽ trình bày chi tiết thiết kế và triển khai hệ thống này.

Chương 3

Thiết kế hệ thống

Chương này trình bày chi tiết thiết kế hệ thống CHRONIX, bao gồm kiến trúc tổng thể, thiết kế cơ sở dữ liệu, hàm chi phí 6 yếu tố, hệ thống trọng số thích ứng 3 lớp, thuật toán lập lịch tham lam, trợ lý AI chat, và cơ chế đồng bộ Google Calendar.

3.1 Kiến trúc tổng thể

3.1.1 Mô hình kiến trúc

CHRONIX được xây dựng theo mô hình kiến trúc module nguyên khối (modular monolith), trong đó toàn bộ mã nguồn nằm trong một ứng dụng duy nhất nhưng được tổ chức thành các module chức năng độc lập. Kiến trúc bao gồm 4 tầng chính.

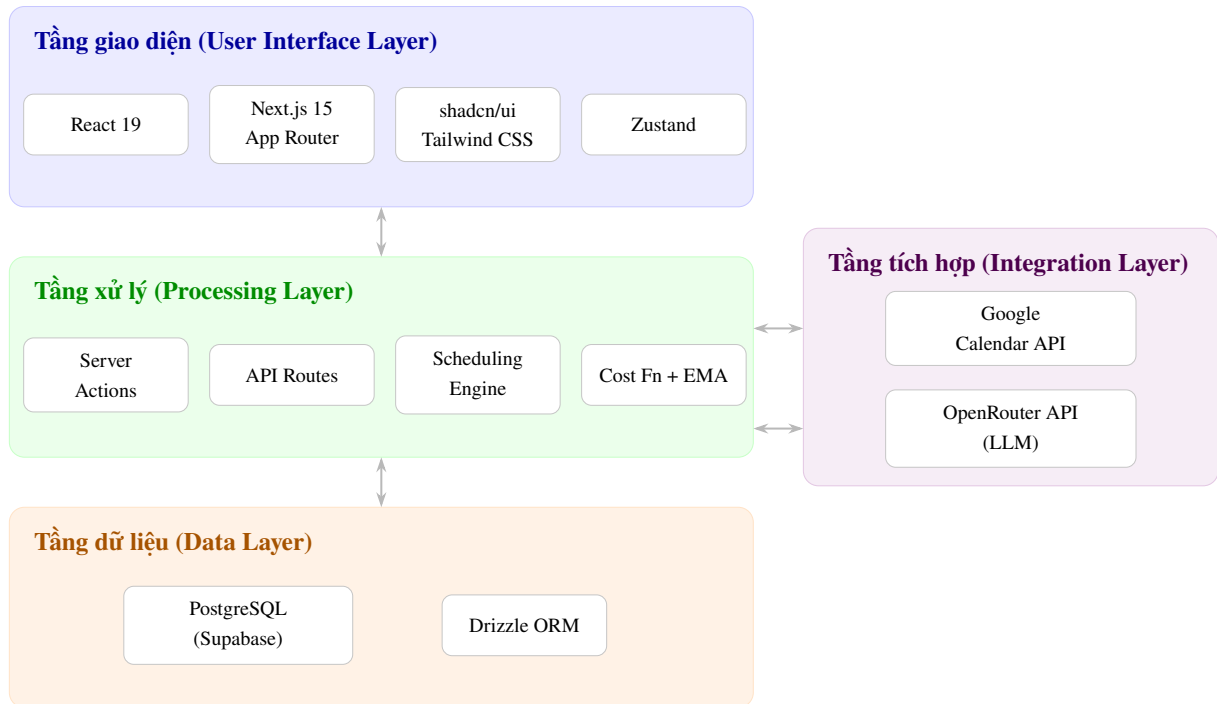
Tầng giao diện (User Interface Layer) sử dụng React 19 kết hợp với Next.js 15 App Router. Giao diện được xây dựng bằng các thành phần (component) của thư viện shadcn/ui, dựa trên Radix UI và Tailwind CSS. Quản lý trạng thái phía client sử dụng Zustand, một thư viện quản lý trạng thái gọn nhẹ cho React. Next.js 15 App Router cho phép kết hợp Server Components (render trên server, giảm JavaScript gửi xuống client) và Client Components (xử lý tương tác người dùng) trong cùng một ứng dụng, tối ưu cả hiệu năng lẫn trải nghiệm người dùng.

Tầng xử lý (Processing Layer) sử dụng Next.js API Routes cho các endpoint REST và Server Actions cho các thao tác mutation (tạo, cập nhật, xóa dữ liệu). Server Actions là tính năng của React 19 cho phép gọi hàm server trực tiếp từ client mà không cần viết API endpoint riêng, giảm đáng kể mã nguồn boilerplate. Tầng này chứa toàn bộ logic nghiệp vụ bao gồm scheduling engine, hàm chi phí, và xử lý phản hồi EMA.

Tầng dữ liệu (Data Layer) sử dụng PostgreSQL thông qua nền tảng Supabase, một dịch vụ cơ sở dữ liệu đám mây (Database as a Service - DBaaS) cung cấp PostgreSQL với các tính năng bổ sung như xác thực, lưu trữ, và API tự động. Truy xuất dữ liệu sử dụng Drizzle ORM, một thư viện ORM (Object-Relational Mapping) cho TypeScript có đặc điểm type-safe (an toàn kiểu dữ liệu), nghĩa là các truy vấn được kiểm tra kiểu dữ liệu tại thời điểm biên dịch, giúp phát

hiện lỗi sớm trước khi chạy ứng dụng.

Tầng tích hợp (Integration Layer) kết nối với hai dịch vụ bên ngoài. Google Calendar API cung cấp khả năng đồng bộ hai chiều (two-way sync) sự kiện lịch, cho phép người dùng sử dụng CHRONIX song song với Google Calendar mà không bị mất đồng bộ. OpenRouter API cung cấp truy cập đến các mô hình ngôn ngữ lớn (large language model, viết tắt LLM) cho tính năng trợ lý AI chat.



Hình 3.1: Kiến trúc tổng thể hệ thống CHRONIX

3.1.2 Tổ chức mã nguồn theo module

Mã nguồn được tổ chức theo cấu trúc feature-based (theo tính năng), trong đó mỗi module chức năng nằm trong một thư mục riêng với đầy đủ các thành phần cần thiết: các thành phần giao diện (thư mục components), logic nghiệp vụ (thư mục lib), hành động phía server (thư mục actions), và React hooks. Cấu trúc này bao gồm 6 module chính.

- **auth:** Quản lý xác thực người dùng qua Google OAuth 2.0.
- **onboarding:** Xử lý quy trình khởi tạo hồ sơ người dùng mới, bao gồm bài khảo sát chronotype và lựa chọn hồ sơ lập lịch.
- **scheduling:** Chứa scheduling engine với hàm chi phí, thuật toán tham lam, đường cong năng lượng, hệ thống trọng số, và bộ xử lý phản hồi (feedback processor).
- **calendar:** Quản lý hiển thị lịch và đồng bộ với Google Calendar.
- **chat:** Cung cấp giao diện trợ lý AI chat.
- **dashboard:** Tổng hợp giao diện chính của ứng dụng.

3.1.3 Luồng dữ liệu chính

Hệ thống hoạt động theo luồng dữ liệu tuần tự như sau. Đầu tiên, người dùng đăng nhập vào ứng dụng thông qua Google OAuth 2.0 với phương thức PKCE (Proof Key for Code Exchange), đảm bảo an toàn cho luồng xác thực. Sau khi đăng nhập lần đầu, người dùng được chuyển đến quy trình onboarding gồm 7 bước, trong đó bước quan trọng nhất là hoàn thành bài khảo sát chronotype 5 câu hỏi (dựa trên AutoMEQ) để xác định nhóm chronotype. Tiếp theo, hệ thống thực hiện đồng bộ ban đầu (initial sync) với Google Calendar, lấy toàn bộ sự kiện trong 14 ngày trước và 14 ngày tới, đồng thời lưu syncToken cho các lần đồng bộ tăng dần (incremental sync) sau này. Khi người dùng tạo công việc mới (qua giao diện hoặc AI chat), scheduling engine tìm tất cả khe thời gian khả dụng, tính điểm chi phí cho từng khe bằng hàm chi phí 6 yếu tố, và chọn khe có chi phí thấp nhất. Sau khi hoàn thành công việc, hệ thống thu thập phản hồi ngầm định từ hành vi người dùng (thời gian hoàn thành, hành vi dời lịch, thay đổi thời lượng, v.v.). Cuối cùng, thuật toán EMA cập nhật trọng số thích ứng dựa trên phản hồi ngầm định, cải thiện chất lượng lập lịch cho các lần tiếp theo.

3.2 Thiết kế xác thực và onboarding

3.2.1 Xác thực với Better Auth và Google OAuth

Hệ thống xác thực sử dụng Better Auth, một framework xác thực mã nguồn mở cho TypeScript, kết hợp với Google OAuth 2.0 làm phương thức đăng nhập duy nhất. Lý do chọn Google OAuth là vì hệ thống cần truy cập Google Calendar API để đồng bộ sự kiện, do đó người dùng bắt buộc phải có tài khoản Google. Việc giới hạn một phương thức đăng nhập cũng đơn giản hóa việc quản lý token.

Trong quá trình đăng nhập, hệ thống yêu cầu 4 phạm vi quyền (scope) từ Google: openid (xác thực danh tính), email (địa chỉ email), profile (tên và ảnh đại diện), và hai scope của Google Calendar gồm calendar.readonly (đọc sự kiện) và calendar.events (tạo, sửa, xóa sự kiện). Tham số responseType: "offline" và prompt: "consent" đảm bảo hệ thống nhận được refresh token, cho phép làm mới access token khi hết hạn mà không cần người dùng đăng nhập lại.

Phiên đăng nhập (session) có thời hạn 7 ngày, được tự động gia hạn mỗi 24 giờ khi người dùng hoạt động. Cookie phiên được đặt tiền tố chronix và sử dụng chế độ bảo mật (Secure Cookie) trong môi trường production. Ngoài ra, cookie cache được bật với thời hạn 5 phút để giảm số lần truy vấn cơ sở dữ liệu khi kiểm tra phiên.

3.2.2 Quy trình onboarding

Sau khi đăng nhập lần đầu, người dùng được chuyển đến quy trình onboarding gồm 7 bước tuần tự. Các bước bao gồm: chào mừng (bước 1), bài khảo sát chronotype 5 câu hỏi (bước 2 đến 6), kết quả chronotype và lựa chọn hồ sơ lập lịch (bước 7).

Bài khảo sát chronotype gồm 5 câu hỏi được thiết kế dựa trên bảng câu hỏi AutoMEQ (Automated Morningness-Eveningness Questionnaire). Mỗi câu hỏi có 4 đến 5 lựa chọn, mỗi lựa chọn tương ứng với một điểm số từ 1 đến 5. Câu hỏi thứ nhất hỏi về thời gian thức dậy tự nhiên khi không bị ràng buộc. Câu hỏi thứ hai đánh giá mức độ tỉnh táo trong 30 phút đầu sau khi thức dậy. Câu hỏi thứ ba xác định thời điểm hiệu suất cao nhất cho công việc đòi hỏi tư duy. Câu hỏi thứ tư hỏi về thời điểm bắt đầu cảm thấy mệt vào buổi tối. Câu hỏi thứ năm đánh giá chất lượng giấc ngủ tổng thể.

Thuật toán phân loại chronotype hoạt động như sau. Trước tiên, nếu điểm chất lượng giấc ngủ (câu hỏi 5) bằng 1 (giấc ngủ không đều, thường xuyên bị gián đoạn), người dùng được phân vào nhóm Dolphin bất kể các câu trả lời khác, vì đây là đặc điểm nổi bật nhất của nhóm Dolphin. Nếu không phải Dolphin, tổng điểm 5 câu hỏi được tính (tối thiểu 5, tối đa 23). Tổng điểm từ 18 trở lên thuộc nhóm Lion (xu hướng sáng mạnh), từ 13 đến 17 thuộc nhóm Bear (trung bình), và dưới 13 thuộc nhóm Wolf (xu hướng tối). Công thức phân loại được định nghĩa như sau:

Gọi x_i là điểm số của câu hỏi thứ i ($i = 1, 2, \dots, 5$), với $x_i \in \{1, 2, 3, 4, 5\}$ cho câu hỏi 1, 3, 4 và $x_i \in \{1, 2, 3, 4\}$ cho câu hỏi 2, 5. Đặt $S = \sum_{i=1}^5 x_i$ là tổng điểm ($5 \leq S \leq 23$). Hàm phân loại chronotype C được xác định:

$$C(x_1, x_2, \dots, x_5) = \begin{cases} \text{Dolphin} & \text{nếu } x_5 = 1 \text{ (giấc ngủ không đều)} \\ \text{Lion} & \text{nếu } x_5 \neq 1 \text{ và } S \geq 18 \\ \text{Bear} & \text{nếu } x_5 \neq 1 \text{ và } 13 \leq S < 18 \\ \text{Wolf} & \text{nếu } x_5 \neq 1 \text{ và } S < 13 \end{cases} \quad (3.1)$$

Thuật toán này ưu tiên kiểm tra chất lượng giấc ngủ trước, phản ánh đặc trưng của nhóm Dolphin - những người có giấc ngủ bất thường, khác biệt rõ rệt so với 3 nhóm còn lại. Với các nhóm còn lại, phân loại dựa trên tổng điểm cho phép xác định chronotype nhanh chóng chỉ với 5 câu hỏi, thay vì bài khảo sát dài 19 câu của MEQ gốc.

Tại bước 7, sau khi xem kết quả chronotype, người dùng chọn một trong 5 hồ sơ lập lịch (scheduling profile): Hustle, Balanced, Wellness, Parent, hoặc Creative. Hồ sơ này quyết định bộ trọng số cơ sở (preset weights) cho hàm chi phí, sẽ được trình bày chi tiết ở phần hệ thống trọng số 3 lớp.

3.3 Thiết kế cơ sở dữ liệu

3.3.1 Tổng quan

Cơ sở dữ liệu PostgreSQL được thiết kế với 10 bảng chính, sử dụng Drizzle ORM để định nghĩa schema và thực hiện truy vấn. Toàn bộ bảng được định nghĩa trong một file duy nhất (`src/db/schema.ts`), đảm bảo tính nhất quán và dễ theo dõi mối quan hệ giữa các bảng.

Cơ sở dữ liệu sử dụng 7 kiểu enum tùy chỉnh để đảm bảo tính hợp lệ của dữ liệu:

- `chronotype`: lion, bear, wolf, dolphin.

- event_source: google, chronix, ai.
- sync_status: synced, pending_push, pending_pull, conflict.
- energy_level: high, medium, low.
- task_category: analytical, creative, administrative, physical, learning.
- task_status: pending, scheduled, completed, cancelled.
- scheduling_profile: hustle, balanced, wellness, parent, creative.

Bảng 3.1 tóm tắt 10 bảng trong cơ sở dữ liệu cùng mối quan hệ và chức năng của mỗi bảng.

Bảng 3.1: Danh sách các bảng trong cơ sở dữ liệu

Bảng	Mối quan hệ	Mô tả
users	—	Thông tin người dùng + chronotype + múi giờ
sessions	users (N:1)	Phiên đăng nhập (TTL 7 ngày)
accounts	users (N:1)	Tài khoản OAuth (Google)
google_tokens	users (1:1)	Token Google Calendar (mã hóa AES-256)
events	users (N:1)	Sự kiện lịch (Google + Chronix + AI)
tasks	users (N:1), events (1:1)	Công việc cần lập lịch
syncState	users (1:1)	Trạng thái đồng bộ Google Calendar
energyLogs	users (N:1), events (1:1)	Nhật ký năng lượng
weightAdjustments	users (1:1)	Trọng số thích ứng 3 lớp
verifications	—	Token xác thực (Better Auth)

Tất cả các bảng liên quan đến người dùng đều có khóa ngoại user_id tham chiếu đến bảng users với ràng buộc ON DELETE CASCADE, nghĩa là khi xóa một người dùng, toàn bộ dữ liệu liên quan cũng tự động bị xóa theo.

3.3.2 Bảng users

Bảng users là trung tâm của hệ thống, lưu trữ thông tin cơ bản và dữ liệu chronotype. Khóa chính sử dụng kiểu TEXT (UUID dạng chuỗi) thay vì SERIAL để tương thích với Better Auth. Các trường quan trọng bao gồm: chronotype (enum 4 giá trị), wakeTime và sleepTime (kiểu TIME, lưu giờ thức dậy và giờ ngủ tự nhiên), timezone (chuỗi múi giờ IANA, ví dụ “Asia/Ho_Chi_Minh”), và onboardingCompleted (boolean đánh dấu đã hoàn thành onboarding).

Đoạn mã 3.1: Cấu trúc bảng users

```

1 users (
2   id          TEXT PRIMARY KEY,
3   email       TEXT UNIQUE NOT NULL,
4   name        TEXT,
5   image       TEXT,

```

```

6   chronotype      ENUM('lion','bear','wolf','dolphin'),
7   wakeTime        TIME,          -- VD: "07:00"
8   sleepTime       TIME,          -- VD: "23:00"
9   timezone        TEXT DEFAULT 'UTC',
10  onboarding_completed BOOLEAN DEFAULT false
11 )

```

3.3.3 Bảng events

Bảng events lưu trữ tất cả sự kiện lịch từ 3 nguồn: Google Calendar (nguồn “google”), sự kiện tạo trực tiếp từ CHRONIX (nguồn “chronix”), và sự kiện tạo qua AI chat (nguồn “ai”). Thiết kế bảng hỗ trợ cả sự kiện thông thường lẫn sự kiện cả ngày (all-day event) thông qua trường allDay. Trường recurrenceRule lưu quy tắc lặp theo định dạng RRULE (RFC 5545) cho sự kiện định kỳ.

Để tối ưu hiệu năng truy vấn, bảng có 3 chỉ mục (index): chỉ mục kết hợp (userId, startTime) cho truy vấn sự kiện theo ngày, chỉ mục (userId, syncStatus) cho lọc sự kiện cần đồng bộ, và chỉ mục duy nhất (unique index) (userId, googleEventId) đảm bảo mỗi sự kiện Google chỉ tồn tại một bản ghi duy nhất cho mỗi người dùng, đồng thời hỗ trợ thao tác upsert (insert hoặc update nếu đã tồn tại) trong quá trình đồng bộ.

Metadata (siêu dữ liệu) đồng bộ bao gồm: etag (phiên bản sự kiện từ Google, dùng phát hiện xung đột), syncedAt (thời điểm đồng bộ gần nhất), localUpdatedAt (thời điểm cập nhật cục bộ, dùng so sánh với phiên bản Google), và syncStatus (enum 4 giá trị: synced, pending_push, pending_pull, conflict).

Đoạn mã 3.2: Cấu trúc bảng events

```

1  events (
2    id                TEXT PRIMARY KEY,
3    userId            TEXT NOT NULL REFERENCES users(id),
4    -- Danh danh Google Calendar
5    googleEventId     TEXT,
6    googleCalendarId  TEXT DEFAULT 'primary',
7    -- Du lieu su kien
8    title             TEXT NOT NULL,
9    description        TEXT,
10   startTime          TIMESTAMPTZ NOT NULL,
11   endTime            TIMESTAMPTZ NOT NULL,
12   allDay             BOOLEAN DEFAULT false,
13   location           TEXT,
14   recurrenceRule     TEXT,          -- Dinh dang RRULE (RFC 5545)
15   -- Metadata Chronix
16   source             ENUM('google','chronix','ai') NOT NULL,
17   energyLevel        ENUM('high','medium','low'),

```

```

18  -- Metadata dong bo
19  etag          TEXT ,
20  syncedAt      TIMESTAMPTZ ,
21  localUpdatedAt TIMESTAMPTZ DEFAULT now() ,
22  syncStatus    ENUM('synced','pending_push',
23                  'pending_pull','conflict')
24                  DEFAULT 'synced',
25  -- Index: (userId, startTime), (userId, syncStatus)
26  -- Unique index: (userId, googleEventId)
27 )

```

3.3.4 Bảng tasks

Bảng tasks lưu trữ công việc cần lập lịch với đầy đủ thông tin cho scheduling engine. Mỗi công việc có: category (loại công việc, quyết định yêu cầu năng lượng), durationMinutes (thời lượng ước tính, mặc định 30 phút), deadline (hạn chót, có thể null nếu không có), priority (độ ưu tiên 1 đến 5, mặc định 3), và khoảng thời gian ưa thích (preferredTimeStart, preferredTimeEnd) dạng HH:mm. Trường scheduledEventId liên kết đến sự kiện đã được tạo trên lịch khi công việc được lập lịch thành công.

Đoạn mã 3.3: Cấu trúc bảng tasks

```

1  tasks (
2      id          TEXT PRIMARY KEY ,
3      userId      TEXT NOT NULL REFERENCES users(id) ,
4      -- Chi tiet cong viec
5      title       TEXT NOT NULL ,
6      description  TEXT ,
7      category    ENUM('analytical','creative',
8                      'administrative','physical',
9                      'learning')
10                     DEFAULT 'administrative',
11  -- Rang buoc lap lich
12  durationMinutes INTEGER NOT NULL DEFAULT 30 ,
13  deadline        TIMESTAMPTZ ,
14  priority        INTEGER DEFAULT 3,  -- 1-5
15  -- Tuy chon thoi gian
16  preferredTimeStart TIME,          -- VD: "09:00"
17  preferredTimeEnd   TIME,          -- VD: "12:00"
18  -- Trang thai
19  status           ENUM('pending','scheduled',
20                      'completed','cancelled')
21                      DEFAULT 'pending',
22  scheduledEventId TEXT REFERENCES events(id) ,

```

```

23  -- Index: (userId, status), (userId, deadline)
24  )

```

3.3.5 Bảng weightAdjustments

Bảng weightAdjustments là trung tâm của hệ thống trọng số thích ứng 3 lớp. Mỗi người dùng có đúng một bản ghi (ràng buộc UNIQUE trên userId). Bảng lưu trữ 3 nhóm dữ liệu tương ứng với 3 lớp trọng số.

Nhóm thứ nhất là hồ sơ preset (schedulingProfile), lưu tên hồ sơ đang được sử dụng. Nhóm thứ hai là 6 giá trị điều chỉnh thủ công (manualChrono, manualDeadline, manualContext, manualFrag, manualPriority, manualBuffer), mỗi giá trị nằm trong khoảng ± 0.05 . Nhóm thứ ba bao gồm 6 giá trị EMA satisfaction (chronotypeSat, deadlineSat, v.v., mỗi giá trị trong khoảng $[0, 1]$, mặc định 0.5) và 6 giá trị điều chỉnh tự động (chronotypeAdj, deadlineAdj, v.v., mỗi giá trị trong khoảng ± 0.15), cùng với totalObservations đếm tổng số lần cập nhật EMA.

Đoạn mã 3.4: Cấu trúc bảng weightAdjustments (tóm tắt)

```

1  weightAdjustments (
2      id                TEXT PRIMARY KEY,
3      userId            TEXT UNIQUE REFERENCES users(id),
4      -- Lop 1: Preset profile
5      schedulingProfile  ENUM('hustle', 'balanced',
6                              'wellness', 'parent', 'creative'),
7      -- Lop 2: Dieu chinh thu cong (+-5%)
8      manualChrono      REAL DEFAULT 0,
9      manualDeadline    REAL DEFAULT 0,
10     ... (6 truong)
11     -- Lop 3: EMA satisfaction scores (0-1)
12     chronotypeSat      REAL DEFAULT 0.5,
13     deadlineSat        REAL DEFAULT 0.5,
14     ... (6 truong)
15     -- Lop 3: Auto adjustments (+-15%)
16     chronotypeAdj      REAL DEFAULT 0,
17     deadlineAdj        REAL DEFAULT 0,
18     ... (6 truong)
19     totalObservations  INTEGER DEFAULT 0
20 )

```

3.3.6 Các bảng hỗ trợ

Ngoài 4 bảng nghiệp vụ chính, hệ thống còn sử dụng 6 bảng hỗ trợ:

- **sessions**: Lưu phiên đăng nhập do Better Auth quản lý, bao gồm token phiên (duy nhất), thời điểm hết hạn, địa chỉ IP, và thông tin trình duyệt (userAgent). Mỗi người dùng có thể có nhiều phiên đăng nhập đồng thời trên nhiều thiết bị.
- **accounts**: Lưu thông tin tài khoản OAuth do Better Auth quản lý. Mỗi bản ghi chứa providerId (“google”), accessToken, refreshToken, và thời hạn token. Mặc dù thiết kế hỗ trợ nhiều provider, CHRONIX chỉ sử dụng Google OAuth nên mỗi người dùng có đúng 1 bản ghi.
- **verifications**: Lưu token xác thực tạm thời do Better Auth sử dụng nội bộ cho các tác vụ như xác minh email. Mỗi token có identifier, value, và thời hạn hết hạn.
- **google_tokens**: Lưu token Google Calendar được mã hóa AES-256. Ngoài việc được tách riêng khỏi bảng accounts để tăng bảo mật, token truy cập Calendar API còn được mã hóa tại tầng ứng dụng trước khi lưu vào cơ sở dữ liệu. Bảng cũng lưu danh sách scope đã được cấp quyền.
- **syncState**: Lưu trạng thái đồng bộ Google Calendar cho mỗi người dùng (quan hệ 1:1). Bao gồm syncToken (token đồng bộ tăng dần từ Google), lastSyncAt (thời điểm đồng bộ gần nhất), nextSyncAt (thời điểm dự kiến đồng bộ tiếp theo), và syncError (thông báo lỗi nếu đồng bộ thất bại).
- **energyLogs**: Lưu nhật ký năng lượng sau khi hoàn thành công việc, phục vụ hệ thống học thích ứng EMA. Mỗi bản ghi chứa energyBefore và energyAfter (mức năng lượng trước/sau khi làm việc, thang 1–5), productivityRating (đánh giá năng suất, thang 1–5), và ghi chú tùy chọn. Quan hệ 1:1 với bảng events.

3.4 Thiết kế hàm chi phí 6 yếu tố

3.4.1 Tổng quan

Hàm chi phí (cost function) là thành phần cốt lõi của scheduling engine (bộ máy lập lịch), đánh giá mức độ phù hợp của một khe thời gian (time slot) cho một công việc cụ thể. Chi phí càng thấp thì khe thời gian càng phù hợp. Hàm chi phí tổng hợp 6 thành phần hình phạt (penalty), mỗi thành phần có giá trị trong khoảng $[0, 1]$, được nhân với trọng số (weight) tương ứng:

$$C(s) = \sum_{i=1}^6 w_i \cdot P_i(s) = w_c \cdot P_{\text{chrono}} + w_d \cdot P_{\text{deadline}} + w_p \cdot P_{\text{priority}} + w_b \cdot P_{\text{buffer}} + w_{cs} \cdot P_{\text{context}} + w_f \cdot P_{\text{frag}} \quad (3.2)$$

trong đó s là khe thời gian cần đánh giá, $P_i(s) \in [0, 1]$ là giá trị hình phạt của thành phần thứ i , và $w_i \geq 0$ là trọng số tương ứng thỏa mãn ràng buộc chuẩn hóa $\sum_{i=1}^6 w_i = 1$. Kết quả trả về bao gồm tổng chi phí $C(s)$ và bảng phân tích chi tiết từng thành phần $w_i \cdot P_i(s)$, cho phép hệ thống giải thích lý do đề xuất cho người dùng.

3.4.2 Hình phạt năng lượng sinh học - Chronotype Penalty (P_{chrono})

Thành phần này đánh giá sự chênh lệch giữa mức năng lượng sinh học của người dùng tại thời điểm slot và yêu cầu năng lượng của công việc. Gọi E_{req} là năng lượng yêu cầu theo loại công việc (Bảng 3.2) và E_{avail} là năng lượng trung bình khả dụng của người dùng trong khoảng thời gian slot, được tra cứu từ đường cong năng lượng 24 giờ tương ứng với chronotype. Công thức tính:

$$P_{chrono}(s) = \frac{\max(0, E_{req} - E_{avail})}{100} \quad (3.3)$$

Để tính E_{avail} chính xác, hệ thống chuyển đổi thời điểm slot sang múi giờ cục bộ của người dùng trước khi tra cứu đường cong năng lượng, đảm bảo kết quả đúng bất kể người dùng ở múi giờ nào. Với công việc kéo dài từ giờ h_1 đến giờ h_2 , E_{avail} được tính bằng trung bình cộng:

$$E_{avail} = \frac{1}{h_2 - h_1} \sum_{h=h_1}^{h_2-1} E_{curve}[h] \quad (3.4)$$

trong đó $E_{curve}[h]$ là giá trị năng lượng tại giờ h trên đường cong 24 giờ của chronotype tương ứng.

Hình phạt bằng 0 khi năng lượng khả dụng bằng hoặc lớn hơn năng lượng yêu cầu (slot phù hợp), và tăng dần khi khoảng cách giữa yêu cầu và khả dụng lớn hơn (slot kém phù hợp). Ví dụ, nếu công việc Analytical cần năng lượng 90 nhưng slot chỉ có năng lượng 50, hình phạt sẽ là $(90 - 50)/100 = 0.4$.

Bảng 3.2: Yêu cầu năng lượng theo loại công việc

Loại công việc	E_{req}	Ví dụ
Analytical (Phân tích)	90	Viết code, debug, phân tích dữ liệu, giải toán
Learning (Học tập)	80	Đọc tài liệu, học kỹ năng mới, nghiên cứu
Creative (Sáng tạo)	70	Thiết kế giao diện, viết bài, brainstorm ý tưởng
Physical (Thể chất)	60	Họp trực tiếp, thuyết trình, tập thể dục
Administrative (Hành chính)	40	Soạn email, sắp xếp tài liệu, cập nhật báo cáo

3.4.3 Hình phạt rủi ro deadline - Deadline Risk ($P_{deadline}$)

Thành phần này đánh giá mức độ khẩn cấp dựa trên khoảng cách từ slot đến hạn chót (deadline), có tính đến độ ưu tiên (priority) của công việc. Công việc không có deadline sẽ có hình phạt bằng 0. Gọi t_{slack} là khoảng thời gian dự trữ (slack time) từ khi slot kết thúc đến deadline (tính bằng giờ), $p \in \{1, 2, 3, 4, 5\}$ là độ ưu tiên, và $T_{max} = 168$ giờ (1 tuần) là khoảng cách tối đa để tính rủi ro. Công thức tính:

$$P_{deadline}(s) = \begin{cases} 1 & \text{nếu } t_{slack} < 0 \text{ (đã quá hạn)} \\ \min \left(1, \underbrace{\max \left(0, 1 - \frac{t_{slack}}{T_{max}} \right)}_{\text{rủi ro cơ bản}} \times \underbrace{(1 + (p-3) \times 0.2)}_{\text{hệ số khẩn cấp}} \right) & \text{nếu } t_{slack} \geq 0 \end{cases} \quad (3.5)$$

Hệ số khẩn cấp (urgency multiplier) dựa trên priority tạo ra sự khác biệt rõ rệt: công việc priority 5 có hệ số 1.4 (khẩn cấp hơn 40% so với trung bình), priority 3 có hệ số 1.0 (trung tính), và priority 1 có hệ số 0.6 (ít khẩn cấp hơn 40%). Cách tiếp cận này khiến công việc quan trọng và gần deadline được ưu tiên xếp trước, trong khi công việc ít quan trọng có thể được dời sang thời điểm khác dù gần deadline.

3.4.4 Hình phạt độ ưu tiên - Priority Penalty ($P_{priority}$)

Thành phần này đảm bảo công việc có độ ưu tiên cao được xếp vào khe thời gian tốt hơn. Gọi $p \in \{1, 2, 3, 4, 5\}$ là độ ưu tiên của công việc:

$$P_{priority}(s) = \frac{5-p}{4} \quad (3.6)$$

Công thức ánh xạ tuyến tính từ priority sang hình phạt: priority 5 cho $P = 0.00$ (được ưu tiên cho slot tốt nhất), priority 3 cho $P = 0.50$ (trung tính), và priority 1 cho $P = 1.00$ (nhường slot tốt cho công việc quan trọng hơn). Thành phần này hoạt động ngược lại so với các hình phạt khác: thay vì phạt slot kém, nó phạt công việc kém ưu tiên, đẩy chúng sang slot ít tối ưu hơn.

3.4.5 Hình phạt khoảng nghỉ - Buffer Penalty (P_{buffer})

Thành phần này đảm bảo có đủ thời gian nghỉ (buffer) giữa các công việc liên tiếp. Hình phạt buffer kiểm tra khoảng cách giữa sự kiện trước đó và slot hiện tại có đủ thoải mái hay không, nhằm tránh xếp lịch dày đặc gây căng thẳng.

Gọi g là khoảng cách (tính bằng phút) từ thời điểm bắt đầu slot đến thời điểm kết thúc của sự kiện gần nhất trước đó. Hệ thống định nghĩa hai ngưỡng:

- $g_{ideal} = 15$ phút: khoảng nghỉ lý tưởng - đủ để người dùng chuyển tiếp thoải mái giữa các công việc (nghỉ ngơi, chuẩn bị tài liệu cho công việc tiếp theo).
- $g_{min} = 5$ phút: khoảng nghỉ tối thiểu - dưới ngưỡng này, hai công việc gần như liên tục và không có thời gian chuyển tiếp.

$$P_{buffer}(s) = \begin{cases} 0 & \text{nếu } g \geq g_{ideal} \text{ (nghỉ đủ, không phạt)} \\ 1 - \frac{g - g_{min}}{g_{ideal} - g_{min}} & \text{nếu } g_{min} \leq g < g_{ideal} \text{ (nghỉ ngắn, phạt tuyến tính)} \\ 1 & \text{nếu } g < g_{min} \text{ (gần như liên tục, phạt tối đa)} \end{cases} \quad (3.7)$$

Ví dụ: nếu $g = 10$ phút, hình phạt là $1 - (10 - 5)/(15 - 5) = 0.5$. Nếu $g = 7$ phút, hình phạt là $1 - (7 - 5)/(15 - 5) = 0.8$. Nếu có nhiều sự kiện trước slot, hình phạt lấy giá trị cao nhất (trường hợp xấu nhất).

Ngoài hình phạt buffer (đánh giá khoảng nghỉ *trước* slot), hệ thống còn có chức năng *gợi ý* thời gian nghỉ *sau* khi lập lịch thành công. Tỷ lệ nghỉ cơ bản lấy cảm hứng từ nghiên cứu 52:17 (52 phút làm việc, 17 phút nghỉ), tuy nhiên tỷ lệ này được thiết kế cho một chu kỳ làm việc đơn lẻ và không phù hợp khi áp dụng tuyến tính cho mọi thời lượng. Do đó, hệ thống áp dụng giới hạn trên và dưới:

$$t_{break} = \text{clamp}\left(\frac{t_{work} \times 17}{52}, 5, 20\right) \text{ phút} \quad (3.8)$$

trong đó t_{work} là thời lượng công việc (phút) và hàm $\text{clamp}(x, a, b) = \max(a, \min(b, x))$ giới hạn kết quả trong khoảng $[a, b]$.

- Cận dưới 5 phút: khoảng nghỉ dưới 5 phút không đủ ý nghĩa để phục hồi.
- Cận trên 20 phút: với công việc dài (ví dụ 120 phút), tỷ lệ 52:17 cho ra ≈ 39 phút nghỉ. Tuy nhiên, công việc dài trong thực tế thường đã có các khoảng nghỉ nhỏ bên trong (micro-break), nên không cần nghỉ dài sau khi kết thúc. Ngoài ra, nghỉ quá 20 phút có thể gây mất đà làm việc và khó quay lại trạng thái tập trung.

Ví dụ: công việc 30 phút được gợi ý nghỉ ≈ 10 phút, công việc 60 phút nghỉ ≈ 20 phút, công việc 120 phút nghỉ 20 phút (bị giới hạn).

3.4.6 Hình phạt chuyển đổi ngữ cảnh - Context Switch Penalty ($P_{context}$)

Thành phần này phạt việc chuyển đổi giữa các loại công việc khác nhau liên tiếp, dựa trên nghiên cứu của Ophir và cộng sự (2009) cho thấy cần trung bình $T_{recover} = 23$ phút để phục hồi sự tập trung sau khi chuyển đổi ngữ cảnh (context switching). Gọi g là khoảng cách thời gian (phút) đến công việc trước đó:

$$P_{context}(s) = \begin{cases} 0 & \text{nếu không có công việc trước đó} \\ 0 & \text{nếu cùng loại (category) với công việc trước} \\ \left(1 - \min\left(1, \frac{g}{T_{recover}}\right)\right) \times 0.5 & \text{nếu khác loại} \end{cases} \quad (3.9)$$

Khi khác loại, hình phạt phụ thuộc vào khoảng cách thời gian giữa hai công việc: g càng lớn, người dùng có càng nhiều thời gian phục hồi, hình phạt càng thấp. Khi $g \geq 23$ phút, hình phạt giảm về 0 vì đã đủ thời gian phục hồi hoàn toàn. Hệ số 0.5 giới hạn hình phạt tối đa ở mức 50%, phản ánh rằng chuyển đổi ngữ cảnh gây bất tiện nhưng không nghiêm trọng bằng vi phạm deadline hay thiếu năng lượng.

3.4.7 Hình phạt phân mảnh lịch trình - Fragmentation Penalty (P_{frag})

Thành phần này tránh tạo ra các khoảng trống nhỏ (dưới 30 phút) giữa slot và các sự kiện hiện có, vì các khoảng trống quá ngắn không đủ để làm việc hiệu quả nhưng vẫn chiếm thời gian trên lịch. Gọi E là tập hợp tất cả sự kiện hiện có và $T_{frag} = 30$ phút là ngưỡng phân mảnh. Với mỗi sự kiện $e \in E$, hệ thống kiểm tra hai khoảng cách:

- Khoảng trống *trước* slot: từ thời điểm kết thúc của e đến thời điểm bắt đầu của slot.
- Khoảng trống *sau* slot: từ thời điểm kết thúc của slot đến thời điểm bắt đầu của e .

Gọi $d(s, e)$ là khoảng cách thời gian (phút) cho mỗi cặp kiểm tra trên. Hàm chỉ thị phân mảnh được định nghĩa:

$$\delta(s, e) = \begin{cases} 0.2 & \text{nếu } 0 < d(s, e) < T_{frag} \\ 0 & \text{trường hợp còn lại} \end{cases} \quad (3.10)$$

Tổng hình phạt được giới hạn tối đa ở 1.0:

$$P_{frag}(s) = \min \left(1, \sum_{e \in E} \delta(s, e) \right) \quad (3.11)$$

Mỗi khoảng trống nhỏ đóng góp 0.2 vào hình phạt, nghĩa là cần ít nhất 5 khoảng trống nhỏ để đạt hình phạt tối đa. Trong thực tế, hình phạt 1.0 rất hiếm khi xảy ra vì đòi hỏi lịch trình phải có nhiều sự kiện ngăn xếp sát nhau xung quanh slot - giới hạn 1.0 chỉ đóng vai trò cận trên lý thuyết.

3.5 Thiết kế thuật toán lập lịch

3.5.1 Tìm khe thời gian khả dụng (Slot Finder)

Trước khi tính chi phí, hệ thống cần xác định tất cả các khe thời gian khả dụng trong khoảng tìm kiếm. Slot Finder hoạt động theo thuật toán quét (sweep) qua từng ngày trong khoảng tìm kiếm.

Với mỗi ngày, Slot Finder xác định cửa sổ làm việc (working window) dựa trên giờ thức dậy (wakeTime) và giờ ngủ (sleepTime) của người dùng. Nếu người dùng chỉ định thời gian ưa thích (preferred time), cửa sổ làm việc được thu hẹp về khoảng giao giữa giờ thức/ngủ và thời gian ưa thích, biến preferred time thành ràng buộc cứng (hard constraint) thay vì ràng buộc

mềm. Tiếp theo, các khe thời gian được sinh ra bằng cách trượt một cửa sổ có kích thước bằng thời lượng công việc qua cửa sổ làm việc, mỗi bước nhảy 15 phút (slot granularity). Mỗi khe được kiểm tra xung đột (overlap) với các sự kiện hiện có. Khe không xung đột được thêm vào danh sách kết quả. Khe bị xung đột mặc định sẽ bị loại bỏ. Tuy nhiên, trong chế độ cho phép xung đột (allowOverlap), khe bị xung đột vẫn được thêm nhưng được đánh dấu hasConflict kèm danh sách sự kiện bị ảnh hưởng. Chế độ này được kích hoạt trong hai trường hợp: (1) khi người dùng chủ động yêu cầu tạo công việc bất chấp xung đột (force create), và (2) như chiến lược dự phòng (fallback) - nếu lần tìm kiếm đầu tiên không tìm được khe nào khả dụng (lịch trình đã kín), hệ thống tự động thử lại với allowOverlap để vẫn trả về kết quả thay vì báo lỗi không có khe trống.

Toàn bộ tính toán thời gian được thực hiện trong múi giờ cục bộ của người dùng (sử dụng toZonedTime và fromZonedTime từ thư viện date-fns-tz), đảm bảo kết quả chính xác cho người dùng ở bất kỳ múi giờ nào.

3.5.2 Thuật toán tham lam (Greedy Scheduling)

Trong luồng sử dụng chính, thuật toán lập lịch xử lý **từng công việc một** (single-task scheduling): mỗi khi người dùng tạo công việc mới (qua giao diện hoặc AI chat), hệ thống tìm khe tốt nhất cho công việc đó dựa trên trạng thái lịch trình hiện tại. Quy trình gồm 4 bước:

Bước 1: Xác định khoảng tìm kiếm. Khoảng tìm kiếm được xác định dựa trên thông tin công việc: nếu có ngày ưa thích (preferredDate), chỉ tìm trong ngày đó; nếu có deadline nhưng không có ngày ưa thích, tìm từ hiện tại đến deadline; nếu không có cả hai, tìm trong 7 ngày tới (mặc định).

Bước 2: Tìm khe khả dụng. Slot Finder sinh ra tất cả khe thời gian khả dụng trong khoảng tìm kiếm, tôn trọng giờ thức/ngủ và thời gian ưa thích (nếu có).

Bước 3: Tính chi phí. Hàm chi phí 6 yếu tố được áp dụng cho từng khe, kết hợp thông tin chronotype, deadline, priority, và các sự kiện hiện có trên lịch.

Bước 4: Chọn khe tốt nhất. Khe có tổng chi phí thấp nhất được chọn. Nếu không tìm được khe nào (lịch đã kín), hệ thống thử lại với chế độ allowOverlap như đã trình bày ở phần Slot Finder.

Ngoài chế độ tự động chọn khe tốt nhất, hệ thống còn cung cấp chế độ đề xuất nhiều phương án (scheduling options), trả về top 3 khe thời gian tốt nhất kèm giải thích chi tiết để người dùng lựa chọn.

3.5.3 Giải thích kết quả lập lịch

Sau khi chọn khe thời gian, hệ thống sinh ra lời giải thích bằng ngôn ngữ tự nhiên (tiếng Anh) cho người dùng. Giải thích bao gồm thông tin về mức năng lượng tại thời điểm được chọn (dựa trên đường cong năng lượng và chronotype), nhận xét về priority nếu công việc quan trọng, cảnh báo deadline nếu gần hạn, và gợi ý thời gian nghỉ sau công việc. Ví dụ: “Perfect timing!

9:00 AM is when you're at your sharpest. As an early bird, your mornings are golden. Consider taking a 10-minute break afterwards to stay fresh.” (Tạm dịch: “Thời điểm hoàn hảo! 9:00 sáng là lúc bạn tỉnh táo nhất. Là người dậy sớm, buổi sáng là thời gian vàng của bạn. Hãy nghỉ 10 phút sau đó để duy trì sự tập trung.”)

3.6 Hệ thống trọng số thích ứng 3 lớp

3.6.1 Tổng quan kiến trúc

Trọng số cuối cùng cho mỗi thành phần hàm chi phí được tính bằng cách kết hợp 3 lớp. Gọi $i \in \{c, d, p, b, cs, f\}$ là chỉ số tương ứng với 6 thành phần hình phạt (chronotype, deadline, priority, buffer, context switch, fragmentation). Với mỗi thành phần i , trọng số cuối cùng được tính qua 3 bước:

Bước 1: Tổng hợp. Cộng trọng số từ 3 lớp:

$$w_i^{raw} = w_i^{preset} + \Delta w_i^{manual} + \Delta w_i^{auto} \quad (3.12)$$

trong đó:

- w_i^{preset} : trọng số cơ sở từ hồ sơ preset (Lớp 1), được chọn khi onboarding.
- Δw_i^{manual} : giá trị điều chỉnh thủ công của người dùng (Lớp 2), $\Delta w_i^{manual} \in [-0.05, +0.05]$.
- Δw_i^{auto} : giá trị điều chỉnh tự động từ thuật toán EMA (Lớp 3), $\Delta w_i^{auto} \in [-0.15, +0.15]$.

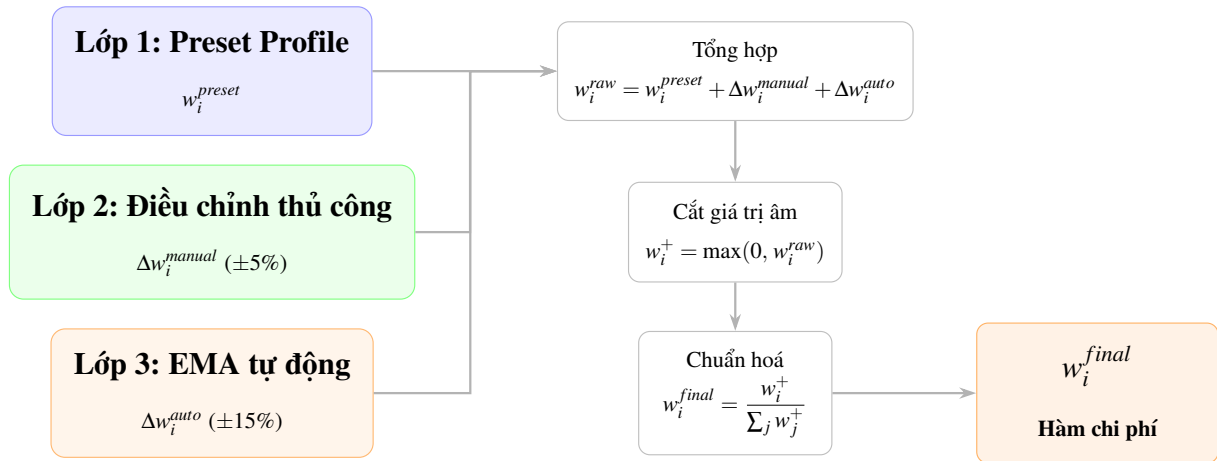
Bước 2: Cắt giá trị âm. Đảm bảo trọng số không âm (trọng số âm không có ý nghĩa vì sẽ biến hình phạt thành phần thưởng):

$$w_i^+ = \max(0, w_i^{raw}) \quad (3.13)$$

Bước 3: Chuẩn hóa. Chia mỗi trọng số cho tổng để đảm bảo $\sum w_i^{final} = 1$:

$$w_i^{final} = \frac{w_i^+}{\sum_j w_j^+} \quad (3.14)$$

Nếu $\sum_j w_j^+ = 0$ (trường hợp bất thường khi tất cả trọng số bị triệt tiêu), hệ thống sử dụng bộ trọng số mặc định của hồ sơ Balanced.



Hình 3.2: Hệ thống trọng số thích ứng 3 lớp

3.6.2 Lớp 1: Hồ sơ thiết lập trước (Preset Profile)

Lớp đầu tiên cung cấp bộ trọng số cơ sở w_i^{preset} được thiết kế sẵn cho 5 phong cách lập lịch khác nhau. Mỗi hồ sơ phản ánh một triết lý làm việc cụ thể, đảm bảo hệ thống hoạt động hợp lý ngay từ lần sử dụng đầu tiên mà không cần dữ liệu phản hồi. Người dùng chọn hồ sơ tại bước 7 của quy trình onboarding. Bảng 3.3 trình bày giá trị trọng số chi tiết của mỗi hồ sơ.

Bảng 3.3: Trọng số preset w_i^{preset} cho 5 hồ sơ lập lịch

Profile	w_c	w_d	w_p	w_b	w_{cs}	w_f
Hustle	0.15	0.35	0.25	0.05	0.10	0.10
Balanced	0.30	0.20	0.15	0.15	0.10	0.10
Wellness	0.35	0.10	0.10	0.25	0.10	0.10
Parent	0.25	0.20	0.20	0.25	0.05	0.05
Creative	0.30	0.15	0.05	0.05	0.25	0.20

Hustle (Năng suất cao): ưu tiên deadline ($w_d = 0.35$) và priority ($w_p = 0.25$), phù hợp với người làm việc theo deadline và cần hoàn thành nhiều việc nhanh. Buffer rất thấp ($w_b = 0.05$) cho thấy người dùng chấp nhận hy sinh thời gian nghỉ để tối ưu thành quả và năng suất.

Balanced (Cân bằng): phân bổ trọng số đều hơn, với chronotype ($w_c = 0.30$) là yếu tố quan trọng nhất, kết hợp cân bằng giữa deadline, priority, và buffer. Đây là hồ sơ mặc định khi người dùng bỏ qua bước chọn hồ sơ.

Wellness (Sức khỏe): đặt trọng tâm vào năng lượng sinh học ($w_c = 0.35$) và thời gian nghỉ ($w_b = 0.25$), phù hợp với người ưu tiên sức khỏe tinh thần và muốn tránh làm việc quá sức.

Parent (Phụ huynh): cân bằng giữa buffer ($w_b = 0.25$), chronotype ($w_c = 0.25$), deadline ($w_d = 0.20$) và priority ($w_p = 0.20$), phản ánh nhu cầu linh hoạt và có thời gian nghỉ cho việc gia đình.

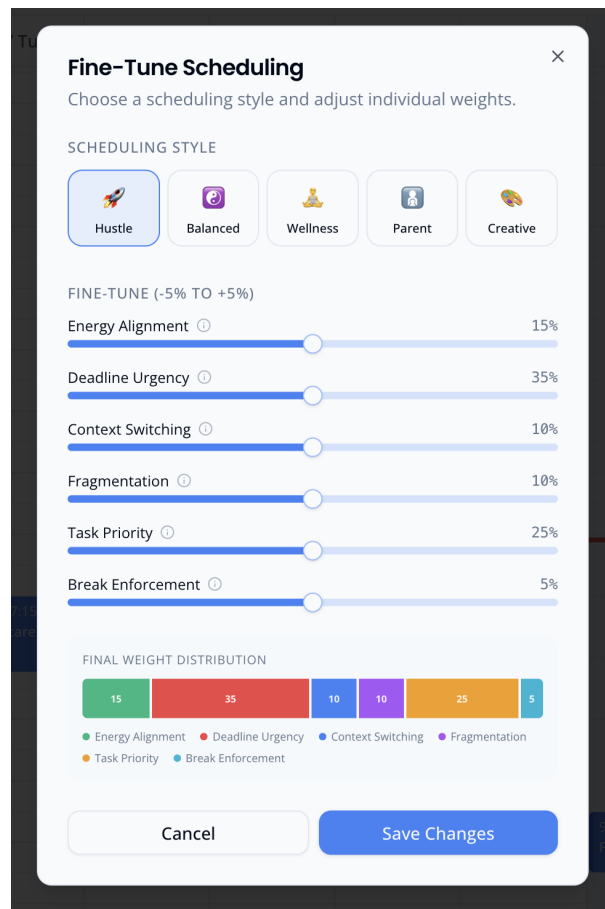
Creative (Sáng tạo): đặt trọng tâm vào context switch ($w_{cs} = 0.25$) và fragmentation ($w_f = 0.20$), giúp nhóm các công việc cùng loại lại gần nhau và tạo các khối thời gian dài liên

tục cho dòng chảy sáng tạo (creative flow).

3.6.3 Lớp 2: Điều chỉnh thủ công $\Delta w_i^{manual} (\pm 5\%)$

Lớp thứ hai cho phép người dùng tinh chỉnh từng thành phần trọng số thông qua giao diện thanh trượt (slider) trong modal “Fine-Tune Scheduling” trên sidebar. Mỗi thành phần có một giá trị điều chỉnh $\Delta w_i^{manual} \in [-0.05, +0.05]$, mặc định là 0 (không điều chỉnh).

Biên độ $\pm 5\%$ được chọn có chủ đích: đủ lớn để người dùng cảm nhận sự khác biệt trong kết quả lập lịch, nhưng đủ nhỏ để không phá vỡ cấu trúc trọng số tổng thể của hồ sơ preset. Ví dụ, với hồ sơ Balanced ($w_c^{preset} = 0.30$), người dùng có thể điều chỉnh chronotype trong khoảng $[0.25, 0.35]$, thay đổi tương đối khoảng $\pm 17\%$ so với giá trị gốc.



Hình 3.3: Giao diện modal Fine-Tune Scheduling với 6 thanh trượt điều chỉnh thủ công trọng số.

3.6.4 Lớp 3: Tự động điều chỉnh EMA $\Delta w_i^{auto} (\pm 15\%)$

Lớp thứ ba sử dụng thuật toán trung bình trượt hàm mũ (Exponential Moving Average - EMA) để tự động điều chỉnh trọng số dựa trên phản hồi ngầm định (implicit feedback) của người dùng. Quá trình gồm 3 bước:

Bước 1: Thu thập tín hiệu phản hồi. Hệ thống thu thập 5 loại tín hiệu ngầm định từ hành vi người dùng, như đã trình bày ở Mục 2.5.3. Mỗi tín hiệu tạo ra một giá trị $signal_i \in [0, 1]$ cho một hoặc nhiều thành phần trọng số, trong đó 1 biểu thị hài lòng hoàn toàn và 0 biểu thị không

hài lòng.

Bước 2: Cập nhật điểm hài lòng qua EMA. Mỗi thành phần i duy trì một điểm hài lòng (satisfaction score) $sat_i \in [0, 1]$, khởi tạo bằng 0.5 (trung tính). Khi nhận tín hiệu mới, sat_i được cập nhật:

$$sat_i^{new} = (1 - \alpha) \cdot sat_i^{old} + \alpha \cdot signal_i \quad (3.15)$$

với hệ số làm mượt $\alpha = 0.3$. Giá trị $\alpha = 0.3$ cho phép tín hiệu mới chiếm 30% trọng số, trong khi 70% còn lại phản ánh xu hướng tích lũy từ trước. Điều này giúp hệ thống phản ứng tương đối nhanh với thay đổi hành vi nhưng không bị dao động mạnh bởi tín hiệu bất thường.

Bước 3: Tính giá trị điều chỉnh tự động. Từ điểm hài lòng, giá trị điều chỉnh được tính:

$$\Delta w_i^{auto} = \text{clamp}(sat_i - 0.5, -0.15, +0.15) \quad (3.16)$$

trong đó $\text{clamp}(x, a, b) = \max(a, \min(b, x))$ giới hạn giá trị x trong khoảng $[a, b]$, đảm bảo điều chỉnh tự động không vượt quá biên độ cho phép.

Ý nghĩa: giá trị $sat_i = 0.5$ là điểm cân bằng, tại đó $sat_i - 0.5 = 0$ nên $\Delta w_i^{auto} = 0$ (không điều chỉnh). Khi $sat_i > 0.5$ (tín hiệu cho thấy người dùng hài lòng với yếu tố này), trọng số tăng (tối đa +0.15), nghĩa là hệ thống sẽ ưu tiên yếu tố này hơn. Khi $sat_i < 0.5$ (không hài lòng), trọng số giảm (tối đa -0.15), giảm ảnh hưởng của yếu tố này. Ví dụ: nếu qua nhiều lần sử dụng, sat_{buffer} tăng lên 0.65, thì $\Delta w_{buffer}^{auto} = 0.65 - 0.5 = 0.15$, đạt mức điều chỉnh tối đa.

Biên độ $\pm 15\%$ gấp 3 lần biên độ thử công ($\pm 5\%$). Lý do là vì điều chỉnh thủ công là quyết định tức thời của người dùng tại một thời điểm, nên cần giới hạn chặt để tránh thay đổi quá mạnh do thử nghiệm bốc đồng. Trong khi đó, điều chỉnh EMA là kết quả tích lũy từ nhiều lần phản hồi qua thời gian, mỗi tín hiệu chỉ dịch chuyển sat_i một lượng nhỏ (30% theo α), nên cần biên độ lớn hơn để phản ánh xu hướng dài hạn thực sự của người dùng.

3.7 Thiết kế trợ lý AI Chat

3.7.1 Kiến trúc đường ống xử lý (Pipeline)

Tính năng trợ lý AI Chat cho phép người dùng tương tác bằng ngôn ngữ tự nhiên để tạo, truy vấn, và quản lý lịch trình. Kiến trúc đường ống xử lý (pipeline) gồm 4 thành phần chính.

Câu lệnh hệ thống (System Prompt) được tạo ra dựa trên hồ sơ người dùng, bao gồm thông tin chronotype, giờ thức/ngủ, múi giờ, và danh sách sự kiện 7 ngày tới. Điều này cho phép mô hình ngôn ngữ lớn (Large Language Model - LLM) hiểu ngữ cảnh lịch trình hiện tại để đưa ra gợi ý phù hợp.

OpenRouter API đóng vai trò cổng trung gian (proxy) để truy cập mô hình ngôn ngữ. Hệ thống sử dụng OpenRouter thay vì gọi trực tiếp API của nhà cung cấp mô hình, cho phép dễ dàng chuyển đổi giữa các mô hình (model switching) mà không thay đổi mã nguồn. Giao tiếp sử

dùng sự kiện phía máy chủ (Server-Sent Events - SSE) để truyền kết quả theo dòng (streaming), giúp người dùng thấy phản hồi ngay lập tức thay vì phải chờ toàn bộ câu trả lời.

Gọi hàm (Function Calling) cho phép mô hình ngôn ngữ kích hoạt các chức năng của hệ thống. Khi mô hình nhận diện ý định (intent) của người dùng (ví dụ “lên lịch viết báo cáo 2 tiếng chiều mai”), nó tạo một lệnh gọi hàm (function call) với tham số phù hợp. Lệnh này được gửi về phía máy khách (client) trước, hiển thị xác nhận cho người dùng (ví dụ “Lên lịch Viết báo cáo, 120 phút, ngày mai?”), và chỉ thực thi khi người dùng đồng ý.

Bộ thực thi công cụ (Tool Executor) nhận lệnh gọi hàm đã được xác nhận, thực hiện đồng bộ tăng dần với Google Calendar để đảm bảo dữ liệu mới nhất, sau đó gọi bộ máy lập lịch (scheduling engine) hoặc thao tác CRUD (tạo, đọc, cập nhật, xóa) tương ứng.

3.7.2 Các công cụ hàm (Function Tools)

Trợ lý AI Chat cung cấp 7 công cụ hàm (function tools) tương ứng với 7 thao tác chính.

Bảng 3.4: Danh sách công cụ hàm trong trợ lý AI Chat

Công cụ	Mô tả
schedule_task	Tạo công việc mới và lập lịch tự động. Tham số: tiêu đề, loại, thời lượng, ngày, thời gian ưa thích, độ ưu tiên, hạn chót
query_schedule	Truy vấn sự kiện trong khoảng ngày, hiển thị lịch trình và khe trống
reschedule_event	Dời sự kiện sang ngày/giờ mới
delete_event	Xóa sự kiện khỏi lịch và Google Calendar
resize_task	Thay đổi thời lượng sự kiện
complete_task	Đánh dấu công việc hoàn thành
get_scheduling_options	Hiển thị top 3 khe thời gian tốt nhất kèm giải thích để người dùng chọn

Mỗi thao tác qua trợ lý AI Chat đều kích hoạt thu thập tín hiệu phản hồi ngầm định (implicit feedback). Ví dụ, khi người dùng yêu cầu dời lịch qua chat, hệ thống tính toán tín hiệu dời lịch ($S_{reschedule}$) và cập nhật điểm hài lòng EMA tương ứng. Khi người dùng chọn 1 trong 3 phương án từ `get_scheduling_options`, hệ thống tính tín hiệu lựa chọn phương án ($S_{option,i}$) dựa trên sự khác biệt giữa phương án được chọn và phương án tối ưu.

3.7.3 Giới hạn tốc độ (Rate Limiting)

Để bảo vệ hệ thống khỏi lạm dụng và kiểm soát chi phí API, tính năng trợ lý AI Chat áp dụng giới hạn 10 tin nhắn mỗi phút cho mỗi người dùng. Khi vượt giới hạn, hệ thống trả về mã lỗi 429 (quá nhiều yêu cầu) kèm thời gian chờ cụ thể.

3.8 Đồng bộ Google Calendar

Hệ thống đồng bộ hai chiều (two-way sync) với Google Calendar được thiết kế theo 3 chế độ hoạt động, đảm bảo dữ liệu giữa CHRONIX và Google Calendar luôn nhất quán.

Chế độ 1: Đồng bộ ban đầu (Initial Sync). Chế độ này được thực hiện một lần khi người dùng hoàn thành quy trình khởi tạo (onboarding), hoặc khi người dùng chủ động yêu cầu làm mới toàn bộ dữ liệu. Hệ thống gọi Google Calendar API để lấy tất cả sự kiện trong cửa sổ thời gian 28 ngày (14 ngày trước và 14 ngày tới), giới hạn tối đa 250 sự kiện mỗi lần gọi.

Chiến lược đồng bộ sử dụng phương pháp “xóa rồi chèn lại” (delete-then-reinsert), gồm 4 bước:

- **Lưu liên kết:** Trước khi xóa, hệ thống lưu lại ánh xạ giữa công việc (task) và sự kiện (event) để khôi phục sau.
- **Xóa sạch:** Xóa toàn bộ sự kiện có googleEventId trong cửa sổ thời gian khỏi cơ sở dữ liệu cục bộ.
- **Chèn lại theo lô (batch insert):** Chèn lại toàn bộ sự kiện từ Google theo lô 50 bản ghi, sử dụng thao tác onConflictDoUpdate (chèn mới hoặc cập nhật nếu đã tồn tại) để xử lý trường hợp trùng lặp.
- **Khôi phục liên kết:** Gán lại liên kết task-event đã lưu ở bước 1.

Phương pháp này đơn giản và đáng tin cậy hơn so với cập nhật từng sự kiện riêng lẻ, vì tránh được các trường hợp biên khi dữ liệu cục bộ và Google không khớp. Sau khi đồng bộ xong, hệ thống lưu syncToken (mã đồng bộ) từ Google để sử dụng cho chế độ đồng bộ tăng dần.

Chế độ 2: Đồng bộ tăng dần (Incremental Sync). Thay vì lấy lại toàn bộ sự kiện, chế độ này sử dụng syncToken để chỉ lấy những thay đổi kể từ lần đồng bộ trước. Điều này giảm đáng kể lượng dữ liệu truyền tải và thời gian xử lý, một yêu cầu đồng bộ tăng dần thường chỉ trả về vài sự kiện thay đổi thay vì hàng trăm sự kiện.

Hệ thống có cơ chế điều tiết tần suất (throttling): nếu lần đồng bộ gần nhất diễn ra trong vòng 60 giây, yêu cầu mới sẽ bị bỏ qua để tránh gọi API quá nhiều. Khi syncToken hết hạn (Google trả về mã lỗi 410 Gone), hệ thống tự động quay về chế độ đồng bộ ban đầu để lấy lại toàn bộ dữ liệu.

Các thay đổi nhận được từ Google được phân loại thành 2 nhóm xử lý riêng:

- **Sự kiện bị hủy** (trạng thái “cancelled”): xóa khỏi cơ sở dữ liệu cục bộ, đồng thời xóa liên kết task-event nếu có.
- **Sự kiện tạo mới hoặc cập nhật:** chèn mới hoặc cập nhật thông tin (tiêu đề, thời gian, địa điểm, v.v.) vào cơ sở dữ liệu cục bộ.

Chế độ 3: Đẩy ngược (Push to Google). Chế độ này hoạt động theo chiều ngược lại. Khi sự kiện được tạo hoặc cập nhật từ phía CHRONIX (qua giao diện hoặc trợ lý AI Chat), sự kiện cần được đẩy lên Google Calendar. Quy trình thực hiện theo chiến lược “lưu cục bộ trước, đẩy

lên sau” (local-first):

- Sự kiện được lưu vào cơ sở dữ liệu cục bộ ngay lập tức với trạng thái `pending_push` (chờ đẩy lên).
- Sau đó, hệ thống đẩy sự kiện lên Google Calendar chạy ngầm (background): nếu sự kiện đã tồn tại trên Google (có `googleEventId`), gọi `events.update` (cập nhật); nếu là sự kiện mới, gọi `events.insert` (tạo mới).
- Sau khi đẩy thành công, cập nhật `googleEventId` (mã sự kiện trên Google), `etag` (phiên bản sự kiện), và đổi `syncStatus` thành `synced` (đã đồng bộ).

Chiến lược local-first đảm bảo người dùng không phải chờ Google phản hồi, sự kiện xuất hiện trên lịch ngay lập tức, việc đồng bộ với Google diễn ra không đồng bộ (asynchronous) phía sau.

3.9 Tóm tắt

Chương này đã trình bày chi tiết thiết kế hệ thống CHRONIX với các thành phần chính sau.

Thứ nhất, kiến trúc modular monolith sử dụng Next.js 15 với 4 tầng (giao diện, xử lý, dữ liệu, tích hợp) cung cấp nền tảng vững chắc cho việc phát triển và bảo trì, kết hợp ưu điểm của monolith (đơn giản, dễ triển khai) với tính tổ chức của kiến trúc module (6 module chức năng độc lập).

Thứ hai, cơ sở dữ liệu PostgreSQL với 10 bảng được thiết kế tối ưu cho cả lưu trữ dữ liệu lẫn hiệu năng truy vấn, sử dụng chỉ mục kết hợp, ràng buộc unique, và kiểu enum để đảm bảo tính toàn vẹn dữ liệu.

Thứ ba, hàm chi phí 6 yếu tố kết hợp thuật toán tham lam tạo thành scheduling engine có khả năng cân bằng đa mục tiêu: năng lượng sinh học, deadline, priority, khoảng nghỉ, chuyển đổi ngữ cảnh, và phân mảnh lịch trình.

Thứ tư, hệ thống trọng số 3 lớp (preset + thủ công + EMA) cho phép cá nhân hóa sâu, vừa hoạt động tốt ngay từ đầu với preset profile, vừa tự động cải thiện theo thời gian qua phản hồi ngầm định.

Thứ năm, AI Chat với 7 function tools và đồng bộ Google Calendar hai chiều cung cấp trải nghiệm người dùng hoàn chỉnh, cho phép quản lý lịch trình bằng cả giao diện đồ họa lẫn ngôn ngữ tự nhiên.

Chương tiếp theo sẽ trình bày chi tiết quá trình triển khai (implementation) các thiết kế này bằng mã nguồn cụ thể.

Chương 4

Triển khai

Chương này trình bày chi tiết cách triển khai hệ thống CHRONIX từ thiết kế ở Chương 3. Mỗi phần mô tả mã nguồn thực tế, giải thích các quyết định kỹ thuật, và minh họa luồng dữ liệu giữa các thành phần.

4.1 Công nghệ sử dụng

Bảng 4.1: Công nghệ và thư viện chính

Thành phần	Công nghệ	Phiên bản
Framework	Next.js (App Router)	15.x
Thư viện giao diện	React	19.x
Ngôn ngữ	TypeScript	5.7
Giao diện	Tailwind CSS + shadcn/ui	3.4
Quản lý trạng thái	Zustand (phía máy khách) + TanStack Query (phía máy chủ)	5.x
ORM	Drizzle ORM	0.41
Cơ sở dữ liệu	PostgreSQL (Supabase)	—
Xác thực	Better Auth + Google OAuth	1.4
Lịch	FullCalendar.js	6.1
AI	OpenRouter	—
Kiểm thử	Vitest	4.0

- Lý do chọn các công nghệ chính:
- **Next.js 15 App Router:** Hỗ trợ Server Components (thành phần phía máy chủ) giúp tải dữ liệu trực tiếp trên máy chủ mà không cần API trung gian. Server Actions (hành động phía máy chủ) cho phép gọi hàm phía máy chủ từ giao diện mà không cần viết REST endpoint riêng.
 - **Drizzle ORM:** ORM kiểu TypeScript-first, tạo truy vấn SQL an toàn kiểu dữ liệu (type safe) tại thời điểm biên dịch. Hỗ trợ `onConflictDoUpdate` cho thao tác upsert (chèn hoặc

cập nhật) cần thiết trong đồng bộ lịch.

- **Better Auth:** Thư viện xác thực framework-agnostic (không phụ thuộc framework), quản lý phiên đăng nhập qua httpOnly cookie, hỗ trợ OAuth 2.0 với Google.
- **OpenRouter:** Cổng trung gian (proxy) cho phép truy cập nhiều mô hình ngôn ngữ lớn (LLM) qua một API duy nhất, dễ dàng chuyển đổi mô hình khi cần.

4.2 Cấu trúc dự án

Dự án được tổ chức theo kiến trúc dựa trên tính năng (feature-based architecture), mỗi tính năng là một thư mục độc lập chứa logic, giao diện, và hành động phía máy chủ riêng:

Đoạn mã 4.1: Cấu trúc thư mục chính

```

1 src/
2   app/                                # Next.js App Router
3     api/                              # REST API endpoints
4       chat/route.ts                  # AI chat streaming endpoint
5       cron/                          # Cron jobs (dong bo lich)
6       dashboard/                     # Trang dashboard chinh
7       onboarding/                    # Trang khao sat chronotype
8       auth/                          # Trang dang nhap
9   features/                           # Cac module tinh nang
10     scheduling/                      # Scheduling engine
11     lib/                             # cost-function, energy-curves,
12                                     # slot-finder, weight-profiles,
13                                     # feedback-processor, scheduler
14     actions/                         # Server actions (schedule-task,
15                                     # weight-actions)
16   chat/                              # AI chat
17     lib/system-prompt                # Cau lenh he thong
18     hooks/use-chat                   # React hook cho chat
19   calendar/                          # Google Calendar sync
20     actions/sync                     # Dong bo 2 chieu
21   onboarding/                        # Khao sat chronotype
22     lib/chronotype                  # Tinh toan chronotype
23     components/                     # Wizard 7 buoc
24   db/                                # Drizzle schema + connection
25   lib/                               # Tien ich dung chung
26     google-calendar/                # Google Calendar client + sync
27     openrouter/tools                 # Dinh nghia 7 cong cu AI
28     rate-limit                       # Gioi han tan suat
29   store/                             # Zustand stores

```

Điểm đáng chú ý trong cấu trúc:

- **Tách biệt lib và actions:** Thư mục lib/ chứa hàm thuần (pure function) không phụ thuộc cơ sở dữ liệu, có thể kiểm thử đơn vị (unit test) độc lập. Thư mục actions/ chứa server action gọi cơ sở dữ liệu.
- **Scheduling engine thuần túy:** Toàn bộ scheduling/lib/ gồm 6 tệp TypeScript thuần, không import trực tiếp từ cơ sở dữ liệu. Điều này cho phép bộ kiểm chuẩn (benchmark suite) chạy hoàn toàn ngoài máy chủ.

4.3 Triển khai Scheduling Engine (Bộ lập lịch)

4.3.1 Đường cong năng lượng (Energy Curves)

Mỗi chronotype được định nghĩa bởi một cấu trúc EnergyCurve gồm 3 thành phần: mảng 24 giá trị năng lượng theo giờ (0–100), cửa sổ đỉnh năng lượng (peak window), và cửa sổ suy giảm năng lượng (dip window):

Đoạn mã 4.2: Cấu trúc đường cong năng lượng

```
1 interface EnergyCurve {
2   chronotype: Chronotype;
3   hourlyEnergy: number[]; // 24 giá trị, 0-100
4   peakWindow: { start: number; end: number };
5   dipWindow: { start: number; end: number };
6 }
```

Bảng 4.2: Cửa sổ đỉnh và suy giảm năng lượng theo chronotype

Chronotype	Đỉnh (Peak)	Suy giảm (Dip)
Lion (Sư tử)	6:00 – 10:00	13:00 – 15:00
Bear (Gấu)	10:00 – 14:00	14:00 – 16:00
Wolf (Sói)	17:00 – 21:00	8:00 – 10:00
Dolphin (Cá heo)	10:00 – 12:00	14:00 – 16:00

Hàm getAverageEnergyForSlot() tính năng lượng trung bình cho một khoảng thời gian bằng cách lấy trung bình cộng năng lượng của từng giờ mà slot bao phủ:

Đoạn mã 4.3: Tính năng lượng trung bình cho khe thời gian

```
1 function getAverageEnergyForSlot(
2   chronotype: Chronotype,
3   startHour: number,
4   durationMinutes: number
5 ): number {
6   const endHour = startHour + Math.ceil(
7     durationMinutes / 60
```

```

8   );
9   let total = 0, count = 0;
10  for (let h = Math.floor(startHour);
11       h <= endHour && h < 24; h++) {
12      total += getEnergyAtHour(chronotype, h);
13      count++;
14  }
15  return count > 0 ? total / count : 50;
16 }

```

4.3.2 Hàm chi phí (Cost Function)

Hàm `calculateSlotCost()` là hàm cốt lõi của scheduling engine. Nó nhận đầu vào gồm khe thời gian (slot), thông tin công việc (task), ngữ cảnh lập lịch (scheduling context), và trọng số tùy chỉnh (custom weights):

Đoạn mã 4.4: Giao diện hàm chi phí – đầu vào

```

1 function calculateSlotCost(
2   slot: TimeSlot,
3   task: {
4     category: TaskCategory;
5     deadline: Date | null;
6     durationMinutes: number;
7     priority: number;
8   },
9   context: SchedulingContext,
10  weights?: CostWeights // Tu he thong trong so
11 ): SlotScore

```

Kết quả trả về bao gồm tổng chi phí và bảng phân tích 6 thành phần:

Đoạn mã 4.5: Giao diện hàm chi phí – đầu ra

```

1 interface SlotScore {
2   slot: TimeSlot;
3   totalCost: number; // Tong chi phi [0, 1]
4   breakdown: {
5     chronotypePenalty: number; // Nang luong
6     deadlineRisk: number; // Deadline
7     contextSwitchCost: number; // Chuyen doi
8     fragmentationPenalty: number; // Phan manh
9     priorityPenalty: number; // Do uu tien
10    bufferPenalty: number; // Khoang nghi
11  };

```

12 }

Tổng chi phí được tính bằng tổng có trọng số (weighted sum) của 6 thành phần:

Đoạn mã 4.6: Tính tổng chi phí

```
1 const totalCost =
2   w.chronotype * chronotypePenalty +
3   w.deadline * deadlineRisk +
4   w.contextSwitch * contextSwitchCost +
5   w.fragmentation * fragmentationPenalty +
6   w.priority * priorityPenalty +
7   w.buffer * bufferPenalty;
```

Trong đó w là đối tượng trọng số từ hệ thống 3 lớp (xem Mục 4.4). Nếu không truyền trọng số, hệ thống sử dụng bộ trọng số mặc định `balanced`.

Hình phạt năng lượng sinh học

Hàm `calculateChronotypePenalty()` chuyển đổi thời gian slot sang múi giờ (timezone) của người dùng trước khi tra cứu năng lượng, đảm bảo kết quả chính xác cho mọi vị trí địa lý:

Đoạn mã 4.7: Hình phạt năng lượng sinh học

```
1 function calculateChronotypePenalty(
2   slot, taskCategory, chronotype, timezone
3 ) {
4   // Chuyển sang múi giờ người dùng
5   const zonedTime = toZonedTime(slot.start, timezone);
6   const slotHour = zonedTime.getHours();
7
8   const available = getAverageEnergyForSlot(
9     chronotype, slotHour, duration
10  );
11  const required =
12    TASK_ENERGY_REQUIREMENTS[taskCategory];
13
14  // Chi phạt khi yêu cầu > khả dụng
15  return Math.max(0, required - available) / 100;
16 }
```

Yêu cầu năng lượng theo loại công việc:

- analytical (phân tích): 90 - Viết code, debug, phân tích dữ liệu, giải toán
- learning (học tập): 80 - Đọc tài liệu, học kỹ năng mới, nghiên cứu
- creative (sáng tạo): 70 - Thiết kế giao diện, viết bài, brainstorm ý tưởng
- physical (vận động): 60 - Họp trực tiếp, thuyết trình, tập thể dục

- administrative (hành chính): 40 - Soạn email, sắp xếp tài liệu, cập nhật báo cáo

Hình phạt deadline

Hàm `calculateDeadlineRisk()` kết hợp khoảng cách đến deadline với hệ số khẩn cấp theo độ ưu tiên (priority). Công việc ưu tiên cao gần deadline sẽ bị phạt nặng hơn:

Đoạn mã 4.8: Hình phạt deadline

```
1 function calculateDeadlineRisk(  
2   slot, deadline, priority  
3 ) {  
4   if (!deadline) return 0;  
5  
6   const slackHours = differenceInHours(  
7     deadline, slot.end  
8   );  
9   if (slackHours < 0) return 1; // Qua deadline  
10  
11  // Suy giảm tuyến tính trong 1 tuần  
12  const maxSlack = 168; // 1 tuần = 168 giờ  
13  const baseRisk = Math.max(  
14    0, 1 - slackHours / maxSlack  
15  );  
16  
17  // Hệ số khẩn cấp theo độ ưu tiên  
18  // Priority 5 = 1.4x, Priority 3 = 1.0x  
19  const urgency = 1 + (priority - 3) * 0.2;  
20  return Math.min(1, baseRisk * urgency);  
21 }
```

Chi phí chuyển đổi ngữ cảnh

Dựa trên nghiên cứu về thời gian phục hồi 23 phút sau mỗi lần chuyển đổi tác vụ (Mark et al., 2008):

Đoạn mã 4.9: Chi phí chuyển đổi ngữ cảnh

```
1 function calculateContextSwitchCost(  
2   taskCategory, previousCategory, gapMinutes  
3 ) {  
4   if (!previousCategory) return 0;  
5   if (taskCategory === previousCategory) return 0;  
6  
7   const basePenalty = 23; // phút phục hồi  
8   const recovery = gapMinutes
```



```

9      ? Math.min(1, gapMinutes / basePenalty)
10     : 0;
11     return (1 - recovery) * 0.5;
12 }

```

Nếu hai công việc liên tiếp cùng loại (ví dụ: cả hai đều là analytical), chi phí chuyển đổi bằng 0. Nếu khác loại nhưng khoảng cách giữa chúng ≥ 23 phút, chi phí cũng giảm về 0 do não đã phục hồi đủ.

Hình phạt phân mảnh

Hàm calculateFragmentationPenalty() kiểm tra khoảng trống trước và sau slot. Mỗi khoảng trống < 30 phút được cộng thêm 0.2 điểm phạt, tổng giới hạn tại 1.0:

Đoạn mã 4.10: Hình phạt phân mảnh

```

1 function calculateFragmentationPenalty(
2   slot, existingEvents
3 ) {
4   let penalty = 0;
5   for (const event of existingEvents) {
6     const gapBefore = differenceInMinutes(
7       slot.start, event.end
8     );
9     if (gapBefore > 0 && gapBefore < 30)
10      penalty += 0.2;
11
12     const gapAfter = differenceInMinutes(
13       event.start, slot.end
14     );
15     if (gapAfter > 0 && gapAfter < 30)
16      penalty += 0.2;
17   }
18   return Math.min(penalty, 1);
19 }

```

Hình phạt khoảng nghỉ

Hàm calculateBufferPenalty() đánh giá khoảng cách giữa slot và sự kiện trước đó. Khoảng nghỉ lý tưởng là 15 phút, tối thiểu 5 phút:

Đoạn mã 4.11: Hình phạt khoảng nghỉ

```

1 function calculateBufferPenalty(
2   slot, existingEvents
3 ) {

```

```
4   const idealBuffer = 15; // phut
5   const minBuffer = 5;
6   let maxPenalty = 0;
7
8   for (const event of existingEvents) {
9       const gap = differenceInMinutes(
10          slot.start, event.end
11      );
12      if (gap >= 0 && gap < idealBuffer) {
13          let penalty;
14          if (gap < minBuffer) {
15              penalty = 1.0; // Gan lien tuc
16          } else {
17              // Noi suy tuyen tinh 5-15 phut
18              penalty = 1 - (gap - minBuffer)
19                  / (idealBuffer - minBuffer);
20          }
21          maxPenalty = Math.max(maxPenalty, penalty);
22      }
23  }
24  return maxPenalty;
25 }
```

Hình phạt độ ưu tiên

Hàm `calculatePriorityPenalty()` đơn giản nhất, nó ánh xạ tuyến tính từ thang điểm 1 - 5 sang phạt 1.0 - 0.0:

Đoạn mã 4.12: Hình phạt độ ưu tiên

```
1 function calculatePriorityPenalty(priority) {
2     const clamped = Math.max(1, Math.min(5, priority));
3     return (5 - clamped) / 4;
4     // Priority 5 -> 0.00 (duoc slot tot nhat)
5     // Priority 3 -> 0.50 (trung tinh)
6     // Priority 1 -> 1.00 (nhan slot con lai)
7 }
```

4.3.3 Bộ tìm khe thời gian (Slot Finder)

Hàm `findAvailableSlots()` tạo danh sách tất cả khe thời gian khả dụng trong một khoảng ngày. Đây là bước đầu tiên trong pipeline lập lịch, phải lọc ra các slot hợp lệ trước khi tính chi phí.

Đoạn mã 4.13: Giao diện bộ tìm khe thời gian

```

1 function findAvailableSlots(
2   dateRange: { start: Date; end: Date },
3   existingEvents: { start: Date; end: Date }[],
4   options: {
5     wakeTime: string;    // "07:00"
6     sleepTime: string;   // "23:00"
7     minSlotDuration: number; // phút
8     slotGranularity: number; // 15 phút
9     preferredTime?: PreferredTimeWindow;
10    allowOverlap?: boolean;
11    timezone?: string;
12  }
13 ): TimeSlot []

```

Thuật toán thực hiện các bước sau cho mỗi ngày trong khoảng tìm kiếm:

1. **Xác định cửa sổ làm việc:** Từ wakeTime đến sleepTime trong múi giờ người dùng. Xử lý trường hợp giờ ngủ qua nửa đêm (ví dụ: Wolf ngủ lúc 01:00 sáng).
2. **Áp dụng ràng buộc cứng thời gian ưa thích:** Nếu preferredTime được chỉ định, thu hẹp cửa sổ làm việc về đúng khoảng đó. Ví dụ: người dùng nói “buổi sáng” thì chỉ tìm slot từ 06:00 đến 12:00.
3. **Bỏ qua thời gian đã qua:** Nếu đang ở ngày hôm nay và thời điểm hiện tại đã qua wakeTime, bắt đầu từ slot tiếp theo (làm tròn lên theo slotGranularity).
4. **Tạo slot theo bước 15 phút:** Duyệt từ đầu đến cuối cửa sổ, tạo slot có độ dài minSlotDuration. Với mỗi slot, kiểm tra xung đột với sự kiện hiện có bằng hàm areIntervalsOverlapping().
5. **Xử lý xung đột:**
 - Không xung đột: thêm slot bình thường
 - Có xung đột và allowOverlap = true: thêm slot với cờ hasConflict = true kèm danh sách sự kiện xung đột
 - Có xung đột và allowOverlap = false: bỏ qua slot

Sau khi tìm slot, hàm mergeAdjacentSlots() gộp các slot liên tiếp (cách nhau ≤ 1 phút) thành khối lớn hơn, và filterSlotsByDuration() lọc chỉ giữ các khối đủ dài cho công việc.

4.3.4 Luồng lập lịch đơn tác vụ (Single-Task Scheduling)

Hàm suggestBestSlot() là điểm vào chính (entry point) của scheduling engine. Nó thực hiện lập lịch cho **một công việc duy nhất** mỗi lần gọi:

Đoạn mã 4.14: Luồng lập lịch đơn tác vụ

```

1 function suggestBestSlot(

```

```

2   task, profile, existingEvents, options, weights
3 ): ScheduleResult | null {
4   // Bước 1: Tìm slot KHÔNG xung đột
5   let results = scheduleTasksGreedy(
6     [task], profile, existingEvents,
7     { lookAheadDays: 3 }, weights
8   );
9
10  // Bước 2: Nếu không tìm thấy và cho phép xung đột
11  if (results.length === 0 && options.allowOverlap) {
12    results = scheduleTasksGreedyWithOverlap(
13      [task], profile, existingEvents,
14      { lookAheadDays: 3 }, weights
15    );
16  }
17
18  return results[0] ?? null;
19 }

```

Bên trong `scheduleTasksGreedy()`, thuật toán tham lam (greedy) thực hiện:

1. **Xác định khoảng tìm kiếm:** Nếu có `preferredDate`, chỉ tìm trong ngày đó. Nếu có `deadline`, tìm từ bây giờ đến `deadline`. Mặc định: 3 ngày tới.
2. **Tìm slot khả dụng:** Gọi `findAvailableSlots()` với các ràng buộc.
3. **Chấm điểm:** Gọi `calculateSlotCost()` cho từng slot với trọng số thích ứng của người dùng.
4. **Chọn slot tốt nhất:** Slot có `totalCost` thấp nhất.
5. **Tạo gợi ý nghỉ ngơi:** Tính thời lượng nghỉ theo tỷ lệ 52:17 bằng hàm `calculateBreakDuration()`, giới hạn trong khoảng 5–20 phút.
6. **Tạo giải thích:** Hàm `generateExplanation()` tạo văn bản giải thích bằng tiếng Anh, nêu lý do chọn slot dựa trên `chronotype`, `deadline`, và năng lượng.

Đoạn mã 4.15: Tính thời lượng nghỉ theo tỷ lệ 52:17

```

1 function calculateBreakDuration(workMinutes) {
2   const breakMinutes = Math.round(
3     workMinutes * (17 / 52)
4   );
5   return Math.max(5, Math.min(20, breakMinutes));
6   // 30 phút làm -> 10 phút nghỉ
7   // 60 phút làm -> 20 phút nghỉ (giới hạn)
8   // 90 phút làm -> 20 phút nghỉ (giới hạn)
9 }

```

4.3.5 Hành động phía máy chủ: `scheduleTask`

Hàm `scheduleTask()` trong `schedule-task.ts` là server action kết nối scheduling engine với cơ sở dữ liệu. Luồng xử lý:

1. Xác thực người dùng qua `requireAuth()`
2. Lấy hồ sơ người dùng (`chronotype`, `wakeTime`, `sleepTime`, `timezone`)
3. Lấy danh sách sự kiện hiện có từ cơ sở dữ liệu
4. Tải trọng số thích ứng từ bảng `weightAdjustments` và tính trọng số cuối cùng qua `computeFinalWeights()`
5. Gọi `suggestBestSlot()` với trọng số đã tính
6. Nếu có xung đột và chưa được xác nhận (`forceCreate = false`): trả về thông tin xung đột cho giao diện hiển thị
7. Nếu thành công: tạo bản ghi trong bảng `tasks`, tạo sự kiện trong bảng `events`, liên kết qua `scheduledEventId`
8. Đẩy sự kiện lên Google Calendar ở chế độ nền (`background`)

Đoạn mã 4.16: Xử lý xung đột trong `scheduleTask`

```
1 // Neu co xung dot va chua xac nhan
2 if (suggestion.hasConflict && !input.forceCreate) {
3   return {
4     success: false,
5     hasConflict: true,
6     conflictsWith: {
7       title: conflict.title,
8       start: conflict.start.toISOString(),
9       end: conflict.end.toISOString(),
10    },
11    message: 'This time overlaps with "'
12      + conflict.title
13      + '". Create anyway?',
14  };
15 }
```

4.4 Triển khai hệ thống trọng số thích ứng

4.4.1 Hồ sơ trọng số mặc định (Preset Profiles)

Có 5 bộ trọng số mặc định, mỗi bộ phản ánh một triết lý lập lịch khác nhau. Tổng trọng số mỗi bộ = 1.0:

Đoạn mã 4.17: 5 bộ trọng số mặc định

```

1  const PRESET_PROFILES = {
2    hustle: { chronotype: 0.15, deadline: 0.35,
3      contextSwitch: 0.10, fragmentation: 0.10,
4      priority: 0.25, buffer: 0.05 },
5    balanced: { chronotype: 0.30, deadline: 0.20,
6      contextSwitch: 0.10, fragmentation: 0.10,
7      priority: 0.15, buffer: 0.15 },
8    wellness: { chronotype: 0.35, deadline: 0.10,
9      contextSwitch: 0.10, fragmentation: 0.10,
10     priority: 0.10, buffer: 0.25 },
11    parent: { chronotype: 0.25, deadline: 0.20,
12      contextSwitch: 0.05, fragmentation: 0.05,
13      priority: 0.20, buffer: 0.25 },
14    creative: { chronotype: 0.30, deadline: 0.15,
15      contextSwitch: 0.25, fragmentation: 0.20,
16      priority: 0.05, buffer: 0.05 },
17  };

```

Ví dụ: hồ sơ hustle (tập trung hiệu suất) ưu tiên deadline (0.35) và priority (0.25), trong khi wellness (ưu tiên sức khỏe) ưu tiên năng lượng sinh học (0.35) và khoảng nghỉ (0.25).

4.4.2 Tính trọng số cuối cùng

Hàm computeFinalWeights() kết hợp 3 lớp thành bộ trọng số cuối cùng:

Đoạn mã 4.18: Kết hợp 3 lớp trọng số

```

1  function computeFinalWeights(adj) {
2    if (!adj) return DEFAULT_WEIGHTS; // balanced
3
4    const preset = PRESET_PROFILES[adj.schedulingProfile];
5    const raw = {
6      chronotype: preset.chronotype
7        + adj.manualChrono // Lop 2: +-5%
8        + adj.chronotypeAdj, // Lop 3: +-15%
9      deadline: preset.deadline
10       + adj.manualDeadline
11       + adj.deadlineAdj,
12      contextSwitch: preset.contextSwitch
13       + adj.manualContext
14       + adj.contextSwitchAdj,
15      fragmentation: preset.fragmentation
16       + adj.manualFrag
17       + adj.fragmentationAdj,
18      priority: preset.priority

```

```

19     + adj.manualPriority
20     + adj.priorityAdj,
21     buffer: preset.buffer
22     + adj.manualBuffer
23     + adj.bufferAdj,
24   };
25   return normalizeWeights(raw);
26 }

```

Hàm `normalizeWeights()` đảm bảo tổng trọng số luôn = 1.0: đặt sào 0 cho mỗi thành phần (tránh trọng số âm), sau đó chia cho tổng. Nếu tất cả bằng 0 (trường hợp hiếm), trả về bộ mặc định `balanced`.

4.4.3 Bộ xử lý phản hồi (Feedback Processor)

Hàm `applySignals()` cập nhật giá trị hài lòng (satisfaction) theo phương pháp EMA (trung bình trượt hàm mũ). Đặc biệt, thao tác cập nhật sử dụng biểu thức SQL nguyên tử (atomic SQL expression) để tránh điều kiện tranh chấp đọc-ghi (read-write race condition):

Đoạn mã 4.19: Cập nhật EMA nguyên tử bằng SQL

```

1  async function applySignals(userId, signals) {
2    const setClauses = {
3      lastUpdatedAt: new Date(),
4      totalObservations:
5        sql`total_observations + 1`,
6    };
7
8    for (const key of Object.keys(signals)) {
9      const signal = signals[key];
10     const satRef = weightAdjustments[satCol];
11     const decay = 0.7; // 1 - alpha
12
13     // EMA: sat_new = sat * 0.7 + signal * 0.3
14     const newSat = sql`${satRef} * ${decay}::real
15       + ${signal}::real * 0.3::real`;
16
17     // Adj = clamp(sat - 0.5, -0.15, +0.15)
18     const newAdj = sql`GREATEST(-0.15::real,
19       LEAST(0.15::real, ${newSat} - 0.5::real))`;
20
21     setClauses[satCol] = newSat;
22     setClauses[adjCol] = newAdj;
23   }
24 }

```

```

25   await db.update(weightAdjustments)
26     .set(setClauses)
27     .where(eq(weightAdjustments.userId, userId));
28 }

```

Điểm quan trọng: toàn bộ phép tính EMA và clamp được thực hiện **trong một câu lệnh SQL UPDATE duy nhất**. Điều này đảm bảo nếu hai yêu cầu cập nhật cùng lúc (ví dụ: người dùng hoàn thành 2 công việc liên tiếp), dữ liệu không bị ghi đè sai.

4.4.4 Nguồn tín hiệu phản hồi

Hệ thống thu thập 5 loại tín hiệu ngầm định (xem Mục 2.5.3). Mỗi tín hiệu là một hàm thuần trả về giá trị trong khoảng $[0, 1]$:

- **Tín hiệu hoàn thành** (computeCompletionTimingSignal): So sánh thời điểm kết thúc dự kiến với thời điểm hoàn thành thực tế. Trả về null nếu trễ > 4 giờ (cần modal hỏi thêm).
- **Tín hiệu chọn phương án** (computeOptionSelectionSignals): Khi người dùng chọn phương án không phải tốt nhất, tính mức chênh lệch cho từng thành phần chi phí, điều chỉnh theo hệ số lan truyền điểm (spread factor).
- **Tín hiệu dời lịch** (computeRescheduleSignals): Phát hiện thay đổi giờ (chronotype), khoảng cách đến sự kiện kế tiếp (buffer), và việc vượt qua deadline.
- **Tín hiệu thay đổi thời lượng** (computeResizeSignals): Chỉ phát tín hiệu khi kéo dài công việc, và chỉ khi khoảng cách đến sự kiện kế tiếp trước khi kéo dài ≤ 30 phút.
- **Tín hiệu đánh giá** (computeRatingSignal): Ánh xạ 4 mức emoji sang giá trị: great = 1.0, good = 0.75, meh = 0.5, tired = 0.25.

Hàm completeTask() trong hành động phía máy chủ (server action) xử lý bước đầu tiên: tính tín hiệu thời gian (timing signal) dựa trên chênh lệch giữa giờ kết thúc dự kiến và giờ hoàn thành thực tế, rồi đẩy tín hiệu đó vào EMA. Bước thứ hai (tín hiệu đánh giá từ emoji) được xử lý riêng khi người dùng chọn emoji trên giao diện:

Đoạn mã 4.20: Kết hợp tín hiệu khi hoàn thành công việc

```

1  async function completeTask(taskId) {
2    // ... cập nhật trạng thái "completed"
3
4    if (updated.scheduledEventId) {
5      const event = await db.query.events.findFirst(
6        { where: eq(events.id, scheduledEventId) }
7      );
8      // Tính tín hiệu thời gian
9      const timing = computeCompletionTimingSignal(
10        event.endTime, now
11      );

```



```
12     if (timing === null) {
13         needsLateModal = true; // Tre > 4h
14     } else {
15         // Áp dụng tin hiệu cho chronotype,
16         // deadline, priority
17         await applySignals(userId, {
18             chronotype: timing,
19             deadline: timing,
20             priority: timing,
21         });
22     }
23 }
24 }
```

4.5 Triển khai giao diện AI Chat

Giao diện AI Chat là lớp ngôn ngữ tự nhiên (natural language interface) đặt trên scheduling engine. Người dùng nhập câu lệnh bằng tiếng Anh hoặc tiếng Việt, mô hình ngôn ngữ lớn (LLM) phân tích ý định và gọi đúng công cụ.

4.5.1 Kiến trúc đường ống xử lý (Pipeline)

Luồng xử lý một tin nhắn:

1. **Xác thực và giới hạn tần suất:** Kiểm tra phiên đăng nhập (session), áp dụng giới hạn 10 yêu cầu/phút/người dùng bằng bộ đếm trong bộ nhớ (in-memory rate limiter).
2. **Xây dựng ngữ cảnh:** Truy vấn hồ sơ người dùng (chronotype, giờ thức/ngủ) và danh sách sự kiện 7 ngày tới. Tạo câu lệnh hệ thống (system prompt) động chứa thông tin cá nhân hóa.
3. **Gọi LLM qua OpenRouter:** Gửi system prompt + lịch sử hội thoại + định nghĩa 7 công cụ (tools) đến OpenRouter API với chế độ phát trực tuyến (streaming).
4. **Phát trực tuyến phản hồi:** Đọc luồng SSE (Server-Sent Events) từ OpenRouter, chuyển tiếp từng đoạn văn bản đến giao diện qua ReadableStream.
5. **Phát hiện lệnh gọi công cụ:** Khi LLM quyết định gọi công cụ (ví dụ: `schedule_task`), thu thập tên và tham số, gửi về giao diện dưới dạng sự kiện `functionCall`.
6. **Thực thi phía giao diện:** Giao diện nhận `functionCall`, gọi server action tương ứng (ví dụ: `scheduleTask()`), hiển thị kết quả.

4.5.2 Câu lệnh hệ thống động (Dynamic System Prompt)

Hàm `generateSystemPrompt()` tạo câu lệnh hệ thống chứa thông tin cá nhân hóa theo thời gian thực:

Đoạn mã 4.21: Ngữ cảnh trong câu lệnh hệ thống

```
1 // Ho so nguoi dung
2 Chronotype: Lion
3 Peak energy: 6:00 - 10:00
4 Energy dip: 13:00 - 15:00
5 Wake time: 06:00, Sleep time: 22:00
6
7 // Ng u c a n h t h o i g i a n (c a p n h a t m o i l a n g o i)
8 Today: 2026-03-12 (Thursday)
9 Current time: 14:30
10
11 // S u k i e n s a p t o i (d e L L M b i e t t e n c h i n h x a c)
12 - "Team_standup" (Today at 3:00 PM)
13 - "Gym_workout" (Tomorrow at 8:00 AM)
```

Danh sách sự kiện sắp tới được nhúng trực tiếp vào system prompt để LLM có thể khớp tên chính xác khi người dùng nói “dời cuộc họp” hoặc “xóa buổi tập gym”.

4.5.3 Định nghĩa 7 công cụ (Tool Definitions)

Mỗi công cụ được định nghĩa theo chuẩn OpenAI Function Calling, gồm tên, mô tả (chỉ dẫn khi nào LLM nên gọi), và schema tham số:

Bảng 4.3: 7 công cụ AI Chat

Công cụ	Chức năng
<code>schedule_task</code>	Tạo công việc mới, gọi scheduling engine
<code>query_schedule</code>	Truy vấn lịch theo khoảng ngày
<code>get_scheduling_options</code>	Lấy 3 phương án lập lịch tốt nhất
<code>reschedule_event</code>	Dời sự kiện sang thời gian mới
<code>delete_event</code>	Xóa sự kiện khỏi lịch và Google Calendar
<code>resize_task</code>	Thay đổi thời lượng sự kiện
<code>complete_task</code>	Đánh dấu công việc hoàn thành

Mỗi mô tả công cụ chứa hướng dẫn chi tiết cho LLM. Ví dụ, `schedule_task` phân biệt rõ giữa `preferred_date` (ngày muốn thực hiện) và `deadline` (hạn chót phải hoàn thành):

Đoạn mã 4.22: Ví dụ hướng dẫn LLM phân biệt ngày và hạn chót

```
1 // D i n h n g h i a t h a m s o t r o n g t o o l s c h e m a :
```

```
2 preferred_date: "The DATE when user wants
3 to do the task. This is NOT a deadline."
4 deadline: "Deadline date. The date the task
5 must be COMPLETED BY."
6
7 // Vi du 1: "study for Chemistry exam on Feb 10"
8 // -> preferred_date: "2026-02-08" (hom nay)
9 // -> deadline: "2026-02-10" (ngay thi)
10 // Nguoi dung muon ON hom nay, XONG truoc 10/2
11
12 // Vi du 2: "meeting tomorrow at 3pm"
13 // -> preferred_date: "2026-02-09" (ngay mai)
14 // -> deadline: null (khong co han chot)
```

4.5.4 Giới hạn tần suất (Rate Limiting)

Hệ thống giới hạn tần suất sử dụng bộ đếm trong bộ nhớ (in-memory counter) với cơ chế cửa sổ cố định (fixed window):

Đoạn mã 4.23: Bộ giới hạn tần suất

```
1 interface RateLimitEntry {
2     count: number;
3     resetTime: number;
4 }
5
6 const RATE_LIMITS = {
7     chat: {
8         maxRequests: 10,    // 10 yeu cau
9         windowMs: 60000,    // moi phut
10    },
11    execute: {
12        maxRequests: 20,
13        windowMs: 60000,
14    },
15 };
```

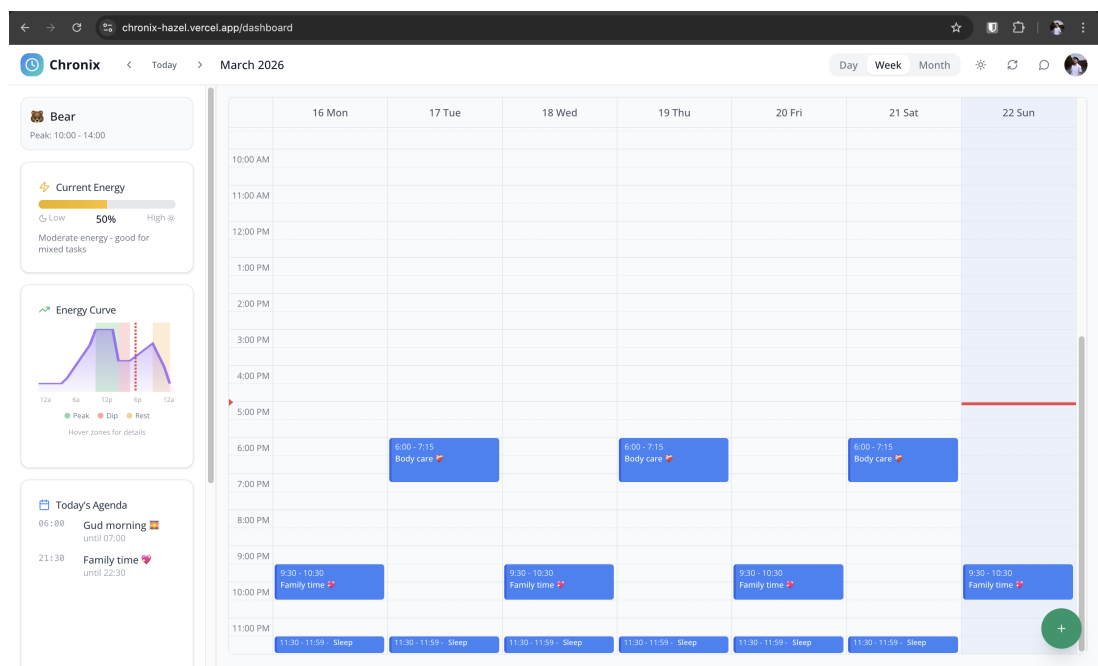
Bộ nhớ được dọn dẹp mỗi 60 giây bằng `setInterval()` để xóa các mục hết hạn. Giải pháp này phù hợp cho triển khai đơn máy chủ (single-server). Với nhiều máy chủ, cần thay bằng Redis.

4.6 Triển khai giao diện người dùng

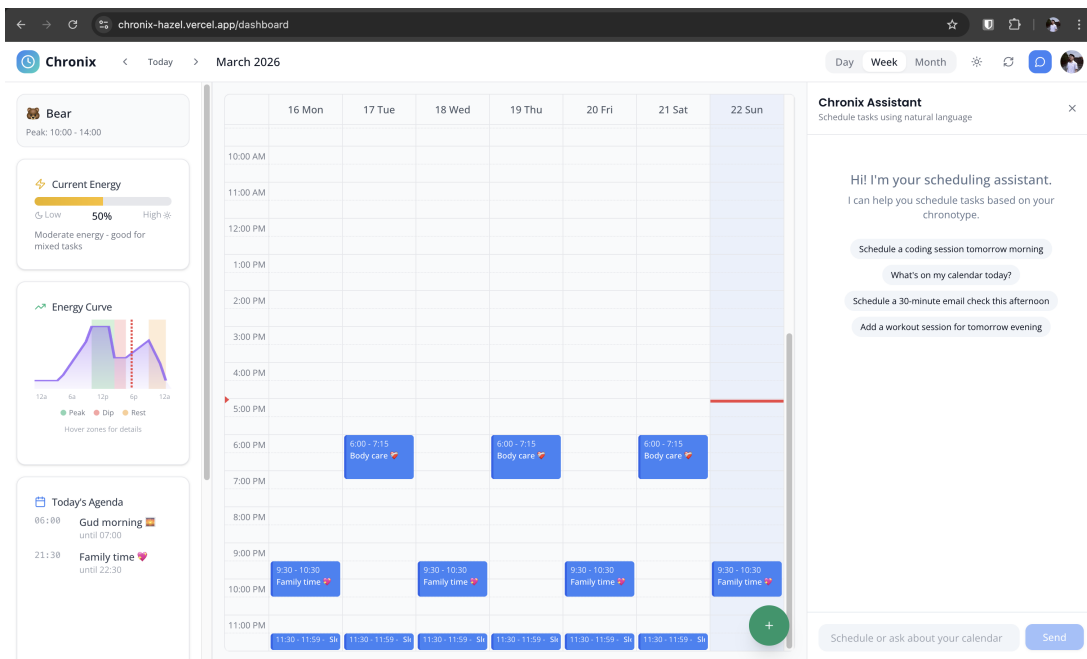
4.6.1 Bảng điều khiển (Dashboard)

Bảng điều khiển chính gồm hai phần chính đặt cạnh nhau:

- **Khung lịch (Calendar View):** Sử dụng FullCalendar.js hiển thị lịch ngày, tuần, hoặc tháng. Sự kiện từ Google Calendar và từ CHRONIX hiển thị cùng nhau.
- **Khung chat (Chat Panel):** Giao diện hội thoại AI ở bên phải, hỗ trợ phát trực tuyến văn bản (text streaming). Khi LLM trả về lệnh gọi công cụ, giao diện tự động thực thi và cập nhật lịch theo thời gian thực.



Hình 4.1: Giao diện bảng điều khiển chính với lịch tuần.

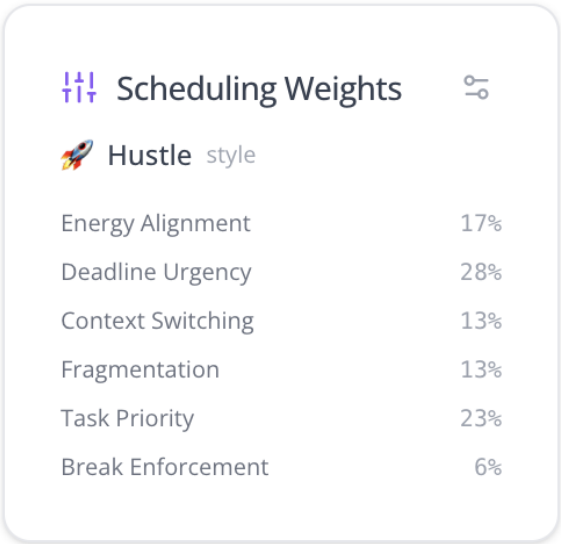


Hình 4.2: Khung chat AI với tính năng lịch sử hội thoại và phát trực tuyến phản hồi.

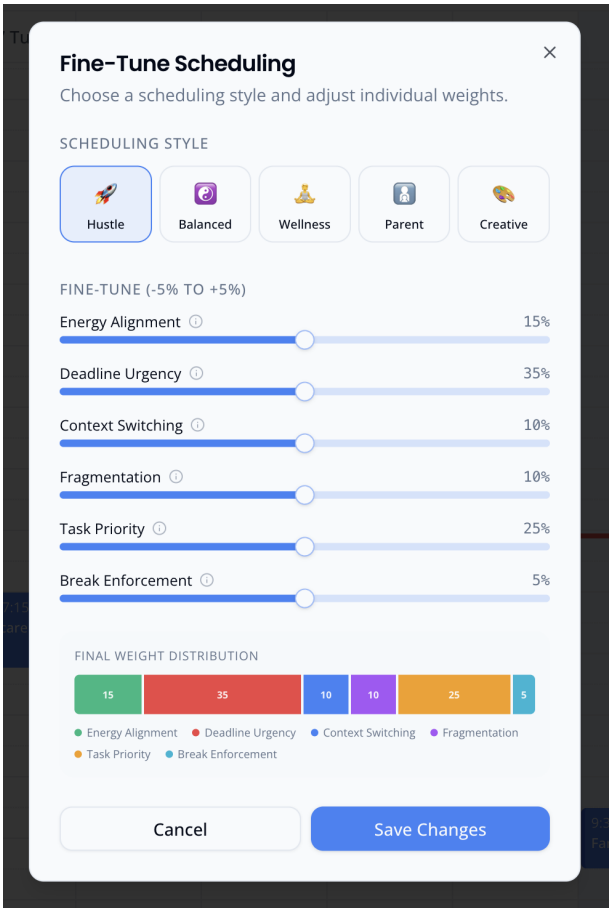
Các thành phần phụ:

- **Hộp thoại tạo nhanh (Quick Add Modal):** Form tạo công việc với các trường: tiêu đề, thời lượng (15–720 phút), deadline, độ ưu tiên (1–5), loại công việc (5 loại), thời gian ưa thích.
- **Bảng trọng số (Weight Stats Panel):** Hiển thị 6 trọng số hiện tại trong thanh bên (sidebar), tải qua useEffect gọi server action.
- **Hộp thoại tinh chỉnh (Fine-Tune Modal):** 6 thanh trượt (slider) cho phép điều chỉnh thủ công trọng số từng thành phần trong phạm vi $\pm 5\%$.

Hình 4.3: Hộp thoại tạo nhanh công việc.



Hình 4.4: Bảng trọng số trên thanh bên.



Hình 4.5: Hộp thoại tinh chỉnh trọng số thủ công.

4.6.2 Quy trình giới thiệu (Onboarding Flow)

Quy trình onboarding gồm 7 bước, triển khai bằng component Wizard điều khiển trạng thái bước hiện tại:

- 1. **Chào mừng** (step-welcome): Giới thiệu CHRONIX và mục đích khảo sát.

2. **Múi giờ** (step-timezone): Tự động phát hiện múi giờ từ trình duyệt, cho phép chỉnh sửa.
3. **Giờ thức/ngủ** (step-time): Nhập giờ thức dậy và giờ đi ngủ tự nhiên.
4. **Khảo sát** (step-quiz, 3 bước): 5 câu hỏi dựa trên bảng AutoMEQ rút gọn, mỗi câu có 4–5 phương án với điểm số. Hiển thị lần lượt để tránh quá tải.
5. **Hồ sơ lập lịch** (step-profile): Chọn 1 trong 5 hồ sơ: hustle, balanced, wellness, parent, creative.
6. **Hoàn thành** (step-complete): Hiển thị kết quả chronotype và chuyển hướng đến dashboard.
Thuật toán xác định chronotype từ câu trả lời:

Đoạn mã 4.24: Xác định chronotype từ điểm số

```

1 function calculateFromScores(scores) {
2   const total = scores.reduce((a, b) => a + b, 0);
3   // Điểm tối đa: 23, Tối thiểu: 5
4
5   // Kiểm tra giấc ngủ bất thường trước
6   const sleepQuality = scores[4];
7   if (sleepQuality === 1) return "dolphin";
8
9   // Phân loại theo tổng điểm
10  if (total >= 18) return "lion";    // Sang sớm
11  if (total >= 13) return "bear";   // Trung bình
12  return "wolf";                    // Tối muộn
13 }
```

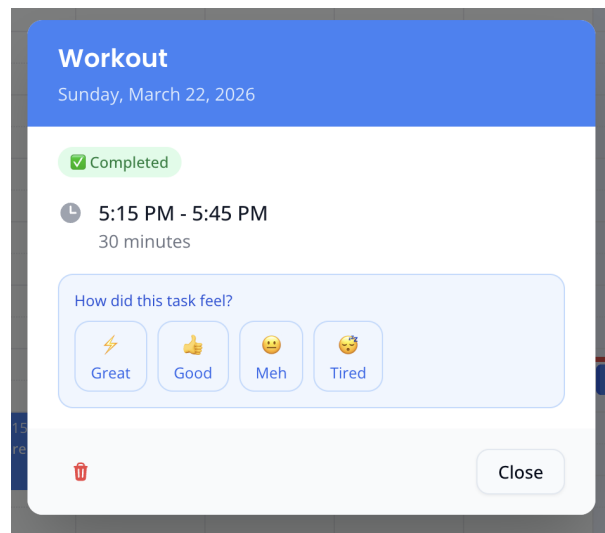
Sau khi hoàn thành, server action `saveOnboardingProfile()` thực hiện:

- Tính chronotype từ câu trả lời
- Cập nhật bảng users: chronotype, wakeTime, sleepTime, timezone, onboardingCompleted = true
- Tạo bản ghi weightAdjustments với hồ sơ lập lịch đã chọn (sử dụng `onConflictDoUpdate` để xử lý trường hợp chạy lại)

4.6.3 Đánh giá sau hoàn thành (Post-Task Rating)

Khi đánh dấu công việc hoàn thành, giao diện hiển thị:

- 4 nút biểu tượng cảm xúc: great (tuyệt vời), good (tốt), meh (bình thường), tired (mệt mỏi)
- Kết hợp tín hiệu thời gian (timing signal, khách quan) với đánh giá cảm xúc (rating, chủ quan) theo công thức: $S = 0.4 \times S_{\text{timing}} + 0.6 \times S_{\text{rating}}$
- Nếu hoàn thành trễ > 4 giờ: hệ thống không thể suy luận tín hiệu thời gian (có thể do quên đánh dấu, hoặc công việc thực sự kéo dài), hiển thị hộp thoại trễ (Late Completion Modal) với 3 lựa chọn



Hình 4.6: Giao diện đánh giá sau khi hoàn thành công việc với 4 mức cảm xúc.

4.7 Triển khai đồng bộ Google Calendar

Hệ thống đồng bộ hai chiều (bidirectional sync) giữa cơ sở dữ liệu nội bộ và Google Calendar, sử dụng 3 chế độ.

4.7.1 Tạo máy khách Google Calendar

Hàm `getGoogleCalendarClient()` tạo phiên xác thực OAuth 2.0 cho mỗi người dùng. Token truy cập (access token) và token làm mới (refresh token) được lưu trong bảng `accounts` (quản lý bởi Better Auth). Khi token hết hạn, sự kiện `tokens` tự động làm mới và cập nhật vào cơ sở dữ liệu:

Đoạn mã 4.25: Tự động làm mới token

```
1  oauth2Client.on("tokens", async (newTokens) => {
2    if (newTokens.access_token) {
3      await db.update(accounts).set({
4        accessToken: newTokens.access_token,
5        refreshToken: newTokens.refresh_token
6        ?? account.refreshToken,
7        accessTokenExpiresAt: newTokens.expiry_date
8        ? new Date(newTokens.expiry_date)
9        : undefined,
10     }).where(eq(accounts.userId, userId));
11   }
12 });
```


4.7.2 Đồng bộ toàn phần (Initial Sync)

Khi người dùng kết nối Google Calendar lần đầu hoặc bấm làm mới thủ công:

1. Gọi Google Calendar API lấy sự kiện trong cửa sổ 28 ngày (14 ngày trước + 14 ngày tới), giới hạn 250 sự kiện.
2. Lấy mã đồng bộ (sync token) để dùng cho lần đồng bộ gia tăng tiếp theo. Google Calendar API chỉ trả sync token khi gọi không có bộ lọc thời gian, nhưng bước 1 đã dùng bộ lọc timeMin/timeMax. Vì vậy, hệ thống phải gọi thêm một yêu cầu riêng không có bộ lọc, rồi lặp qua từng trang (paginate) đến trang cuối để lấy token.
3. Chiến lược xóa-rồi-chèn (delete-then-reinsert):
 - Lưu liên kết task–event hiện có (để khôi phục sau)
 - Xóa tham chiếu khóa ngoại (foreign key) trong bảng tasks và energyLogs
 - Xóa tất cả sự kiện Google trong cửa sổ thời gian
 - Chèn hàng loạt (batch insert) toàn bộ sự kiện từ Google, mỗi lô 50 sự kiện
 - Khôi phục liên kết task–event dựa trên googleEventId
4. Lưu sync token và thời gian đồng bộ tiếp theo (15 phút sau) vào bảng syncState.

Chiến lược xóa-rồi-chèn nhanh hơn upsert từng sự kiện (1 DELETE + vài INSERT thay vì hàng trăm upsert riêng lẻ), đặc biệt quan trọng khi lịch có nhiều sự kiện.

4.7.3 Đồng bộ gia tăng (Incremental Sync)

Tác vụ định kỳ (cron job) chạy mỗi 15 phút qua endpoint /api/cron/calendar-sync, được xác thực bằng CRON_SECRET:

1. Truy vấn bảng syncState lấy danh sách người dùng đến hạn đồng bộ (nextSyncAt < now), giới hạn 50 người mỗi lần.
2. Với mỗi người dùng, gọi Google Calendar API với syncToken để lấy **chỉ các thay đổi** kể từ lần đồng bộ trước.
3. Phân loại thay đổi thành 2 loại:
 - Sự kiện bị xóa (status = "cancelled"): xóa khỏi cơ sở dữ liệu nội bộ, đồng thời hủy liên kết khóa ngoại trước.
 - Sự kiện mới hoặc cập nhật: upsert vào bảng events dựa trên (userId, googleEventId).
4. Cập nhật syncToken mới và đặt nextSyncAt thêm 15 phút.
5. Điều tiết tần suất (throttling): nếu lần đồng bộ gần nhất cách đây < 60 giây, bỏ qua.
6. Xử lý lỗi 410 Gone (sync token hết hạn): tự động chuyển sang đồng bộ toàn phần.

Đoạn mã 4.26: Xử lý lỗi sync token hết hạn

```
1 catch (error) {  
2   if (error.code === 410) {  
3     // Sync token hết hạn, đồng bộ toàn phần
```

```
4     return performInitialSync(userId);
5   }
6   // Ghi log và thu lại sau 15 phút
7   await db.update(syncState).set({
8     syncError: error.message,
9     nextSyncAt: addMinutes(new Date(), 15),
10  });
11 }
```

4.7.4 Đẩy sự kiện lên Google (Push Sync)

Khi người dùng tạo hoặc cập nhật sự kiện qua CHRONIX (chat AI hoặc form):

1. Sự kiện được lưu vào cơ sở dữ liệu nội bộ trước (**local-first**) với `syncStatus = "pending_push"`.
2. Gọi `pushEventToGoogle()` ở chế độ nền (không chờ kết quả): nếu sự kiện đã có `googleEventId` thì cập nhật, nếu chưa thì tạo mới.
3. Sau khi đẩy thành công, cập nhật `googleEventId`, `etag` (thẻ phiên bản), và `syncStatus = "synced"`.

Phương pháp `local-first` đảm bảo giao diện phản hồi ngay lập tức mà không phải chờ Google API (có thể mất 200–500ms).

4.8 Bảo mật

- **Xác thực (Authentication):** OAuth 2.0 qua Better Auth. Mọi server action đều gọi `requireAuth()` để kiểm tra phiên đăng nhập trước khi truy cập dữ liệu.
- **Phiên đăng nhập (Session):** Cookie `httpOnly` (không truy cập được từ JavaScript phía giao diện), cờ `Secure` (chỉ gửi qua HTTPS), `SameSite=Strict` (chống tấn công CSRF).
- **Bảo vệ tuyến đường (Route Protection):** Middleware của Next.js kiểm tra cookie phiên đăng nhập. Trang `/dashboard` và `/api/*` yêu cầu đăng nhập; trang `/auth` và `/onboarding` cho phép truy cập ẩn danh.
- **Bảo mật hàng dữ liệu (Row Level Security):** Tất cả truy vấn cơ sở dữ liệu đều lọc theo `userId` của phiên hiện tại, đảm bảo người dùng chỉ truy cập dữ liệu của chính mình.
- **Giới hạn tần suất (Rate Limiting):** 10 yêu cầu chat/phút và 20 yêu cầu thực thi/phút, ngăn chặn lạm dụng API.
- **Xác thực cron job:** Endpoint `/api/cron/calendar-sync` yêu cầu header `Authorization: Bearer {CRON_SECRET}`, ngăn chặn gọi trái phép.

4.9 Triển khai lên môi trường production

- **Ứng dụng:** Triển khai trên Vercel, tự động build và deploy khi đẩy mã lên nhánh main trên GitHub.
- **Cơ sở dữ liệu:** Supabase PostgreSQL, kết nối qua connection string được mã hóa trong biến môi trường.
- **Biến môi trường:** Quản lý qua Vercel Dashboard.
- **Tác vụ định kỳ:** Vercel Cron gọi `/api/cron/calendar-sync` mỗi 15 phút.
- **Giám sát:** Vercel Analytics theo dõi các chỉ số hiệu suất (Core Web Vitals).

Chương 5

Đánh giá

Chương này trình bày kết quả đánh giá hệ thống CHRONIX thông qua hai bộ kiểm chuẩn (benchmark): kiểm chuẩn chất lượng thuật toán và mô phỏng hội tụ EMA. Toàn bộ thí nghiệm sử dụng dữ liệu tổng hợp (synthetic data) với bộ sinh số ngẫu nhiên có hạt giống (seeded PRNG) để đảm bảo **tính tái tạo hoàn toàn** - chạy lại cho kết quả giống hệt.

5.1 Phương pháp đánh giá

Việc đánh giá được thực hiện thông qua hai bộ kiểm chuẩn:

- Kiểm chuẩn chất lượng thuật toán (Algorithm Quality Benchmark):** So sánh chất lượng lập lịch của CHRONIX với 4 phương pháp cơ bản (baseline) trên 6.000 lượt chạy, sử dụng 8 chỉ tiêu đánh giá.
- Mô phỏng hội tụ EMA (EMA Convergence Simulation):** Kiểm chứng tính hội tụ, phản hồi đúng hướng, và ổn định của hệ thống trọng số thích ứng trên 250 lượt quan sát (observation).

5.1.1 Bộ sinh số ngẫu nhiên tắt định

Để đảm bảo tính tái tạo, benchmark sử dụng bộ sinh số đồng dư tuyến tính (Linear Congruential Generator - LCG) với các tham số từ Numerical Recipes (Press et al., 2007):

Đoạn mã 5.1: Bộ sinh số ngẫu nhiên có hạt giống

```
1 class SeededRandom {
2     private state: number;
3
4     constructor(seed: number) {
5         this.state = seed >>> 0;
6     }
7
8     next(): number { // Tra ve [0, 1)
9         this.state = (this.state * 1664525
```

```
10     + 1013904223) >>> 0;
11     return this.state / 0x100000000;
12 }
13
14 nextInt(min, max) { // So nguyen [min, max]
15     return min + Math.floor(
16         this.next() * (max - min + 1)
17     );
18 }
19
20 pick(array) { // Chọn ngẫu nhiên tu mang
21     return array[Math.floor(
22         this.next() * array.length
23     )];
24 }
25 }
```

Mọi thao tác ngẫu nhiên (tạo kịch bản, chọn slot, sinh tín hiệu EMA) đều sử dụng SeededRandom thay vì Math.random(). Nhờ đó, cùng một hạt giống luôn cho cùng kết quả trên mọi máy.

5.2 Kiểm chuẩn chất lượng thuật toán

5.2.1 Thiết lập thí nghiệm

- **Kịch bản (scenario):** 3 kích thước $\times$ 4 chronotype = 12 cấu hình:
 - Nhỏ: 5 công việc + 3 sự kiện sẵn có, 1 ngày
 - Trung bình: 15 công việc + 8 sự kiện, 3 ngày
 - Lớn: 30 công việc + 20 sự kiện, 7 ngày
- **Phương pháp:** 5 chiến lược lập lịch so sánh trực tiếp
- **Số lần chạy:** 100 hạt giống (seed) mỗi cấu hình
- **Tổng cộng:** $3 \times 4 \times 5 \times 100 = 6.000$ lượt lập lịch
- **Ngày gốc cố định:** 02/03/2026 (Thứ Hai), múi giờ Asia/Bangkok (GMT+7)

Tạo kịch bản tổng hợp

Mỗi kịch bản được tạo tự động từ hạt giống. Công việc được sinh với phân bố có trọng số (weighted distribution) mô phỏng khối lượng công việc thực tế:

Bảng 5.1: Phân bố loại công việc trong kịch bản tổng hợp

Loại công việc	Tỷ trọng	Thời lượng
analytical (phân tích)	30%	15–120 phút (bước 15 phút)
administrative (hành chính)	25%	
creative (sáng tạo)	20%	
learning (học tập)	15%	
physical (vận động)	10%	

Các tham số khác: độ ưu tiên ngẫu nhiên 1–5, 50% công việc có deadline (trong phạm vi kịch bản). Sự kiện sẵn có (existing events) được tạo rải đều trong khoảng thời gian, tránh trùng lặp.

Mỗi chronotype sử dụng giờ thức/ngủ riêng: Lion (06:00–22:00), Bear (07:00–23:00), Wolf (09:00–01:00), Dolphin (07:00–23:00).

5.2.2 5 phương pháp so sánh

Bảng 5.2: 5 phương pháp lập lịch được so sánh

#	Phương pháp	Chiến lược
1	Random (ngẫu nhiên)	Chọn slot ngẫu nhiên cho mỗi công việc
2	First-Fit (đến trước phục vụ trước)	Chọn slot đầu tiên theo thứ tự thời gian
3	Priority-Only (chỉ ưu tiên)	Sắp xếp theo độ ưu tiên giảm + deadline tăng, rồi chọn slot đầu tiên
4	Energy-Only (chỉ năng lượng)	Chấm điểm slot chỉ theo năng lượng chronotype, chọn slot có năng lượng cao nhất
5	Chronix	Hàm chi phí đầy đủ 6 yếu tố với trọng số cân bằng (balanced)

Điểm quan trọng: tất cả 5 phương pháp sử dụng cùng hàm findAvailableSlots() để tạo danh sách slot, chỉ khác nhau ở **chiến lược lựa chọn** slot. Sau khi gán công việc vào slot, slot đó được đánh dấu đã dùng (cơ chế tham lam - greedy). Điều này đảm bảo so sánh công bằng: mọi phương pháp đều có cùng tập slot ứng viên.

Đoạn mã 5.2: Phương pháp Chronix trong benchmark

```
1 function scheduleChronix(scenario, weights) {
2   const scheduled = [];
3   const w = weights || DEFAULT_WEIGHTS;
4
5   // Sắp xếp: ưu tiên giảm, deadline tăng
6   const sorted = [...scenario.tasks].sort(
```

```

7      (a, b) => {
8          if (b.priority !== a.priority)
9              return b.priority - a.priority;
10         if (a.deadline && b.deadline)
11             return a.deadline - b.deadline;
12         return a.deadline ? -1 : 1;
13     }
14 );
15
16 for (const task of sorted) {
17     const slots = getSlots(scenario, task,
18         scheduled);
19     if (slots.length === 0) continue;
20
21     // Tính chi phí dựa vào 6 yếu tố
22     let bestSlot = slots[0];
23     let bestCost = Infinity;
24     for (const slot of slots) {
25         const score = calculateSlotCost(
26             slot, task, context, w
27         );
28         if (score.totalCost < bestCost) {
29             bestCost = score.totalCost;
30             bestSlot = slot;
31         }
32     }
33     scheduled.push({ task, slot: bestSlot });
34 }
35 return scheduled;
36 }

```

5.2.3 8 chỉ tiêu đánh giá (Evaluation Metrics)

5 chỉ tiêu chính (nằm trong công thức tổng hợp)

1. **ETA – Energy-Task Alignment** (Mức khớp năng lượng, 30%): Trung bình $\min(1, E_{avail}/E_{req})$ trên tất cả công việc, trong đó E_{avail} là năng lượng có sẵn tại slot và E_{req} là năng lượng cần thiết cho loại công việc.
2. **DSR – Deadline Satisfaction Rate** (Tỷ lệ đáp ứng deadline, 25%): Số công việc hoàn thành trước deadline / tổng số công việc có deadline. Nếu không có công việc nào có deadline, $DSR = 1.0$.
3. **TCS – Total Cost Score** (Điểm chi phí tổng, 15%): Trung bình $totalCost$ từ hàm chi

phí 6 yếu tố. Đối với chỉ tiêu này, **thấp hơn là tốt hơn**, do đó trong công thức điểm tổng hợp, chỉ tiêu này được đảo thành $(1 - TCS)$.

4. **PWU – Peak Window Utilization** (Tận dụng giờ đỉnh, 20%): Số công việc yêu cầu năng lượng cao (≥ 70 : analytical, learning, creative) được xếp vào giờ đỉnh / tổng số công việc yêu cầu cao.
5. **BCR – Break Compliance Rate** (Tỷ lệ tuân thủ khoảng nghỉ, 10%): Số cặp công việc liên tiếp có khoảng cách ≥ 15 phút / tổng số công việc. Công việc cuối cùng được tính là tuân thủ.

2 chỉ tiêu phụ (chỉ mang tính tham khảo)

- **SFI – Schedule Fragmentation Index** (Chỉ số phân mảnh): Số khoảng trống nhỏ (< 30 phút) / tổng số khoảng trống. Thấp hơn là tốt hơn.
- **CSC – Context Switch Count** (Số lần chuyển đổi ngữ cảnh): Số cặp công việc liên tiếp có loại khác nhau.

Công thức điểm tổng hợp (Composite Score):

$$C = 0.3 \cdot ETA + 0.25 \cdot DSR + 0.15 \cdot (1 - TCS) + 0.2 \cdot PWU + 0.1 \cdot BCR$$

5.2.4 Kết quả

Bảng 5.3: Kết quả kiểm chuẩn chất lượng thuật toán (trung bình trên 6.000 lượt)

Phương pháp	ETA	DSR	TCS↓	PWU	BCR	SFI↓	CSC↓	Composite
Random	0.8248	0.2470	0.3218	0.2181	0.8483	0.0759	11.98	0.5618
First-Fit	0.8243	0.5688	0.3442	0.1855	0.2784	0.0000	11.96	0.5613
Priority-Only	0.8251	0.5875	0.3389	0.1930	0.2837	0.0000	12.09	0.5674
Energy-Only	0.9770	0.2641	0.2395	0.6397	0.8194	0.1177	12.12	0.6408
Chronix	0.9820	0.4834	0.2154	0.5035	0.8538	0.0265	10.26	0.7312

Ghi chú: ↓ = thấp hơn là tốt hơn. Giá trị in đậm = tốt nhất trong cột.

Kết quả chính: CHRONIX đạt điểm composite cao nhất (0.7312), vượt trội so với tất cả phương pháp cơ bản:

- Random: **+30.1%** (0.5618 → 0.7312)
- First-Fit: **+30.2%** (0.5613 → 0.7312)
- Priority-Only: **+28.9%** (0.5674 → 0.7312)
- Energy-Only: **+14.1%** (0.6408 → 0.7312)

5.2.5 Phân tích kết quả

CHRONIX là phương pháp duy nhất **cân bằng tất cả yếu tố đồng thời**. Mỗi phương pháp cơ bản tối ưu một khía cạnh nhưng hy sinh các khía cạnh khác:

- **So với Energy-Only:** Energy-Only đạt ETA gần tương đương (0.977 vs 0.982) nhưng hy sinh DSR nghiêm trọng (0.264 vs 0.483). Nguyên nhân: khi chỉ tối ưu năng lượng, thuật toán xếp mọi công việc vào giờ đỉnh mà bỏ qua deadline, dẫn đến nhiều công việc hoàn thành trễ hạn. CHRONIX cân bằng cả hai nhờ trọng số deadline (20% trong bộ balanced).
- **So với Priority-Only và First-Fit:** Hai phương pháp này đạt DSR khá (0.57–0.59) nhờ xếp theo thứ tự thời gian, nhưng PWU rất thấp (0.19–0.21). Nguyên nhân: chọn slot đầu tiên khả dụng thường là buổi sáng sớm, phù hợp với Lion nhưng không phù hợp với Wolf (đỉnh năng lượng lúc 17–21h). BCR cũng thấp (0.28) vì first-fit xếp công việc liên tiếp không có khoảng nghỉ.
- **So với Random:** Random đạt BCR cao bất ngờ (0.848). Nguyên nhân: khi chọn slot ngẫu nhiên, xác suất hai công việc liên tiếp xếp sát nhau rất thấp, tạo khoảng nghỉ tự nhiên. Tuy nhiên, Random thất bại ở các chỉ tiêu khác (DSR: 0.247, PWU: 0.218).

Về 2 chỉ tiêu phụ:

- **SFI (phân mảnh):** First-Fit và Priority-Only đạt $SFI = 0$ (tốt nhất) vì chúng xếp công việc liên tiếp theo thứ tự thời gian, không tạo khoảng trống nào giữa các công việc. Tuy nhiên, đây chính là lý do BCR của chúng rất thấp (0.28), không có khoảng trống cũng có nghĩa là không có khoảng nghỉ. CHRONIX đạt $SFI = 0.027$ (rất thấp), cho thấy thuật toán tạo rất ít khoảng trống nhỏ vô dụng.
- **CSC (chuyển đổi ngữ cảnh):** CHRONIX có số lần chuyển đổi thấp nhất (10.26) nhờ hình phạt chuyển đổi ngữ cảnh (context switch penalty) trong hàm chi phí, khuyến khích xếp các công việc cùng loại gần nhau. Các phương pháp khác dao động quanh 12 lần vì chúng không xét loại công việc khi chọn slot.

5.2.6 Kết quả theo chronotype

Bảng 5.4: Điểm composite theo chronotype

Phương pháp	Lion	Bear	Wolf	Dolphin
Random	0.5606	0.5527	0.5791	0.5548
First-Fit	0.5700	0.5493	0.5784	0.5476
Priority-Only	0.5719	0.5571	0.5880	0.5527
Energy-Only	0.6378	0.6466	0.6338	0.6449
Chronix	0.7348	0.7224	0.7408	0.7268

CHRONIX **thắng ở tất cả 4 chronotype** với hiệu suất đồng đều (khoảng 0.722–0.741). Điểm đáng chú ý:

- **Wolf đạt cao nhất (0.7408):** Chronotype Wolf có đỉnh năng lượng muộn (17–21h), khác biệt lớn nhất so với lập lịch mặc định buổi sáng. Các phương pháp không xét năng lượng (Random, First-Fit, Priority-Only) xếp công việc khó vào buổi sáng, đúng giờ suy giảm của Wolf. CHRONIX tránh điều này nhờ đường cong năng lượng.
- **Các phương pháp không xét năng lượng gần như không phân biệt chronotype:** Random, First-Fit, Priority-Only cho điểm composite gần bằng nhau ở mọi chronotype (khoảng 0.55–0.59), vì chúng không sử dụng thông tin chronotype.

5.3 Mô phỏng hội tụ EMA

5.3.1 Mục đích

Kiểm chuẩn thứ hai không so sánh CHRONIX với phương pháp khác, mà kiểm tra **hệ thống trọng số thích ứng có hoạt động đúng không**. Cụ thể, cần chứng minh 3 tính chất:

1. **Hội tụ (Convergence):** Satisfaction score (điểm hài lòng) và trọng số ổn định sau một số quan sát hữu hạn, không dao động mãi.
2. **Phản hồi đúng hướng (Responsiveness):** Tín hiệu thấp cho thành phần X dẫn đến giảm trọng số X , tín hiệu cao dẫn đến tăng trọng số X .
3. **Ổn định (Stability):** Tín hiệu nhiễu (ngẫu nhiên) không làm trọng số phân kỳ hay sụp đổ.

5.3.2 Thiết lập thí nghiệm

- **Hồ sơ hành vi (behavior profile):** 5 mẫu người dùng giả lập, mỗi mẫu phát tín hiệu theo quy luật riêng
- **Số quan sát:** 50 mỗi hồ sơ
- **Tổng:** 250 lượt cập nhật EMA
- **Công thức EMA:** $sat_{new} = sat_{old} \times 0.7 + signal \times 0.3$ ($\alpha = 0.3$)
- **So sánh chất lượng:** Với mỗi hồ sơ, lập lịch cùng kích bản (medium, seed=42) bằng trọng số ban đầu (balanced) và trọng số sau 50 quan sát, rồi so sánh điểm composite

5.3.3 5 hồ sơ hành vi

1. **Night Owl on Bear** (Cú đêm dùng hồ sơ Gấu): Tín hiệu chronotype thấp (0.25–0.4), còn lại trung tính. Kết quả mong đợi: giảm trọng số chronotype, chấp nhận slot không theo đỉnh năng lượng.
2. **Break Skipper** (Bỏ qua khoảng nghỉ): Tín hiệu buffer thấp (0.25–0.4), còn lại trung tính-tốt. Kết quả mong đợi: giảm trọng số buffer, ít ưu tiên khoảng nghỉ.
3. **Deadline Focused** (Tập trung deadline): Tín hiệu deadline cao (0.65–0.8), priority tốt. Kết quả mong đợi: tăng trọng số deadline, ưu tiên hoàn thành đúng hạn.

4. **Random/Inconsistent** (Ngẫu nhiên): Tất cả tín hiệu ngẫu nhiên (0.2–0.8). Kết quả mong đợi: dao động quanh giá trị gốc (baseline), không phân kỳ.
5. **Ideal User** (Người dùng lý tưởng): Tất cả tín hiệu cao đều (0.6–0.85). Kết quả mong đợi: ổn định gần cân bằng, không thay đổi nhiều.

Ở mỗi quan sát, hồ sơ phát tín hiệu cho cả 6 thành phần. Ví dụ, Night Owl phát tín hiệu chronotype thấp (0.25–0.4) vì không hài lòng khi bị xếp lịch theo đỉnh năng lượng Bear (10–14h), nhưng phát tín hiệu trung tính cho các thành phần khác.

5.3.4 Phương pháp mô phỏng

Với mỗi hồ sơ, mô phỏng chạy 50 vòng lặp:

1. Sinh tín hiệu cho 6 thành phần dựa trên quy luật hồ sơ (có nhiều ngẫu nhiên)
2. Cập nhật điểm hài lòng bằng EMA: $sat_i^{new} = sat_i^{old} \times 0.7 + signal_i \times 0.3$ (xem công thức 3.15 ở Mục 3.6)
3. Tính giá trị điều chỉnh tự động: $\Delta w_i^{auto} = \text{clamp}(sat_i - 0.5, -0.15, +0.15)$
4. Tính trọng số cuối cùng: w_i^{final} theo công thức 3.14 (trong mô phỏng, $\Delta w_i^{manual} = 0$)
5. Lưu ảnh chụp (snapshot) gồm: satisfaction, adjustment, trọng số cuối

Sau 50 quan sát, so sánh chất lượng bằng cách lập lịch cùng một kịch bản trung bình (15 công việc, 3 ngày) bằng trọng số ban đầu và trọng số đã thích ứng, rồi tính điểm composite.

5.3.5 Kết quả EMA

Bảng 5.5: Kết quả mô phỏng EMA - so sánh chất lượng trước và sau thích ứng

Hồ sơ	Trước	Sau	Delta	Giải thích
Night Owl	0.7023	0.6977	-0.0046	Giảm trọng số chronotype → bớt ưu tiên giờ đỉnh → điểm ETA giảm nhẹ, nhưng phù hợp sở thích
Break Skipper	0.7584	0.7202	-0.0382	Giảm trọng số buffer → bớt khoảng nghỉ → BCR giảm, composite giảm theo
Deadline Focused	0.7023	0.6987	-0.0036	Tăng trọng số deadline, giảm chronotype → hai yếu tố bù trừ, kết quả gần như không đổi
Random	0.5987	0.6549	+0.0562	Tín hiệu ngẫu nhiên triệt tiêu lẫn nhau qua EMA → trọng số hồi quy về mặc định
Ideal User	0.6624	0.6624	+0.0000	Mọi tín hiệu cao đều → mọi trọng số tăng đều → sau chuẩn hóa, tỷ lệ không đổi

5.3.6 Phân tích kết quả EMA

Tính hội tụ (Convergence)

Tất cả 5 hồ sơ có satisfaction score và trọng số ổn định trong 10–15 quan sát đầu tiên. Sau quan sát thứ 12, thay đổi trọng số mỗi bước < 0.005 .

Điều này phù hợp với phân tích lý thuyết: với hệ số suy giảm $1 - \alpha = 0.7$, sau n quan sát, ảnh hưởng của giá trị ban đầu là $(0.7)^n$. Sau 10 quan sát: $(0.7)^{10} = 0.028$ (2.8%), giá trị ban đầu gần như không còn ảnh hưởng. Sau 15 quan sát: $(0.7)^{15} = 0.005$ (0.5%), hệ thống đã hội tụ.

Tính phản hồi đúng hướng (Responsiveness)

Tín hiệu ảnh hưởng đúng hướng lên thành phần trọng số tương ứng:

- **Night Owl:** Gửi tín hiệu chronotype thấp (0.25–0.4) → chronotype satisfaction giảm xuống dưới 0.5 → adjustment âm → trọng số chronotype giảm. Hệ thống “hiểu” rằng người dùng không cần xếp lịch theo đỉnh năng lượng.
- **Break Skipper:** Gửi tín hiệu buffer thấp (0.25–0.4) → trọng số buffer giảm. Hệ thống giảm ưu tiên khoảng nghỉ, cho phép xếp công việc sát nhau hơn.
- **Deadline Focused:** Gửi tín hiệu deadline cao (0.65–0.8) → trọng số deadline tăng. Đồng

thời, tín hiệu chronotype hơi thấp (0.4–0.6) khiến trọng số chronotype giảm nhẹ, phù hợp vì người dùng này ưu tiên hoàn thành đúng hạn hơn xếp đúng giờ năng lượng.

Tính ổn định (Stability)

Hồ sơ Random/Inconsistent phát tín hiệu ngẫu nhiên trong khoảng rộng (0.2–0.8) cho tất cả thành phần. Kết quả:

- Satisfaction dao động quanh 0.5 (trung tính) mà **không phân kỳ** (diverge).
- Không trọng số nào sụp đổ về 0 hay chiếm ưu thế tuyệt đối.
- EMA tính trung bình có trọng số của nhiều tín hiệu nhiều, tạo dao động có giới hạn (bounded oscillation), hệ thống bền vững với đầu vào không nhất quán.

Hồ sơ Ideal User cho thấy tính chất đối xứng: khi tín hiệu đều cao cho mọi thành phần, các adjustment tăng đều, và sau khi chuẩn hóa (normalize), trọng số gần như không đổi so với bộ mặc định. $\Delta = 0.0000$ chứng minh hệ thống không gây nhiễu khi không cần điều chỉnh.

Giải thích delta âm

Delta âm ở Night Owl (−0.0046) và Break Skipper (−0.0382) **không phải lỗi hệ thống**. Delta âm có nghĩa là điểm composite giảm, nhưng điều này xảy ra **có chủ đích**: người dùng đánh đổi một số chỉ tiêu khách quan để lịch trình phù hợp hơn với sở thích chủ quan.

Ví dụ cụ thể: Break Skipper nói “tôi không cần nghỉ giữa các công việc” (thể hiện qua tín hiệu buffer thấp). Hệ thống giảm trọng số buffer, dẫn đến:

- BCR giảm (ít khoảng nghỉ ≥ 15 phút giữa các công việc)
- Composite giảm (BCR chiếm 10% trong công thức)
- Nhưng lịch trình giờ đây phản ánh *mong muốn thực sự* của người dùng

Đây chính là mục đích thiết kế của hệ thống thích ứng: **cá nhân hóa** thay vì áp đặt.

5.4 Mối đe dọa tính hợp lệ (Threats to Validity)

5.4.1 Tính hợp lệ nội tại (Internal Validity)

- **Dữ liệu tổng hợp**: Kịch bản được tạo tự động, không từ người dùng thực. Biện pháp giảm thiểu: phân bố loại công việc có trọng số (30% analytical, 25% admin, 20% creative, 15% learning, 10% physical) mô phỏng khối lượng công việc thực tế. Thời lượng (15–120 phút), độ ưu tiên (1–5), và tỷ lệ deadline (50%) cũng được thiết kế theo phân bố thực tế.
- **Trọng số composite cố định**: Công thức composite (30% ETA, 25% DSR, 15% TCS, 20% PWU, 10% BCR) là **định nghĩa chất lượng do tác giả đặt ra**, không phải sự thật khách quan. Biện pháp giảm thiểu: CHRONIX đạt điểm cao nhất hoặc gần cao nhất ở **từng chỉ tiêu riêng lẻ**, không chỉ composite. SFI và CSC được báo cáo riêng trong bảng kết quả nhưng không đưa vào composite để tránh thiên vị.

5.4.2 Tính hợp lệ ngoại tại (External Validity)

- **Chưa có người dùng thực:** Benchmark tổng hợp chứng minh ưu thế thuật toán nhưng chưa đánh giá chất lượng cảm nhận (perceived quality) của người dùng. Hướng tương lai: nghiên cứu người dùng theo chiều dọc (longitudinal user study) với ít nhất 30 người trong 2 tuần.
- **Một múi giờ:** Tất cả kịch bản sử dụng Asia/Bangkok (GMT+7) không có giờ mùa hè (DST - Daylight Saving Time). Thuật toán sử dụng chuyển đổi múi giờ đầy đủ (toZonedTime, fromZonedTime) nên kết quả **nhìn** tổng quát hóa được, nhưng chưa kiểm tra các trường hợp biên khi giờ mùa hè thay đổi.

5.4.3 Tính hợp lệ cấu trúc (Construct Validity)

- **Hồ sơ EMA giả lập:** Người dùng thực có mẫu hành vi phức tạp hơn nhiều so với 5 hồ sơ giả lập. Tín hiệu thực sự không ổn định theo thời gian (ví dụ: người dùng có thể thay đổi thói quen nghỉ ngơi). Mô phỏng chứng minh **tính chất toán học** của EMA (hội tụ, phản hồi, ổn định) chứ không dự đoán kết quả cụ thể cho từng người dùng.
- **Composite không phải thước đo duy nhất:** Điểm composite là hàm tuyến tính (linear combination), không thể phản ánh mọi khía cạnh chất lượng lập lịch. Ví dụ: sự hài lòng tâm lý (psychological satisfaction) của người dùng không được đo.

5.5 Tính tái tạo (Reproducibility)

Toàn bộ kết quả có tính tất định (deterministic) nhờ bộ sinh số ngẫu nhiên có hạt giống. Chạy lại benchmark trên bất kỳ máy nào với cùng mã nguồn sẽ cho kết quả giống hệt:

Đoạn mã 5.3: Chạy bộ kiểm chuẩn

```
1 npm run benchmark
2 # Kết quả:
3 # src/benchmark/results/algorithm-benchmark.csv
4 # src/benchmark/results/ema-convergence.csv
5 # src/benchmark/results/report.html
```

Mã nguồn benchmark nằm trong thư mục `src/benchmark/`, gồm 7 tệp TypeScript: `types.ts` (kiểu dữ liệu + bộ sinh số), `generate-scenarios.ts` (tạo kịch bản), `baselines.ts` (5 phương pháp), `metrics.ts` (8 chỉ tiêu), `ema-simulation.ts` (mô phỏng EMA), `run-benchmark.ts` (điểm bắt đầu), `report.ts` (xuất báo cáo HTML với biểu đồ).

Chương 6

Kết luận

6.1 Tóm tắt đóng góp

Luận văn đã trình bày CHRONIX, một ứng dụng lập lịch công việc thông minh dựa trên nhịp sinh học cá nhân (chronotype). Hệ thống giải quyết bài toán: làm thế nào để tự động xếp lịch công việc sao cho phù hợp với mức năng lượng nhận thức (cognitive energy) của từng người, đồng thời cân bằng nhiều yếu tố thực tế như deadline, độ ưu tiên, khoảng nghỉ, và tránh phân mảnh lịch trình. Các đóng góp chính bao gồm:

6.1.1 Hàm chi phí đa yếu tố (Multi-Factor Cost Function)

Thiết kế và triển khai hàm chi phí 6 yếu tố:

- Năng lượng sinh học** (chronotype penalty): Đối chiếu năng lượng có sẵn tại khe thời gian với yêu cầu năng lượng của loại công việc, sử dụng đường cong năng lượng 24 giờ riêng cho mỗi chronotype.
- Deadline**: Suy giảm tuyến tính (linear decay) theo khoảng cách đến hạn chót, kết hợp hệ số khẩn cấp theo độ ưu tiên.
- Chuyển đổi ngữ cảnh** (context switch): Phạt khi xếp hai công việc khác loại liên tiếp, giảm dần theo khoảng cách (dựa trên nghiên cứu 23 phút phục hồi (Mark et al., 2008)).
- Phân mảnh** (fragmentation): Phạt khi tạo khoảng trống nhỏ (< 30 phút) không đủ để làm việc có ý nghĩa.
- Độ ưu tiên** (priority): Ánh xạ tuyến tính từ thang 1–5, đảm bảo công việc quan trọng được ưu tiên slot tốt hơn.
- Khoảng nghỉ** (buffer): Khuyến khích khoảng nghỉ ≥ 15 phút giữa các công việc, dựa trên tỷ lệ làm việc:ngủ 52:17.

Kiểm chuẩn (benchmark) trên 6.000 lượt chạy với dữ liệu tổng hợp chứng minh phương pháp đa yếu tố vượt trội so với 4 phương pháp cơ bản (baseline):

- Vượt Random và First-Fit khoảng **+30%** điểm composite

- Vượt Priority-Only **+29%**
- Vượt Energy-Only **+14%**
- Thắng ở **tất cả 4 chronotype** (Lion, Bear, Wolf, Dolphin) với hiệu suất đồng đều (0.722–0.741)

6.1.2 Hệ thống trọng số thích ứng 3 lớp (3-Layer Adaptive Weight System)

Phát triển hệ thống trọng số 3 lớp cho phép cá nhân hóa hàm chi phí theo thời gian:

- **Lớp 1 - Hồ sơ mặc định** (Preset Profile): 5 bộ trọng số có sẵn (hustle, balanced, wellness, parent, creative), người dùng chọn khi onboarding.
- **Lớp 2 - Điều chỉnh thủ công** (Manual Override): Người dùng tinh chỉnh từng thành phần trong phạm vi $\pm 5\%$ qua thanh trượt (slider) trên giao diện.
- **Lớp 3 - Tự động điều chỉnh** (EMA Auto-Adjustment): Hệ thống thu thập 5 loại tín hiệu ngầm định (implicit feedback) từ hành vi người dùng và cập nhật trọng số trong phạm vi $\pm 15\%$ bằng thuật toán EMA ($\alpha = 0.3$).

Mô phỏng trên 5 hồ sơ hành vi giả lập (250 quan sát) xác nhận 3 tính chất:

- **Hội tụ** (convergence): Trọng số ổn định trong 10–15 quan sát, phù hợp phân tích lý thuyết $(0.7)^{10} = 0.028$.
- **Phản hồi đúng hướng** (responsiveness): Tín hiệu thấp cho thành phần X dẫn đến giảm trọng số X , và ngược lại.
- **Ổn định** (stability): Tín hiệu ngẫu nhiên không làm trọng số phân kỳ hay sụp đổ.

6.1.3 Ứng dụng full-stack hoàn chỉnh

Xây dựng ứng dụng web hoàn chỉnh bằng Next.js 15, TypeScript, PostgreSQL, và Drizzle ORM, bao gồm:

- **Khảo sát chronotype**: Bài khảo sát AutoMEQ rút gọn (5 câu hỏi) xác định 1 trong 4 chronotype, tích hợp vào quy trình giới thiệu (onboarding) 7 bước.
- **Đồng bộ Google Calendar**: 3 chế độ đồng bộ hai chiều (bidirectional sync) - toàn phần (initial), gia tăng (incremental) qua sync token, và đẩy lên (push) theo phương pháp local-first.
- **Trợ lý AI Chat**: Giao diện ngôn ngữ tự nhiên (natural language interface) hỗ trợ tiếng Anh và tiếng Việt, với 7 công cụ (function calling) kết nối trực tiếp đến scheduling engine qua OpenRouter.
- **Bảng điều khiển** (Dashboard): Lịch tuần/ngày/tháng bằng FullCalendar.js với lớp phủ năng lượng (energy overlay), bảng trọng số, và hộp thoại tinh chỉnh.
- **Bảo mật**: OAuth 2.0 qua Better Auth, cookie httpOnly, giới hạn tần suất (rate limiting), và bảo mật hàng dữ liệu (row-level security).

6.2 Hạn chế

1. **Chưa đánh giá với người dùng thực:** Kiểm chuẩn sử dụng dữ liệu tổng hợp (synthetic data) với bộ sinh số ngẫu nhiên có hạt giống. Mặc dù chứng minh được ưu thế thuật toán, kết quả chưa phản ánh chất lượng cảm nhận (perceived quality) và mức độ hài lòng thực tế của người dùng.
2. **Thuật toán tham lam (greedy):** Thuật toán chọn slot tốt nhất cho từng công việc một cách tuần tự, không xét tổ hợp tối ưu toàn cục. Ví dụ: xếp công việc A vào slot tốt nhất có thể khiến công việc B (quan trọng hơn) mất slot lý tưởng. Tuy nhiên, thuật toán tham lam có ưu điểm: thời gian chạy nhanh (phù hợp tương tác thời gian thực) và kết quả đủ tốt cho hầu hết trường hợp.
3. **Chỉ hỗ trợ Google Calendar:** Chưa tích hợp Microsoft Outlook, Apple Calendar, hoặc giao thức CalDAV. Người dùng sử dụng lịch khác không thể hưởng lợi từ đồng bộ hai chiều.
4. **Chưa có ứng dụng di động (mobile):** Giao diện web responsive hoạt động trên trình duyệt di động, nhưng chưa có ứng dụng gốc (native app) với thông báo (push notification) và tích hợp hệ điều hành.
5. **Đường cong năng lượng tĩnh:** 4 đường cong năng lượng (Lion, Bear, Wolf, Dolphin) được thiết kế dựa trên nghiên cứu chronobiology chung (Breus, 2016). Mỗi người thực tế có đường cong khác nhau, bị ảnh hưởng bởi giấc ngủ, sức khỏe, và thói quen hàng ngày. Hệ thống hiện tại chưa có khả năng học đường cong năng lượng cá nhân hóa.
6. **Một múi giờ trong kiểm chuẩn:** Tất cả kịch bản benchmark sử dụng Asia/Bangkok (GMT+7) không có giờ mùa hè (DST - Daylight Saving Time). Mã nguồn sử dụng chuyển đổi múi giờ đầy đủ nhưng chưa kiểm tra các trường hợp biên khi DST thay đổi.

6.3 Hướng phát triển tương lai

6.3.1 Ngắn hạn (3–6 tháng)

1. **Nghiên cứu người dùng (user study):** Thực hiện nghiên cứu theo chiều dọc (longitudinal study) với 30–50 người dùng trong 2–4 tuần. Đo lường: tỷ lệ hoàn thành công việc đúng hạn, mức hài lòng (khảo sát Likert 5 điểm), tỷ lệ giữ chân (retention rate) sau 30 ngày, và so sánh với lịch trình tự xếp thủ công.
2. **Đa nền tảng lịch:** Tích hợp Microsoft Outlook (Microsoft Graph API) và Apple Calendar (giao thức CalDAV). Thiết kế lớp trừu tượng (abstraction layer) cho phép thêm nhà cung cấp lịch mới mà không thay đổi lõi scheduling engine.
3. **Ứng dụng di động:** Phát triển ứng dụng React Native cho iOS và Android, tận dụng lại scheduling engine (TypeScript thuần). Tính năng ưu tiên: thông báo đẩy nhắc nhở công việc sắp tới, widget hiển thị lịch trình ngày, và đánh giá nhanh sau hoàn thành.

6.3.2 Trung hạn (6–12 tháng)

1. **Đường cong năng lượng cá nhân hóa:** Kết nối với thiết bị đeo tay (smartwatch, fitness tracker) qua API (Apple HealthKit, Google Health Connect) để thu thập nhịp tim, chất lượng giấc ngủ, và mức hoạt động. Sử dụng dữ liệu thực để hiệu chỉnh đường cong năng lượng theo từng cá nhân thay vì dùng đường cong tĩnh theo chronotype.
2. **Mô hình học máy nâng cao:** Thay thế hoặc bổ sung EMA bằng mô hình phức tạp hơn (ví dụ: XGBoost, mạng nơ-ron đơn giản) để học các mẫu hành vi (behavioral pattern) phi tuyến tính. Ví dụ: phát hiện người dùng ưu tiên deadline vào đầu tuần nhưng ưu tiên năng lượng vào cuối tuần.
3. **Lập lịch nhóm** (team scheduling): Mở rộng hệ thống để hỗ trợ lập lịch cho nhóm, xét đến chronotype của nhiều thành viên khi đề xuất thời gian họp. Ví dụ: tìm khe thời gian mà tất cả thành viên đều có mức năng lượng chấp nhận được.

6.3.3 Dài hạn (12+ tháng)

1. **Khung thử nghiệm A/B** (A/B testing framework): Xây dựng hệ thống để so sánh các cấu hình trọng số và chiến lược lập lịch khác nhau trên người dùng thực. Thu thập dữ liệu dài hạn và năng suất để liên tục cải thiện thuật toán.
2. **Tích hợp lịch trình học tập:** Mở rộng ứng dụng cho sinh viên, tích hợp với hệ thống quản lý học tập (LMS - Learning Management System) để tự động nhập lịch thi, bài tập, và đề xuất lịch ôn tập dựa trên chronotype và khoảng cách đến ngày thi.

6.4 Kết luận

CHRONIX chứng minh rằng việc kết hợp kiến thức chronobiology (nhịp sinh học) với hàm chi phí đa yếu tố và hệ thống trọng số thích ứng có thể tạo ra lịch trình công việc chất lượng cao hơn đáng kể so với các phương pháp truyền thống. Với điểm tổng hợp (composite score) 0.7312, vượt trội 14–30% so với các phương pháp cơ bản, và hệ thống EMA hội tụ trong 10–15 quan sát, đề tài cung cấp nền tảng vững chắc cho nghiên cứu và phát triển tiếp theo trong lĩnh vực lập lịch cá nhân hóa dựa trên nhịp sinh học.

Thuật toán lập lịch của CHRONIX là **100% mã nguồn TypeScript tự phát triển**, không phụ thuộc vào dịch vụ AI bên ngoài cho logic lõi. AI Chat chỉ đóng vai trò giao diện ngôn ngữ tự nhiên, mọi quyết định lập lịch đều do scheduling engine đưa ra dựa trên hàm chi phí và trọng số đã thiết kế. Điều này đảm bảo tính minh bạch (mọi quyết định đều giải thích được), khả năng tái tạo (cùng đầu vào luôn cho cùng kết quả), và không phụ thuộc vào chi phí API của bên thứ ba cho chức năng cốt lõi.

TÀI LIỆU THAM KHẢO

- Adan, A., Archer, S. N., Hidalgo, M. P., Di Milia, L., Natale, V., và Randler, C. (2012). Circadian typology: A comprehensive review. *Chronobiology International*, 29(9):1153–1175. <https://doi.org/10.3109/07420528.2012.719971>.
- Breus, M. (2016). *The Power of When: Discover Your Chronotype—and the Best Time to Eat Lunch, Ask for a Raise, Have Sex, Write a Novel, Take Your Meds, and More*. Little, Brown Spark. <https://archive.org/details/powerofwhendisco0000breu>.
- Facer-Childs, E. R., Campos, B. M., Middleton, B., Skene, D. J., và Bagshaw, A. P. (2019). Circadian phenotype impacts the brain's resting-state functional connectivity, attentional performance, and sleepiness. *Sleep*, 42(5). <https://doi.org/10.1093/sleep/zsz033>.
- Horne, J. A. và Ostberg, O. (1976). A self-assessment questionnaire to determine morningness-eveningness in human circadian rhythms. *International Journal of Chronobiology*, 4(2):97–110. <https://pubmed.ncbi.nlm.nih.gov/1027738/>.
- Koren, Y., Bell, R., và Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37. <https://doi.org/10.1109/MC.2009.263>.
- Mark, G., Gudith, D., và Klocke, U. (2008). The cost of interrupted work: More speed and stress. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 107–110. ACM. <https://doi.org/10.1145/1357054.1357072>.
- Pinedo, M. L. (2016). *Scheduling: Theory, Algorithms, and Systems*. Springer, 5th edition. <https://doi.org/10.1007/978-3-319-26580-3>.
- Press, W. H., Teukolsky, S. A., Vetterling, W. T., và Flannery, B. P. (2007). *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, 3rd edition. <https://numerical.recipes>.
- Roenneberg, T. (2012). *Internal Time: Chronotypes, Social Jet Lag, and Why You're So Tired*. Harvard University Press. <https://doi.org/10.4159/harvard.9780674065482>.
- Sweller, J., Ayres, P., và Kalyuga, S. (2011). *Cognitive Load Theory*. Springer. <https://doi.org/10.1007/978-1-4419-8126-4>.
- Wittmann, M., Dinich, J., Mellow, M., và Roenneberg, T. (2006). Social jetlag: misalignment of biological and social time. *Chronobiology International*, 23(1-2):497–509. <https://doi.org/10.1080/07420520500545979>.

Phụ lục A

Bài khảo sát chronotype trong CHRONIX

Bài khảo sát MEQ gốc (Morningness-Eveningness Questionnaire) (Horne và Ostberg, 1976) có 19 câu hỏi. CHRONIX sử dụng phiên bản rút gọn gồm **5 câu hỏi** được chọn lọc để xác định chronotype nhanh chóng mà vẫn đảm bảo độ phân biệt giữa 4 nhóm. Dưới đây là 5 câu hỏi đúng như trong mã nguồn (`src/features/onboarding/lib/chronotype.ts`).

A.1 5 câu hỏi khảo sát

1. If you had complete freedom, what time would you naturally wake up?

(Nếu hoàn toàn tự do, bạn sẽ thức dậy tự nhiên lúc mấy giờ?)

- a) 5:00 – 6:30 AM (5 điểm)
- b) 6:30 – 7:45 AM (4 điểm)
- c) 7:45 – 9:45 AM (3 điểm)
- d) 9:45 – 11:00 AM (2 điểm)
- e) 11:00 AM – 12:00 PM (1 điểm)

2. How alert do you feel during the first 30 minutes after waking?

(Bạn cảm thấy tỉnh táo đến mức nào trong 30 phút đầu sau khi thức dậy?)

- a) Very alert – Rất tỉnh táo (4 điểm)
- b) Fairly alert – Khá tỉnh táo (3 điểm)
- c) Fairly tired – Khá mệt (2 điểm)
- d) Very tired – Rất mệt (1 điểm)

3. When do you feel your best for demanding mental work?

(Khi nào bạn cảm thấy tốt nhất cho công việc trí óc đòi hỏi cao?)

- a) 8 – 10 AM (5 điểm)
- b) 10 AM – 12 PM (4 điểm)
- c) 12 – 2 PM (3 điểm)
- d) 2 – 5 PM (2 điểm)

e) 5 – 9 PM (1 điểm)

4. When do you start feeling tired in the evening?

(Bạn bắt đầu cảm thấy mệt vào buổi tối lúc mấy giờ?)

a) 8 – 9 PM (5 điểm)

b) 9 – 10:15 PM (4 điểm)

c) 10:15 PM – 12:30 AM (3 điểm)

d) 12:30 – 2:00 AM (2 điểm)

e) 2:00 AM or later – 2:00 sáng trở đi (1 điểm)

5. How would you describe your sleep patterns?

(Bạn mô tả giấc ngủ của mình như thế nào?)

a) Consistent, fall asleep easily – Điều đặn, dễ ngủ (4 điểm)

b) Generally good, occasional issues – Khá tốt, đôi khi có vấn đề (3 điểm)

c) Variable, often take time to fall asleep – Không ổn định, thường mất thời gian để ngủ (2 điểm)

d) Irregular, frequently disrupted – Bất thường, thường xuyên bị gián đoạn (1 điểm)

A.2 Thuật toán phân loại

Điểm tối đa: 23 (5 + 4 + 5 + 5 + 4). Điểm tối thiểu: 5 (1 + 1 + 1 + 1 + 1).

Thuật toán phân loại kiểm tra **câu 5 (chất lượng giấc ngủ) trước**, vì đây là dấu hiệu đặc trưng nhất của chronotype Dolphin:

1. Nếu câu 5 là 1 điểm (giấc ngủ bất thường, thường xuyên bị gián đoạn) → **Dolphin**
2. Nếu tổng điểm ≥ 18 → **Lion**
3. Nếu tổng điểm ≥ 13 → **Bear**
4. Còn lại (tổng < 13) → **Wolf**

Bảng A.1: Tóm tắt phân loại chronotype

Chronotype	Điều kiện	Đỉnh năng lượng	Suy giảm
Dolphin	1 điểm câu 5	10:00 – 12:00	14:00 – 16:00
Lion	Tổng ≥ 18	6:00 – 10:00	13:00 – 15:00
Bear	Tổng ≥ 13	10:00 – 14:00	14:00 – 16:00
Wolf	Tổng < 13	17:00 – 21:00	8:00 – 10:00

Ghi chú: Nếu người dùng bỏ qua bài khảo sát (nút “Skip”), hệ thống sử dụng bộ câu trả lời mặc định cho kết quả Bear - chronotype phổ biến nhất (khoảng 55% dân số).